

FPGA 与 VHDL 期末考试
——20 套模拟试卷

2026 年 6 月 16 日

目录

第一章 第 1 套试卷	1
第二章 第 2 套试卷	9
第三章 第 3 套试卷	15
第四章 第 4 套试卷	23
第五章 第 5 套试卷	29
第六章 第 6 套试卷	35
第七章 第 7 套试卷	43
第八章 第 8 套试卷	51
第九章 第 9 套试卷	65
第十章 第 10 套试卷	77
第十一章 第 11 套试卷	87
第十二章 第 12 套试卷	95
第十三章 第 13 套试卷	109
第十四章 第 14 套试卷	117
第十五章 第 15 套试卷	125
第十六章 第 16 套试卷	133
第十七章 第 17 套试卷	147

第十八章 第 18 套试卷	163
第十九章 第 19 套试卷	181
第二十章 第 20 套试卷	201

第一章 第 1 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列哪种器件属于简单可编程逻辑器件（SPLD）？
 - A. FPGA
 - B. CPLD
 - C. GAL
 - D. ASIC
2. FPGA 芯片中实现组合逻辑功能的基本单元是？
 - A. 触发器（Flip-Flop）
 - B. 查找表（LUT）
 - C. 乘法器（Multiplier）
 - D. 锁相环（PLL）
3. 在 VHDL 中，以下哪个关键字用于定义实体的端口？
 - A. ARCHITECTURE
 - B. PORT
 - C. SIGNAL
 - D. COMPONENT
4. CPLD 与 FPGA 相比，以下说法正确的是？
 - A. CPLD 的逻辑单元密度通常高于 FPGA
 - B. CPLD 基于查找表结构，FPGA 基于乘积项结构
 - C. CPLD 的互连结构通常是连续式的，延时可预测
 - D. CPLD 更适合实现大规模时序电路

5. 在 VHDL 的 PROCESS 语句中，以下关于信号 (SIGNAL) 与变量 (VARIABLE) 的描述，错误的是？
- A. 信号赋值使用 “<=”，变量赋值使用 “:=”
 - B. 变量只能在 PROCESS 内部定义和使用
 - C. 信号赋值立即生效，变量赋值在进程结束时生效
 - D. 信号用于进程间通信，变量用于进程内部计算
6. 一个 4 输入 LUT 可以实现任意最多几个变量的组合逻辑函数？
- A. 2 个
 - B. 3 个
 - C. 4 个
 - D. 8 个

二、填空题（每空 3 分，共 12 分）

1. VHDL 中，标准逻辑库的名称是_____，其中最常用的数据类型是_____。
2. 在 VHDL 中，并发信号赋值语句的关键字是_____，进程的敏感信号表使用的关键字是_____（填英文，大小写均可）。
3. FPGA 器件主要由可编程逻辑块 (CLB)、_____ 和 _____ 三部分组成（写出两种主要资源的名称即可）。
4. VHDL 的关系运算符中，“不等于”写作_____，“大于等于”写作_____。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码，找出其中的 5 处错误，并写出正确的修改方案。

Listing 1.1: 存在错误的 VHDL 代码

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTIFY mux2to1 IS
5     PORT(
6         a, b : IN  STD_LOGIG;
7         sel  : IN  STD_LOGIC;
8         y    : OUT STD_LOGIC
9     );
10 END mux2to1;
11
12 ARCHITE_CTURE behavioral OF mux2to1 IS
13 BEGIN
14     PROCESS(a, b, sel)
15     BEGIN
16         IF sel = '0' THEN
17             y := a;
18         ELSE
19             y := b;
20         END IF;
21     END PROCESS;
22 END behavioral;
```

答题要求：逐行列出错误位置、错误原因及正确写法（每处 1 分，共 5 分）。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 1.2: 分析用 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY decoder38 IS
5      PORT(
6          a  : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
7          en : IN  STD_LOGIC;
8          y  : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
9      );
10 END decoder38;
11
12 ARCHITECTURE behavioral OF decoder38 IS
13 BEGIN
14     PROCESS(a, en)
15     BEGIN
16         IF en = '0' THEN
17             y <= (OTHERS => '1');
18         ELSE
19             CASE a IS
20                 WHEN "000" => y <= "11111110";
21                 WHEN "001" => y <= "11111101";
22                 WHEN "010" => y <= "11111011";
23                 WHEN "011" => y <= "11110111";
24                 WHEN "100" => y <= "11101111";
25                 WHEN "101" => y <= "11011111";
26                 WHEN "110" => y <= "10111111";
27                 WHEN "111" => y <= "01111111";
28                 WHEN OTHERS => y <= (OTHERS => '1');
29             END CASE;
30         END IF;
31     END PROCESS;
32 END behavioral;
```


问题：

- (1) 该 VHDL 代码描述的是什么电路？（1 分）
- (2) 该电路的输入输出端口各有多少位？功能是什么？（2 分）
- (3) 信号 en 的作用是什么？当 en= ‘0’ 时输出 y 的值是什么？（2 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计（20 分）

设计一个 4 位比较器。输入为两个 4 位二进制数 A 和 B，输出为三个标志信号：GT（大于）、EQ（等于）、LT（小于）。当 $A > B$ 时 $GT = '1'$ ；当 $A = B$ 时 $EQ = '1'$ ；当 $A < B$ 时 $LT = '1'$ 。要求：

- (1) 画出顶层模块的端口框图，标注所有端口名称和位宽。（5 分）
- (2) 写出完整的 VHDL 代码，包含库声明、实体定义和结构体。（15 分）

提示：可以使用 IF-ELSE 语句或条件信号赋值语句实现比较逻辑。

设计题 2：时序逻辑设计（20 分）

设计一个模 10 同步计数器（计数范围 0-9，计到 9 后回 0）。要求：

- (1) 端口要求：时钟 clk（输入）、异步复位 rst（输入，低电平有效）、使能 en（输入）、4 位计数输出 q（输出）。（5 分）
- (2) 画出状态转移图。（5 分）
- (3) 写出完整的可综合 VHDL 代码，复位时输出清零。（10 分）

提示：异步复位指复位信号不依赖于时钟边沿，直接生效。

设计题 3：状态机设计（20 分）

设计一个 Moore 型状态机，实现“101”序列检测器。输入信号为 din（1 位串行输入），输出信号为 found（1 位），当检测到连续输入“101”序列时 found 输出‘1’（高电平有效），其余时间输出‘0’。要求：

- (1) 画出状态转移图，标注每个状态的名称、含义及输出值。（8 分）
- (2) 写出完整的 VHDL 代码，采用双进程（或三进程）描述方式。时钟 clk 上升沿触发，复位 rst 为同步复位（高电平有效）。（12 分）

提示：Moore 型状态机的输出只与当前状态有关，与输入无关。建议定义 4 个状态：S0（初始/未匹配）、S1（已匹配“1”）、S2（已匹配“10”）、S3（已匹配“101”）。

第二章 第 2 套试卷

2.1 一、选择题（每题 3 分，共 18 分）

1. 以下关于 GAL（Generic Array Logic）器件的描述，错误的是（ ）
 - (A) GAL 的与阵列是可编程的，或阵列是固定的
 - (B) GAL 的输出逻辑宏单元（OLMC）可配置为组合输出或寄存器输出
 - (C) GAL 采用的是 EEPROM 工艺，可重复编程
 - (D) GAL 的与阵列和或阵列均是可编程的
2. FPGA 中用于实现组合逻辑函数的基本可编程单元是（ ）
 - (A) 可编程连线（PIP）
 - (B) 查找表（LUT）
 - (C) 输入输出块（IOB）
 - (D) Block RAM
3. 以下 VHDL 实体声明中，语法正确的是（ ）
 - (A) ENTITY adder IS PORT (a,b: IN BIT; s: OUT BIT); END adder;
 - (B) ENTITY adder IS PORT (a,b: IN BIT; s: OUT BIT) END adder;
 - (C) ENTITY adder PORT (a,b: IN BIT; s: OUT BIT); END ENTITY adder;
 - (D) ENTITY adder IS PORT (a,b: IN BIT, s: OUT BIT); END adder;
4. 在 VHDL 的 PROCESS 语句中，关于信号（SIGNAL）赋值和变量（VARIABLE）赋值的描述，正确的是（ ）
 - (A) 信号赋值和变量赋值都是立即生效的
 - (B) 信号赋值在进程结束时生效，变量赋值立即生效

- (C) 信号赋值立即生效, 变量赋值在进程结束时生效
(D) 信号赋值和变量赋值都在进程结束时生效
5. CPLD 与 FPGA 在结构上的主要区别是 ()
- (A) CPLD 基于查找表结构, FPGA 基于乘积项结构
(B) CPLD 基于乘积项结构, FPGA 基于查找表结构
(C) 两者均基于相同的可编程结构
(D) CPLD 不能实现时序逻辑, FPGA 可以
6. 一个 4 输入 LUT 可以实现任意 () 变量的组合逻辑函数
- (A) 2 个及以下
(B) 3 个及以下
(C) 4 个及以下
(D) 8 个及以下

2.2 二、填空题 (每题 3 分, 共 12 分)

1. 在 VHDL 中, 使用 IEEE 标准库中的标准逻辑类型时, 需在代码开头添加的库声明语句为: `LIBRARY ____; USE IEEE.STD_LOGIC_1164.ALL;`。
2. VHDL 中, `STD_LOGIC_VECTOR` 是一种____数据类型, 它表示一组标准逻辑信号的集合。(填“标量”或“复合”)
3. 在 VHDL 中, 拼接运算符的符号是____。
4. 一个 VHDL 设计的结构体 (ARCHITECTURE) 内部可以包含多条____语句, 用于描述多个并发的硬件模块。(填关键字)

2.3 三、程序改错题 (5 分)

下面是一个 3 线-8 线译码器的 VHDL 代码, 其中包含 5 处错误。请指出错误所在行 (可用行号标注) 并写出正确的代码。

```
1 LIBRARY ieee;  
2 USE ieee.std_logic_1164.ALL;  
3
```

```
4 ENTITY decoder3t8 IS
5   PORT (
6     sel : IN  INTEGER RANGE 0 TO 7;
7     en  : IN  std_logic;
8     y   : OUT STD_LOGIC_VECTOR(7 DOWNT0 1)
9   );
10 END decoder3t8;
11
12 ARCHITECTURE behav OF decoder3t8 IS
13 BEGIN
14   PROCESS (sel)
15   BEGIN
16     y <= (OTHERS => '0');
17     IF en = '1' THEN
18       CASE sel IS
19         WHEN 0 => y(0) <= '1';
20         WHEN 1 => y(1) <= '1';
21         WHEN 2 => y(2) <= '1';
22         WHEN 3 => y(3) <= '1';
23         WHEN 4 => y(4) <= '1';
24         WHEN 5 => y(5) <= '1';
25         WHEN 6 => y(6) <= '1';
26         WHEN 7 => y(7) <= '1';
27       END CASE
28     END IF
29   END PROCESS
30 END behav;
```

2.4 四、程序阅读分析题（5 分）

阅读以下 VHDL 代码，回答后面的问题。

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY mux4to1 IS
5   PORT (
6     a, b, c, d : IN  STD_LOGIC;
7     sel        : IN  STD_LOGIC_VECTOR(1 DOWNT0 0);
```

```
8      y          : OUT STD_LOGIC
9      );
10 END mux4to1;
11
12 ARCHITECTURE behav OF mux4to1 IS
13 BEGIN
14     PROCESS (a, b, c, d, sel)
15     BEGIN
16         CASE sel IS
17             WHEN "00"    => y <= a;
18             WHEN "01"    => y <= b;
19             WHEN "10"    => y <= c;
20             WHEN "11"    => y <= d;
21             WHEN OTHERS => y <= '0';
22         END CASE;
23     END PROCESS;
24 END behav;
```

问题:

- (1) 该代码描述的是一个什么电路? (1 分)
- (2) 该电路的输入信号有哪些? 输出信号有哪些? (1 分)
- (3) 当 `sel = "10"` 时, 输出 `y` 的值由哪个输入信号决定? (1 分)
- (4) 如果在 Vivado 等综合工具中综合该代码, `sel` 被综合成什么物理资源? (1 分)
- (5) 该电路的行为描述使用了哪种 VHDL 语句? 它与 IF 语句在使用场景上的主要区别是什么? (1 分)

2.5 五、综合设计题 (共 60 分)

2.5.1 设计题 1: 组合逻辑设计——8 位数值比较器 (20 分)

请使用 VHDL 语言设计一个 8 位数值比较器, 要求如下:

- (1) 端口定义 (4 分): 定义实体端口, 包括两个 8 位输入数据端口 `a`、`b` (类型为 `STD_LOGIC_VECTOR(7 DOWNTO 0)`), 以及三个输出端口 `agtb` (`a` 大于 `b`)、`aeqb` (`a` 等于 `b`)、`altb` (`a` 小于 `b`)。

- (2) **功能实现 (10 分)**: 在结构体中用行为描述方式 (使用 IF 语句或条件信号赋值语句) 实现比较功能。
- (3) **设计思路说明 (3 分)**: 用注释简要说明设计思路。
- (4) **代码规范 (3 分)**: 代码缩进规范、信号命名合理、注释清晰。

2.5.2 设计题 2: 时序逻辑设计——模 6 计数器 (20 分)

请使用 VHDL 语言设计一个模 6 计数器 (计数范围: 0~5), 要求如下:

- (1) **端口定义 (4 分)**: 端口包括时钟信号 `clk`、异步复位信号 `rst` (高电平复位)、计数使能信号 `en`, 以及 4 位计数输出 `q` (类型为 `STD_LOGIC_VECTOR(3 DOWNTO 0)`) 和进位输出 `cout` (计数值为 5 且 `en` 有效时输出高电平)。
- (2) **功能实现 (10 分)**: 在进程 (PROCESS) 中使用 IF 语句描述时序行为: 异步复位优先级最高, 复位时 `q <= "0000"`; 在时钟上升沿, 若 `en = '1'` 则进行加 1 计数, 计数值从 0~5 循环。
- (3) **设计思路说明 (3 分)**: 用注释说明模 6 计数器的工作流程。
- (4) **代码规范 (3 分)**: 时钟边沿检测语法正确、信号类型使用恰当。

2.5.3 设计题 3: 状态机设计——“101”序列检测器 (Mealy 型) (20 分)

请使用 VHDL 语言设计一个 Mealy 型状态机, 用于检测串行输入序列 “101”。要求如下:

- (1) **功能说明 (2 分)**: 当输入端 `din` 连续收到 “101” 序列时 (每个时钟周期输入 1 位), 输出端 `dout` 在检测到完整序列的同一时钟周期输出高电平 “1”, 其余情况输出 “0”。允许序列重叠检测 (即 “10101” 中应能检测出两次 “101”)。
- (2) **状态图绘制 (4 分)**: 画出 Mealy 型状态机的状态转换图 (可手绘后拍照贴入答题区, 或用文字描述各状态及转换条件)。标注状态名称、转换条件及输出值。
- (3) **端口定义 (3 分)**: 端口包括时钟 `clk`、异步复位 `rst` (低电平复位)、串行输入 `din`、检测输出 `dout`。
- (4) **VHDL 代码 (8 分)**: 使用双进程 (或三进程) 结构编写完整的 Mealy 型状态机代码。
- (5) **设计思路说明与代码规范 (3 分)**: 代码结构清晰, 状态编码合理, 有适当注释。

第三章 第 3 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列关于 CPLD 和 FPGA 的说法中，错误的是（ ）。
 - A. CPLD 基于乘积项（Product Term）结构，适合实现复杂的组合逻辑
 - B. FPGA 基于查找表（LUT）结构，可重复编程
 - C. FPGA 的基本逻辑单元是可编程逻辑块（CLB），包含 LUT 和触发器
 - D. CPLD 的延时不可预测，不适合时序要求严格的电路
2. FPGA 中查找表（LUT）的核心功能是（ ）。
 - A. 对输入信号进行放大
 - B. 实现任意组合逻辑函数
 - C. 产生时钟信号
 - D. 存储程序指令
3. 在 VHDL 中，ENTITY 声明部分用于描述（ ）。
 - A. 电路的内部实现细节
 - B. 电路的对外接口（端口）
 - C. 电路的仿真激励
 - D. 电路的时序约束
4. 下列关于 VHDL 中信号（Signal）和变量（Variable）的描述，正确的是（ ）。

- A. 信号和变量都可以在 PROCESS 外部声明
 - B. 信号的赋值立即生效，变量的赋值在进程结束时生效
 - C. 变量只能在 PROCESS 或子程序内部声明，赋值立即生效
 - D. 信号只能在 PROCESS 内部使用
5. 在 VHDL 的 PROCESS 语句中，敏感信号表（Sensitivity List）的作用是（ ）。
- A. 定义进程的优先级
 - B. 指定触发进程执行的信号
 - C. 限制进程的执行次数
 - D. 定义进程的局部变量
6. VHDL 中的 STD_LOGIC 数据类型定义于下列哪个库和包中？（ ）
- A. LIBRARY std; USE std.standard.ALL
 - B. LIBRARY work; USE work.types.ALL
 - C. LIBRARY IEEE; USE IEEE.STD_LOGIC_1164.ALL
 - D. LIBRARY IEEE; USE IEEE.NUMERIC_STD.ALL

二、填空题（每题 3 分，共 12 分）

1. VHDL 中,一个完整的设计单元由 _____(声明接口)和 _____(描述功能)两大部分组成。
2. VHDL 中,信号赋值使用符号 _____,变量赋值使用符号 _____;并置(拼接)运算符是 _____。
3. IEEE 标准库中,定义了 STD_LOGIC 和 STD_LOGIC_VECTOR 类型的包名为 _____定义了 UNSIGNED 和 SIGNED 类型的包名为 _____。
4. 在 VHDL 中,可用的端口模式有 IN、OUT、_____ (双向)和 _____ (缓冲输出,可内部回读)。

三、程序改错题（共 5 分）

下面的 VHDL 程序描述的是一个 4 选 1 多路选择器，但程序中存在 6 处错误。请指出每处错误的位置（行号）并给出正确的写法。

```
1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3
4 ENTITY mux4to1 IS
5     PORT(
6         d : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
7         s : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
8         y : OUT STD_LOGIC
9     )
10 END mux4to1;
11
12 ARCHITECTURE behav OF mux4to1 IS
13 BEGIN
14     PROCESS(d, s)
15     BEGIN
16         CASE s IS
17             WHEN "00" => y <= d(0);
18             WHEN "01" => y <= d(1);
19             WHEN "10" => y <= d(2)
20             WHEN "11" => y <= d(3);
21             WHEN OTHERS => y = '0';
22         END CASE
23     END PROCESS;
24 END behavior;
```

要求：在答题纸上按以下格式依次写出各错误：

错误 1（第 ____ 行）：	_____	改为 _____
错误 2（第 ____ 行）：	_____	改为 _____
错误 3（第 ____ 行）：	_____	改为 _____
错误 4（第 ____ 行）：	_____	改为 _____
错误 5（第 ____ 行）：	_____	改为 _____
错误 6（第 ____ 行）：	_____	改为 _____

四、程序阅读分析题（共 5 分）

阅读下面的 VHDL 程序，回答问题。

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY dff_arst IS
5      PORT(
6          clk    : IN    STD_LOGIC;
7          d      : IN    STD_LOGIC;
8          rst    : IN    STD_LOGIC;
9          q      : OUT   STD_LOGIC
10     );
11 END dff_arst;
12
13 ARCHITECTURE behav OF dff_arst IS
14 BEGIN
15     PROCESS(clk, rst)
16     BEGIN
17         IF rst = '1' THEN
18             q <= '0';
19         ELSIF rising_edge(clk) THEN
20             q <= d;
21         END IF;
22     END PROCESS;
23 END behav;
```

- (1) (2 分) 该程序描述的是什么电路？它的基本功能是什么？
- (2) (2 分) 请说明信号 `clk`、`d`、`rst`、`q` 各自的作用。该电路是同步复位还是异步复位？为什么？
- (3) (1 分) 如果将第 19 行的 `rising_edge(clk)` 改为 `clk'event AND clk = '1'`，功能是否相同？请简要说明。

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——8 线-3 线优先编码器（20 分）

设计一个 8 线-3 线优先编码器，具体要求如下：

- 输入信号：din(7 downto 0)，低电平有效（0 表示有请求），优先级 din(7) 最高，din(0) 最低。
- 输出信号：dout(2 downto 0)，输出优先级最高的有效输入编号（二进制编码）。
- 额外输出：gs（Group Signal），当任意输入有效时 gs = '1'，否则 gs = '0'。

要求：

- (1)（8 分）列出该优先编码器的真值表（可适当简化，至少包括关键输入组合）。
- (2)（3 分）写出 dout(2)、dout(1)、dout(0) 和 gs 的逻辑表达式。
- (3)（9 分）用 VHDL 编写完整的 ENTITY 和 ARCHITECTURE 代码，使用条件信号赋值语句（WITH-SELECT-WHEN 或 WHEN-ELSE）实现。

设计题 2：时序逻辑设计——模 10 计数器（20 分）

设计一个模 10 计数器（BCD 码计数器），计数范围 0~9，具体要求如下：

- 输入信号：clk（时钟）、rst（复位，高电平有效，同步复位）。
- 输出信号：q(3 downto 0)（计数值，BCD 码输出）、tc（进位输出，当计数值为 9 且下一时钟变为 0 时产生一个时钟周期的高电平脉冲）。

要求：

- (1)（4 分）画出该计数器的状态转换图（状态图），标注状态值和转换条件。
- (2)（3 分）说明 tc 信号的产生逻辑。
- (3)（13 分）用 VHDL 编写完整的 ENTITY 和 ARCHITECTURE 代码，使用 PROCESS 实现。要求：包含一个内部信号 count 用于保存当前计数值，在 PROCESS 中描述计数逻辑。

设计题 3：状态机设计——序列检测器（20 分）

设计一个 Moore 型有限状态机，用于检测串行输入序列“101”（重叠检测），具体要求如下：

- 输入信号：clk（时钟）、rst（复位，高电平有效，异步复位）、x（串行数据输入，1 位）。
- 输出信号：z（检测结果输出，1 位），当检测到“101”序列时 $z = '1'$ ，否则 $z = '0'$ 。
- 重叠检测的含义：输入序列“10101”应检测出两次“101”。

要求：

- (1)（6 分）画出该 Moore 型状态机的状态转换图，标注状态名、状态编码、转换条件和各状态的输出值。需明确说明状态编码方案。
- (2)（4 分）列出状态转换表，包含当前状态、输入、次态和输出。
- (3)（10 分）用 VHDL 编写完整的 ENTITY 和 ARCHITECTURE 代码。要求：
 - 使用用户自定义枚举类型（如 `TYPE state_type IS (S0, S1, S2, S3)`）定义状态；
 - 采用双进程（或三进程）描述风格：一个时序进程描述状态寄存器，一个组合进程描述次态逻辑和输出逻辑；
 - 必须包含异步复位逻辑。

第四章 第 4 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列哪种可编程逻辑器件属于与阵列可编程、或阵列固定的结构？
 - A. CPLD
 - B. FPGA
 - C. PAL
 - D. GAL
2. 在 VHDL 中，ENTITY（实体）的主要作用是？
 - A. 描述电路的内部逻辑功能
 - B. 定义电路的输入输出端口
 - C. 声明电路中使用的信号
 - D. 指定仿真激励信号
3. FPGA 中查找表（LUT）的核心组成部分是？
 - A. 触发器和锁存器
 - B. 多路选择器和与或阵列
 - C. SRAM 存储单元和译码器
 - D. 寄存器和加法器
4. 在 VHDL 进程中，下列关于信号（SIGNAL）的赋值语句描述正确的是？
 - A. 信号赋值立即生效，类似于变量的行为

- B. 信号赋值在当前进程结束时才更新，进程内读到的是旧值
 - C. 信号赋值不受进程控制，随时更新
 - D. 信号只能在进程外部进行赋值
5. 某 LUT 有 4 个输入信号，请问该 LUT 能实现的任意逻辑函数的最大输入变量数为？
- A. 2
 - B. 4
 - C. 8
 - D. 16
6. 一个典型的 FPGA 逻辑单元（Logic Element）通常包含？
- A. 一个查找表和一个触发器
 - B. 一个乘法器和一个 RAM 块
 - C. 一个微处理器核
 - D. 一个模拟比较器

二、填空题（每题 3 分，共 12 分）

1. VHDL 中，用于定义电路内部互连信号的关键字是_____，用于定义常量的关键字是_____。
2. 在 IEEE 库中，定义标准逻辑类型 STD_LOGIC 和 STD_LOGIC_VECTOR 的程序包名称是_____。
3. VHDL 中，并行信号赋值语句的常用形式有三种：简单信号赋值语句、条件信号赋值语句（_____）和选择信号赋值语句（_____）。
4. CPLD（_____ Programmable Logic Device）的英文全称是 Complex Programmable Logic Device，其内部主要由_____ 阵列和宏单元组成。

三、程序改错题（5 分）

题目要求：以下 VHDL 代码描述的是一个 4 选 1 多路选择器（MUX），但代码中存在 6 处错误。请找出所有错误并说明改正方法。（找出 5 处即得满分）

Listing 4.1: 4 选 1 多路选择器（含 6 处错误）

```
1 library ieee
2 use ieee.std_logic_1164.all
3
4 entity mux4to1 is
5     port (
6         d0, d1, d2, d3 : in  std_logic_vector(3 downto 0);
7         sel             : in  std_logic_vector(1 downto 0);
8         y               : out std_logic_vector(3 downto 0)
9     );
10 end mux4to1;
11
12 architecture behavioral of mux4to1 is
13 begin
14     process(sel)
15         variable temp : std_logic_vector(3 downto 0)
16     begin
17         case sel is
18             when "00" =>
19                 temp <= d0;
20             when "01" =>
21                 temp <= d1;
22             when "10" =>
23                 temp <= d2;
24             when "11" =>
25                 temp <= d3;
26             when others =>
27                 temp <= (others => '0');
28         end case;
29         y := temp;
30     end process;
31 end behavioral;
```

四、程序阅读分析题（5 分）

题目要求：阅读以下 VHDL 代码，回答：（1）该电路实现的是什么功能？（2 分）（2）该电路是组合逻辑还是时序逻辑？说明判断依据。（3 分）

Listing 4.2: 程序阅读分析

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity priority_enc is
5     port (
6         din   : in   std_logic_vector(3 downto 0);
7         dout  : out std_logic_vector(1 downto 0);
8         v     : out std_logic
9     );
10 end priority_enc;
11
12 architecture rtl of priority_enc is
13 begin
14     process(din)
15     begin
16         v <= '1';
17         if din(3) = '1' then
18             dout <= "11";
19         elsif din(2) = '1' then
20             dout <= "10";
21         elsif din(1) = '1' then
22             dout <= "01";
23         elsif din(0) = '1' then
24             dout <= "00";
25         else
26             dout <= "00";
27             v <= '0';
28         end if;
29     end process;
30 end rtl;
```

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——8 位二进制比较器（20 分）

题目要求：设计一个 8 位二进制数值比较器，比较两个 8 位输入 $A[7:0]$ 和 $B[7:0]$ 的大小关系，输出三个信号：AGTBOUT（A 大于 B 时为 1）、AEQBOUT（A 等于 B 时为 1）、ALTBOUT（A 小于 B 时为 1）。

- (1) 画出顶层模块的端口框图，标注所有输入输出信号及位宽。（5 分）
- (2) 写出完整的 VHDL 代码（实体 + 结构体）。（10 分）
- (3) 用 VHDL 编写测试激励（Testbench），至少覆盖三种比较结果（ $A > B$ 、 $A = B$ 、 $A < B$ ）的测试用例。（5 分）

设计题 2：时序逻辑设计——模 10 计数器（20 分）

题目要求：设计一个带同步清零和使能端的模 10 计数器（计数范围 0~9）。

- (1) 画出状态转移图，标注所有状态及转移条件。（5 分）
- (2) 写出完整的 VHDL 代码。要求：
 - 输入：clk（时钟）、rst（同步清零，高有效）、en（计数使能，高有效）
 - 输出：q[3:0]（当前计数值）、cout（进位输出，计数值为 9 且 en=1 时为 1）

（10 分）
- (3) 若需要将计数范围扩展为 0~59（模 60 计数器），请写出修改后的 VHDL 代码，并在代码注释中说明关键改动。（5 分）

设计题 3：状态机设计——“101”序列检测器（20 分）

题目要求：设计一个 Moore 型状态机，用于检测串行输入序列“101”。每当检测到完整的“101”序列时，输出 detect 为 1（维持 1 个时钟周期），其余情况输出为 0。允许序列重叠检测（即“10101”中应检测到两次“101”）。

- (1) 画出状态转移图，标注各状态的名称、含义、输出值以及转移条件。(6 分)
- (2) 列出状态转移表（状态编码自定）。(4 分)
- (3) 写出完整的 VHDL 代码（实体 + 结构体），要求使用双进程（或三进程）描述风格，且状态编码清晰可读。(10 分)

第五章 第 5 套试卷

FPGA 与 VHDL 基础试卷（五）

（满分：100 分 考试时间：120 分钟）

5.1 一、选择题（共 6 题，每题 3 分，共 18 分）

1. 下列关于 CPLD 与 FPGA 结构的说法，正确的是（ ）。
 - A. CPLD 基于查找表结构，FPGA 基于乘积项结构
 - B. CPLD 基于乘积项结构，适合组合逻辑；FPGA 基于查找表结构，适合时序逻辑
 - C. CPLD 和 FPGA 均基于查找表结构
 - D. CPLD 的集成度通常远高于 FPGA
2. FPGA 中的查找表（LUT）实质上是一个（ ）。
 - A. 由 D 触发器构成的阵列
 - B. 由 SRAM 构成的小容量存储器
 - C. 专用的硬件乘法器
 - D. 锁相环（PLL）模块
3. 在 VHDL 中，以下关于 SIGNAL（信号）与 VARIABLE（变量）的描述，**错误**的是（ ）。
 - A. 信号用于进程间通信，变量只能在进程内部使用
 - B. 变量的赋值立即生效，信号赋值在进程挂起时才更新
 - C. 信号使用<=赋值，变量使用:=赋值
 - D. 信号只能在进程内声明，变量可以在结构体中声明
4. IEEE.STD_LOGIC_1164 标准中,STD_LOGIC 数据类型一共包含多少种逻辑值?()

- A. 2 种（'0' 和 '1'）
B. 4 种（'0', '1', 'Z', 'X'）
C. 7 种
D. 9 种
5. 下列关于 VHDL 中 PROCESS（进程）语句的说法，正确的是（ ）。
A. 一个结构体（ARCHITECTURE）中最多只能包含一个进程
B. 进程的敏感信号列表可以省略，不影响综合
C. 当敏感信号列表中任一信号发生变化时，进程被激活执行
D. 进程内部不能使用 IF 语句，只能使用 CASE 语句
6. PAL（可编程阵列逻辑）器件的基本结构特点是（ ）。
A. 与阵列可编程，或阵列固定
B. 与阵列固定，或阵列可编程
C. 与阵列和或阵列均可编程
D. 与阵列和或阵列均固定

5.2 二、填空题（共 4 题，每题 3 分，共 12 分）

1. FPGA 的英文全称是_____，中文译为_____。
2. 在 VHDL 设计中，ENTITY（实体）描述电路的_____，ARCHITECTURE（结构体）描述电路的_____。
3. VHDL 中，并置（拼接）运算符符号是_____；信号赋值符号是_____；变量赋值符号是_____。
4. 在 VHDL 的 PROCESS 语句中，当时钟上升沿到来时执行操作的典型写法是：
IF _____ 'EVENT AND _____ = '1' THEN。

5.3 三、程序改错题（共 1 题，每题 5 分，共 5 分）

题目：以下 VHDL 程序意图设计一个带异步复位和使能端的 8 位二进制加法计数器（模 256），但程序中共有 6 处错误。请找出这些错误，并写出正确的代码。

Listing 5.1: 带错误的 8 位计数器程序（共 6 处错误）

1 LIBRARY IEEE

```
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  ENTITY counter8 IS
6      PORT(
7          clk      : IN  STD_LOGIC;
8          rst      : IN  STD_LOGIC;
9          en       : IN  STD_LOGIC;
10         count    : OUT INTEGER RANGE 0 TO 255
11     );
12 END counter8;
13
14 ARCHITECTURE behavioral OF counter8 IS
15     SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNT0 0);
16 BEGIN
17     PROCESS(clk)
18     BEGIN
19         IF rst = '1' THEN
20             cnt <= 0;
21         ELSIF clk'EVENT AND clk = '1' THEN
22             IF en = '1' THEN
23                 cnt = cnt + 1;
24             END IF;
25         END PROCESS;
26     END IF;
27
28     count <= cnt;
29 END behavioral;
```

要求:

- (1) 逐一指出每处错误的位置和错误原因;
- (2) 给出修改后的正确 VHDL 代码。

5.4 四、程序阅读分析题（共 1 题，每题 5 分，共 5 分）

题目：阅读以下 VHDL 程序，回答下列问题。

Listing 5.2: 待分析 VHDL 程序

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY circuit_x IS
5      PORT(
6          din   : IN   STD_LOGIC_VECTOR(2 DOWNTO 0);
7          en    : IN   STD_LOGIC;
8          dout  : OUT  STD_LOGIC_VECTOR(7 DOWNTO 0)
9      );
10 END circuit_x;
11
12 ARCHITECTURE behav OF circuit_x IS
13 BEGIN
14     PROCESS(din, en)
15     BEGIN
16         IF en = '0' THEN
17             dout <= (OTHERS => '0');
18         ELSE
19             CASE din IS
20                 WHEN "000" => dout <= "00000001";
21                 WHEN "001" => dout <= "00000010";
22                 WHEN "010" => dout <= "00000100";
23                 WHEN "011" => dout <= "00001000";
24                 WHEN "100" => dout <= "00010000";
25                 WHEN "101" => dout <= "00100000";
26                 WHEN "110" => dout <= "01000000";
27                 WHEN "111" => dout <= "10000000";
28                 WHEN OTHERS => dout <= "00000000";
29             END CASE;
30         END IF;
31     END PROCESS;
32 END behav;
```

问题:

- (1) 该程序实现的是什么电路? 其功能是什么? (2 分)
- (2) 信号en在电路中起什么作用? 当en = '0'时输出是什么? (1 分)
- (3) 若输入din = "011"且en = '1', 输出dout的值是什么? (1 分)

- (4) 该电路属于组合逻辑电路还是时序逻辑电路？请说明判断依据。（1 分）

5.5 五、综合设计题（共 3 题，共 60 分）

5.5.1 设计题 1：8 线-3 线优先编码器设计（20 分）

设计一个 8 线-3 线优先编码器（8-to-3 Priority Encoder），输入为 $I_7 \sim I_0$ （ I_7 优先级最高），输出为 $Y_2Y_1Y_0$ （二进制编码），同时包含一个有效输出标志 GS（Group Signal，当至少有一个输入有效时 $GS=1$ ）。

要求：

- (1) 列出 8-3 优先编码器的真值表（5 分）；
- (2) 写出 Y_2 、 Y_1 、 Y_0 和 GS 的逻辑表达式（5 分）；
- (3) 编写该优先编码器的 VHDL 代码（要求使用 IF-ELSIF 语句描述优先级）（6 分）；
- (4) 画出该编码器的外部接口（符号）框图，标注所有输入输出信号（4 分）。

5.5.2 设计题 2：模 60 计数器设计（20 分）

设计一个同步模 60 计数器（计数范围 0-59），可用于数字钟的秒/分计数。计数器包含时钟输入 clk、异步复位 rst（高电平有效）、使能输入 en（高电平有效），以及两个 BCD 码输出：十位输出 tens（3~0）和个位输出 units（3~0）。当计数值达到 59 后，下一个时钟脉冲使计数值回到 0。

要求：

- (1) 画出该模 60 计数器的状态转换图（至少画出 0-59 中若干有代表性的状态迁移，并标注复位后的初始状态）（4 分）；
- (2) 编写完整的 VHDL 代码实现该计数器（8 分）；
- (3) 编写一个简单的测试平台（Testbench）代码，验证计数器的功能，包括复位、使能计数和模 60 循环（6 分）；
- (4) 简述该计数器在数字钟系统中的应用原理（2 分）。

5.5.3 设计题 3：“101”序列检测器（Moore 状态机）设计（20 分）

设计一个 Moore 型状态机，用于检测串行输入序列中的“101”模式。每当时钟上升沿到来时，采样一位串行输入 din，状态机检测到完整的“101”序列后，输出 dout 置

为'1'（持续一个时钟周期），否则 dout 为'0'。允许检测重叠的序列（即“10101”中应检测出两次“101”）。

例如输入序列：...0 1 0 1 1 0 1...，则检测输出为：...0 1 0 1 0 0 1...（加粗位置输出 1）。

要求：

- (1) 画出该 Moore 状态机的**状态转换图**，标注每个状态的名称、含义及其输出值（6 分）；
- (2) 列出**状态转换表**（当前状态、输入、次态、输出）（5 分）；
- (3) 编写完整的 VHDL 代码实现该序列检测器（用双进程或三进程方式描述）（7 分）；
- (4) 若不希望检测重叠序列（即“10101”中只检测出一次“101”），状态转换图应如何修改？请简要说明修改方法（2 分）。

——试卷结束——

第六章 第 6 套试卷

一、选择题（每题 3 分，共 18 分）

1. 关于 CPLD 与 FPGA 互连结构的比较，以下说法**正确**的是？
 - A. CPLD 采用分段式互连，延时不固定；FPGA 采用连续式互连，延时确定
 - B. CPLD 采用连续式互连（Crossbar），延时可预测；FPGA 采用分段式互连，延时与布线有关
 - C. CPLD 和 FPGA 均采用相同的互连结构，延时均可精确预测
 - D. FPGA 的互连资源全部为全局连线，不存在局部互连
2. 下列哪一种**不是** FPGA 常用的配置（下载）模式？
 - A. 主串模式（Master Serial）
 - B. 从并模式（Slave Parallel）
 - C. JTAG 边界扫描模式
 - D. DMA 直接存储器访问模式
3. 在 VHDL 中，以下关于并发语句（Concurrent Statement）和顺序语句（Sequential Statement）的描述，**错误**的是？
 - A. 结构体（ARCHITECTURE）内部的语句默认为并发执行
 - B. PROCESS 内部的语句为顺序执行
 - C. 并发信号赋值语句（如 $y \leq a \text{ AND } b$ ）可出现在结构体内部
 - D. IF 语句和 CASE 语句属于并发语句，可出现在结构体中的任何位置
4. 关于 Moore 型与 Mealy 型状态机的输出时序，以下说法**正确**的是？

- A. Moore 型输出仅取决于当前状态, Mealy 型输出取决于当前状态和输入
 - B. Mealy 型输出仅取决于当前状态, Moore 型输出取决于当前状态和输入
 - C. 两者的输出均仅取决于当前状态
 - D. 两者的输出均取决于当前状态和输入
5. VHDL 中逻辑运算符的优先级描述, 正确的是?
- A. AND 优先级最高, OR 和 NOT 优先级相同
 - B. NOT 优先级最高, AND、OR、XOR 等二元逻辑运算符优先级相同
 - C. XOR 优先级最高, 其次是 AND, 最后是 OR
 - D. 所有逻辑运算符 (NOT、AND、OR、XOR) 优先级均相同
6. 在 VHDL 中, 关于 GENERATE 语句的描述, 错误的是?
- A. FOR GENERATE 用于重复生成一组结构相同的硬件单元
 - B. IF GENERATE 可根据条件选择性地生成硬件
 - C. GENERATE 语句只能出现在结构体 (ARCHITECTURE) 内部
 - D. GENERATE 语句属于顺序语句, 只能在 PROCESS 内部使用

二、填空题 (每题 3 分, 共 12 分)

1. 在 VHDL 中, 信号 (SIGNAL) 通常在_____中声明, 变量 (VARIABLE) 通常在_____中声明, 常量 (CONSTANT) 可在_____中声明 (填声明位置: 结构体/进程/两者均可)。
2. VHDL 中, GENERIC 语句用于向实体传递_____ (如位宽、延时参数), COMPONENT 语句用于声明_____ (即将被实例化的底层模块接口)。
3. GENERATE 语句的两种基本形式是_____和_____, 它们均属于**并发**语句。
4. 在 VHDL 中, 若一个进程 (PROCESS) 描述的是组合逻辑电路, 则其敏感信号列表中必须包含_____, 否则综合工具可能推断出_____ (填一种非预期的存储元件)。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码（意图实现一个组合逻辑二选一多路选择器），找出其中的 5 处错误，并写出正确的修改方案。

Listing 6.1: 存在 5 处错误的 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY mux_err IS
5      PORT(
6          a, b : IN  STD_LOGIC;
7          sel  : IN  STD_LOGIC;
8          y    : OUT STD_LOGIC
9      );
10 END mux_err;
11
12 ARCHITECTURE behav OF mux_err IS
13 BEGIN
14     PROCESS(a)                -- 敏感信号列表
15         VARIABLE tmp : STD_LOGIC;
16     BEGIN
17         IF sel = '1' THEN
18             tmp := a;
19             -- 此处缺少 ELSE 分支
20         END IF;
21         y := tmp;              -- 输出赋值
22         tmp <= b;              -- 变量赋值
23     END PROCESS;
24     y <= tmp;                  -- 进程外语句
25 END behav;
```

答题要求：逐行指出错误位置（可用行号）、错误原因及正确写法（每处 1 分，找出 5 处即得满分 5 分）。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 6.2: 分析用 VHDL 代码——分频器

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY freq_div IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;
8          clk_out  : OUT STD_LOGIC
9      );
10 END freq_div;
11
12 ARCHITECTURE behav OF freq_div IS
13     SIGNAL count  : INTEGER RANGE 0 TO 3 := 0;
14     SIGNAL temp   : STD_LOGIC := '0';
15 BEGIN
16     PROCESS(clk, rst)
17     BEGIN
18         IF rst = '1' THEN
19             count <= 0;
20             temp  <= '0';
21         ELSIF rising_edge(clk) THEN
22             IF count = 3 THEN
23                 count <= 0;
24                 temp  <= NOT temp;
25             ELSE
26                 count <= count + 1;
27             END IF;
28         END IF;
29     END PROCESS;
30     clk_out <= temp;
31 END behav;
```

问题：

- (1) 该 VHDL 代码描述的是什么电路？（1 分）
- (2) 若输入时钟`clk`的频率为 8 MHz，输出`clk_out`的频率是多少？请写出计算过程。（2 分）
- (3) 信号`temp`在电路中的作用是什么？为什么不能直接用`count`作为输出？（1 分）
- (4) 该电路的复位`rst`是同步复位还是异步复位？请说明判断依据。（1 分）

五、综合设计题（共 60 分）

设计题 1：BCD 码—七段数码管译码器（20 分）

设计一个 BCD 码（0-9）到七段共阳极数码管的译码器。七段数码管的段标识为 a、b、c、d、e、f、g（共阳极：段输出为‘0’时对应段点亮）。输入为 4 位 BCD 码 `bcd(3 downto 0)`，输出为 7 位段码 `seg(6 downto 0)`，其中 `seg(6)` 对应 a 段，`seg(0)` 对应 g 段。当输入超出 0-9 范围（即 10-15）时，所有段熄灭（全输出‘1’）。

要求：

- (1) 列出完整的真值表（输入 BCD 码 0-9 以及非法输入 10-15）。（6 分）
- (2) 画出顶层模块端口框图，标注所有端口名称、方向和位宽。（4 分）
- (3) 写出完整的 VHDL 代码（实体 + 结构体）。要求使用 CASE 语句或 WITH-SELECT 选择信号赋值语句（数据流风格）实现译码逻辑。（10 分）

提示：共阳极数码管中，段输出‘0’点亮、‘1’熄灭。七段排列如下：

```
a
---
f|  |b
-g-
e|  |c
---
d
```

`seg(6 downto 0) = {a, b, c, d, e, f, g}`。例如：显示数字“0”时，仅 g 段熄灭，故 `seg = "0000001"`。

设计题 2：4 位双向移位寄存器（20 分）

使用 VHDL 的 GENERATE 语句和组件实例化（COMPONENT + PORT MAP）的结构化描述方式，设计一个 4 位双向移位寄存器。要求：

- 输入端口：`clk`（时钟）、`rst`（异步复位，高电平有效）、`dir`（方向控制：‘0’= 右移，‘1’= 左移）、`sil`（左移串行输入）、`sir`（右移串行输入）。
- 输出端口：`q(3 downto 0)`（4 位并行输出）。

- 功能: `rst='1'` 时异步清零 (`q <= "0000"`); 时钟上升沿到来时, 若 `dir='0'` 则右移一位 (`q <= sir & q(3 downto 1)`), 若 `dir='1'` 则左移一位 (`q <= q(2 downto 0) & sil`)。

要求:

- (1) 画出该双向移位寄存器的顶层端口框图, 标注所有端口信号名称、方向和位宽。(4 分)
- (2) 先设计一个带使能的 D 触发器基本单元 (`dff_en` 实体), 然后用 `FOR...GENERATE` 语句和 `IF...GENERATE` 语句, 结合 `COMPONENT` 实例化 (`PORT MAP`), 搭建 4 位双向移位寄存器的结构体。要求: 通过 `GENERATE` 区分首尾级与中间级的连接方式。(12 分)
- (3) 简述使用 `GENERATE` 语句相比手动编写 4 次 D 触发器实例化的优势。(4 分)

提示: 基本 D 触发器 `dff_en` 的端口建议为: `clk`, `rst`, `d`, `q`。各级的 D 输入端由 `dir` 控制的多路选择逻辑决定: 第 0 级在左移时选择 `sil`、右移时选择 `q(1)`; 第 3 级在右移时选择 `sir`、左移时选择 `q(2)`; 中间级根据 `dir` 选择 `q(i+1)` 或 `q(i-1)`。

设计题 3: “1011” 序列检测器 (20 分)

设计一个 “1011” 序列检测器 (重叠检测)。输入信号为 `din` (1 位串行输入), 当时钟上升沿到来时采样 `din`。当连续检测到 “1011” 序列时, 输出 `found` 有效。要求分别设计 **Moore 型** 和 **Mealy 型** 两种状态机, 并对比分析二者的差异。

重叠检测示例: 输入序列 “1011011” 中, 包括两次 “1011” (第 1-4 位和第 3-6 位), 应输出两次检测信号。

要求:

- (1) **Moore 型设计 (8 分):**
 - (a) 画出 Moore 型状态转换图, 标注每个状态的名称、含义及输出值。(3 分)
 - (b) 写出完整的 Moore 型 VHDL 代码, 采用双进程描述风格。时钟 `clk` 上升沿触发, 复位 `rst` 为同步复位 (高电平有效)。(5 分)
- (2) **Mealy 型设计 (8 分):**

- (a) 画出 Mealy 型状态转换图，标注状态名称、转换条件和输出值。(3 分)
- (b) 写出完整的 Mealy 型 VHDL 代码，采用双进程（或三进程）描述风格。复位要求同上。(5 分)

(3) 对比分析 (4 分):

- (a) 分别写出 Moore 型和 Mealy 型检测到“1011”时，`found`信号有效的时间（相对于输入序列最后一位的时钟周期）。(2 分)
- (b) 分别列出 Moore 型和 Mealy 型所需的最少状态数，并从状态数量和输出时序两方面比较二者的优劣。(2 分)

提示：Moore 型检测“1011”需要 5 个状态（S0-S4），Mealy 型需要 4 个状态（S0-S3）。Moore 型输出仅与当前状态有关，Mealy 型输出与当前状态和输入有关。验证重叠检测：序列“1011011”应在第 4 和第 6 个时钟周期后输出`found='1'`（Moore 型）或第 4 和第 6 个时钟周期同周期输出`found='1'`（Mealy 型）。

第七章 第 7 套试卷

一、选择题（每题 3 分，共 18 分）

1. 以下哪种不是 FPGA 常见的配置（编程）模式？
 - A. Master Serial 模式（主串模式）
 - B. Slave Parallel 模式（从并模式）
 - C. JTAG 模式
 - D. DMA 模式
2. 在 VHDL 中，关于并发语句和顺序语句的描述，正确的是？
 - A. PROCESS 内部的语句是并发执行的
 - B. 结构体中的 PROCESS 语句之间是顺序执行的
 - C. 结构体中的信号赋值语句（如 `y <= a AND b;`）是并发执行的
 - D. 顺序语句不能出现在结构体中
3. Moore 型状态机与 Mealy 型状态机的根本区别在于？
 - A. Moore 型状态机只能使用同步复位
 - B. Moore 型状态机的输出仅取决于当前状态，与输入无关
 - C. Mealy 型状态机的状态数一定比 Moore 型少
 - D. Mealy 型状态机不能检测重叠序列
4. 在 VHDL 中，下列运算符中优先级最高的是？
 - A. AND

- B. OR
- C. NOT
- D. XOR

5. 关于 CPLD 与 FPGA 的对比，以下说法正确的是？

- A. CPLD 基于 SRAM 工艺，掉电后配置数据丢失；FPGA 基于 Flash 工艺，掉电数据保留
- B. CPLD 通常具有确定性的互连延时可预测，FPGA 的延时与布局布线有关
- C. CPLD 的逻辑容量通常远大于 FPGA
- D. FPGA 的引脚到引脚延时通常小于 CPLD

6. 在 VHDL 中，FOR ... GENERATE 语句的主要用途是？

- A. 在仿真中循环生成测试向量
- B. 在 PROCESS 内部实现循环计算
- C. 重复生成多个相同的硬件结构（如元件例化）以简化代码
- D. 替代 CASE 语句进行多路选择

二、填空题（每空 3 分，共 12 分）

1. 在 VHDL 中，GENERIC 关键字用于定义实体的_____参数；结构体中例化另一个设计单元时，需先使用_____关键字声明该子模块的接口。
2. VHDL 中的 GENERATE 语句有 FOR GENERATE 和_____两种形式。
3. 在 VHDL 中，SIGNAL 可以在_____中声明（如结构体的声明区），具有全局性；而 VARIABLE 只能在_____内部声明，作用范围小。
4. 描述组合逻辑的 PROCESS 语句，其敏感信号表中应包含_____，否则综合时可能产生不一致的结果或推断出锁存器（Latch）。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码，该代码意图描述一个带使能的 4 位加法器，但存在 5 处错误。请指出错误所在行（或位置），说明错误原因并给出正确的修改方案。

Listing 7.1: 存在错误的 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY adder4 IS
6      PORT(
7          a, b : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
8          en   : IN  STD_LOGIC;
9          sum  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
10         cout : OUT STD_LOGIC
11     );
12 END adder4;
13
14 ARCHITECTURE behav OF adder4 IS
15     SIGNAL temp : INTEGER RANGE 0 TO 15;
16 BEGIN
17     PROCESS(a, b)
18     BEGIN
19         IF en = '1' THEN
20             temp <= TO_INTEGER(UNSIGNED(a)) + TO_INTEGER(UNSIGNED(b));
21             sum <= STD_LOGIC_VECTOR(TO_UNSIGNED(temp, 4));
22             IF temp > 15 THEN
23                 cout := '1';
24             ELSE
25                 cout := '0';
26             END IF;
27         END IF;
28     END PROCESS;
29 END behav;
```

答题要求：逐条列出错误位置、错误原因及正确写法。每处 1 分，共 5 分。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 7.2: 分析用 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY freq_div IS
6      GENERIC(
7          N : INTEGER := 4
8      );
9      PORT(
10         clk    : IN  STD_LOGIC;
11         rst    : IN  STD_LOGIC;
12         clk_out : OUT STD_LOGIC
13     );
14 END freq_div;
15
16 ARCHITECTURE behav OF freq_div IS
17     SIGNAL count : INTEGER RANGE 0 TO (2**N)-1 := 0;
18     SIGNAL temp  : STD_LOGIC := '0';
19 BEGIN
20     PROCESS(clk, rst)
21     BEGIN
22         IF rst = '1' THEN
23             count <= 0;
24             temp  <= '0';
25         ELSIF rising_edge(clk) THEN
26             IF count = (2**N)-1 THEN
27                 count <= 0;
28                 temp  <= NOT temp;
29             ELSE
30                 count <= count + 1;
31             END IF;
32         END IF;
33     END PROCESS;
34     clk_out <= temp;
```

35 `END behav;`

问题：

- (1) 该 VHDL 代码描述的是什么电路？（1 分）
- (2) 参数 N 的作用是什么？当 N=4 时，输出信号 `clk_out` 的频率与输入时钟 `clk` 的频率之间的关系是什么？（2 分）
- (3) 该电路中的复位信号 `rst` 是同步复位还是异步复位？请根据代码说明理由。（2 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——BCD 码转 7 段数码管译码器（20 分）

设计一个 BCD 码到 7 段共阳极数码管的译码器。输入为一个 4 位 BCD 码 `bcd(3 downto 0)`（取值范围 0000~1001，即 0~9），输出为 7 段数码管的段选信号 `seg(6 downto 0)`，分别对应数码管的 a、b、c、d、e、f、g 段（共阳极：段选信号为‘0’时该段点亮，为‘1’时熄灭）。当输入不是有效的 BCD 码（1010~1111）时，所有段熄灭（全输出‘1’）。要求：

- (1) 列出完整的真值表（输入 BCD 码与输出 7 段码的对应关系）。（5 分）
- (2) 画出顶层模块的端口框图，标注所有端口名称、方向和位宽。（4 分）
- (3) 写出完整的 VHDL 代码（实体 + 结构体），要求使用选择信号赋值语句（WITH ... SELECT）实现数据流风格的描述。（8 分）
- (4) 在代码中以注释方式说明设计思路。（3 分）

提示：共阳极数码管，7 段 a~g 对应 `seg(6)~seg(0)`。数字 0 的段码为"1000000"，数字 1 为"1111001"，等等。有效的 BCD 码只有 0~9（二进制 0000~1001），其余输入为无效状态。

设计题 2：时序逻辑设计——4 位双向移位寄存器（结构化设计）（20 分）

使用结构化 VHDL 描述方式，设计一个 4 位双向移位寄存器。要求先设计一个带二选一多路选择器的 D 触发器（`dff_mux`），再通过 COMPONENT 声明和 PORT MAP 例化，配合 FOR ... GENERATE 语句，构建完整的 4 位移位寄存器。端口定义如下：

- 输入：clk（时钟）、rst（异步复位，高有效）、dir（方向控制：‘1’时右移，‘0’时左移）、si_left（左移时的串行输入端）、si_right（右移时的串行输入端）。
- 输出：q(3 downto 0)（4 位并行输出）。

- (1) 画出顶层移位寄存器的结构框图，标注各 D 触发器之间的级联关系、多路选择器的作用以及所有信号。（5 分）
- (2) 写出底层模块 `dff_mux` 的完整 VHDL 代码（实体 + 结构体）。（5 分）
- (3) 写出顶层模块的完整 VHDL 代码（实体 + 结构体），使用 COMPONENT 声明 `dff_mux`，并用 FOR ... GENERATE 语句例化 4 个 `dff_mux`，注意首尾级联信号的正确处理。（10 分）

提示：`dff_mux` 内部包含一个 2 选 1 MUX 和一个 D 触发器。MUX 的两个数据输入端分别连接左移串行输入和右移串行输入，方向信号 `dir` 控制 MUX 的选择。顶层中使用 `GENERATE` 循环例化时，需用 `IF GENERATE` 处理边界位置（第 0 位和第 3 位）的特殊连接。

设计题 3：状态机设计——“1011”序列检测器（20 分）

设计一个 **Moore 型** 有限状态机，用于检测串行输入序列“1011”。输入信号为 `din`（1 位串行输入），输出信号为 `found`（1 位）。当时钟上升沿采样到完整的“1011”序列后，`found` 在下一时钟周期输出‘1’（持续一个周期），其余时间输出‘0’。允许序列重叠检测（例如“101011”中应能检测出两次“1011”）。要求：

- (1) 画出 Moore 型状态机的状态转移图。标注每个状态的名称（建议使用 `S0~S4`）、含义以及各状态下的输出值。标注所有的状态转移条件和方向。（8 分）
- (2) 列出状态转移表（当前状态、输入 `din`、次态、输出 `found`）。（4 分）
- (3) 写出完整的 VHDL 代码。要求：
 - 使用用户自定义枚举类型定义状态（如 `TYPE state_type IS (S0, S1, S2, S3, S4)`）；（1 分）
 - 采用三进程描述风格：时序进程（状态寄存器）、组合进程（次态逻辑）、组合进程（输出逻辑）；（2 分）
 - 时钟 `clk` 上升沿触发，复位 `rst` 为异步复位（高有效）。（2 分）

代码规范（3 分）：缩进规范、信号命名清晰、注释合理。

提示：Moore 型状态机输出仅与当前状态有关。对于“1011”序列检测（重叠检测），建议定义 5 个状态：`S0`（初始/未匹配）、`S1`（已匹配“1”）、`S2`（已匹配“10”）、`S3`（已匹配“101”）、`S4`（已匹配“1011”）。注意正确绘制所有状态转移——例如在 `S2` 状态收到‘0’应如何处理？在 `S4` 状态收到不同输入时又应如何处理（考虑重叠）？

第八章 第 8 套试卷

FPGA 与 VHDL 基础试卷（八）

（满分：100 分 考试时间：120 分钟）

一、选择题（共 6 题，每题 3 分，共 18 分）

1. FPGA 中一个典型的逻辑单元（Slice）通常包含以下哪组资源？（ ）
A. 仅由 LUT 和 D 触发器组成
B. LUT、D 触发器、进位链和多路选择器
C. 完整的嵌入式微处理器核
D. 模拟锁相环（PLL）和时钟管理模块
2. 在 VHDL 中，若需将 STD_LOGIC_VECTOR 类型转换为 UNSIGNED 类型以进行算术运算，应引用以下哪个包？（ ）
A. IEEE.STD_LOGIC_1164
B. IEEE.STD_LOGIC_ARITH
C. IEEE.NUMERIC_STD
D. IEEE.STD_LOGIC_SIGNED
3. 下列关于同步复位（Synchronous Reset）与异步复位（Asynchronous Reset）的描述，正确的是（ ）。
A. 异步复位不依赖时钟即可生效，但对复位信号的毛刺较为敏感
B. 同步复位的响应速度比异步复位更快
C. 异步复位在 VHDL 代码中不需要将复位信号列入敏感信号表
D. 同步复位比异步复位占用更多的逻辑资源
4. 关于有限状态机（FSM）的状态编码方式，以下描述正确的是（ ）。

- A. 二进制编码使用最少的状态寄存器,但译码逻辑较复杂
 - B. 独热码 (One-Hot) 需要最少的状态寄存器
 - C. 格雷码 (Gray Code) 在相邻状态转换时有多位同时翻转
 - D. 独热码的抗干扰能力最差
5. 在 VHDL 中, SUBTYPE 关键字的主要作用是 ()。
- A. 创建一个全新的数据类型
 - B. 从已有类型派生子类型并添加范围约束
 - C. 声明信号间的物理连接关系
 - D. 定义进程内部使用的局部变量
6. FPGA 中的 Block RAM 与 Distributed RAM 的主要区别是 ()。
- A. Distributed RAM 由专用硬核实现, Block RAM 由 LUT 构成
 - B. Block RAM 是片内专用存储硬核, Distributed RAM 由 LUT 逻辑资源构成
 - C. 两者完全等价, 没有本质区别
 - D. Block RAM 仅用于存储程序代码, Distributed RAM 仅用于存储数据

二、填空题（共 4 题，每题 3 分，共 12 分）

1. VHDL 中常用的信号属性包括：

- `clk'EVENT`：表示信号值_____（填“发生变化”或“达到稳定”）；
- `rising_edge(clk)` 等价于_____ AND _____；
- `data'RANGE` 返回信号 `data` 的_____ 范围。

2. 在 VHDL 中描述三态输出：

- 高阻态的字符表示为_____；
- 描述双向总线时，端口模式应使用_____（填 `IN`、`OUT`、`INOUT` 或 `BUFFER`）；
- 当多个三态门驱动同一总线时，同一时刻最多只能有_____ 个门的使能有效。

3. 同步复位与异步复位的 VHDL 代码模板有本质区别（以高电平复位为例）：

- 同步复位的典型结构：

```
IF rising_edge(clk) THEN
    _____ rst = '1' THEN ... END IF;
END IF;
```

- 异步复位的典型结构：

```
IF rst = '1' THEN ...
    _____ rising_edge(clk) THEN ... END IF;
```

4. 在 `IEEE.NUMERIC_STD` 包中，将 `UNSIGNED` 类型转换为 `INTEGER` 类型的函数是_____；将 `INTEGER` 转换为 `UNSIGNED` 类型的函数是_____（需额外指定目标位宽参数）；将 `STD_LOGIC_VECTOR` 转换为 `UNSIGNED` 类型使用_____ 函数。

三、程序改错题（共 5 分）

题目：以下 VHDL 程序意图设计一个带异步复位和使能端的 8 位二进制加法计数器（模 256），但代码中存在 5 处错误。请逐一指出错误位置并写出正确的修改方案。

Listing 8.1: 带错误的 8 位计数器程序（找出 5 处错误，每处 1 分）

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY counter8 IS
5      PORT(
6          clk    : IN    STD_LOGIC;
7          rst    : IN    STD_LOGIC;
8          en     : IN    STD_LOGIC;
9          q      : OUT  STD_LOGIC_VECTOR(7 DOWNT0 0)
10     );
11 END counter8;
12
13 ARCHITECTURE behav OF counter8 IS
14     SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNT0 0)
15 BEGIN
16     PROCESS(clk)
17     BEGIN
18         IF rst = '1' THEN
19             cnt := (OTHERS => '0');
20         ELSIF rising_edge(clk) THEN
21             IF en = '1' THEN
22                 cnt <= cnt + '1';
23             END IF;
24         END IF;
25     END PROCESS;
26     q <= cnt;
27 END behav;
```

答题要求：在答题纸上按以下格式依次写出各错误：

错误 1(第 ____ 行):_____ 改为 _____(原因:_____)

错误 2(第 ____ 行):_____ 改为 _____(原因:_____)

错误 3(第 ____ 行):_____ 改为 _____(原因:_____)

错误 4(第 ____ 行):_____ 改为 _____(原因:_____)

错误 5(第 ____ 行):_____ 改为 _____(原因:_____)

四、程序阅读分析题（共 5 分）

题目：阅读以下 VHDL 程序，回答下列问题。

Listing 8.2: 待分析 VHDL 程序——环形计数器

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY ring_counter IS
5      PORT(
6          clk    : IN    STD_LOGIC;
7          rst    : IN    STD_LOGIC;
8          q      : OUT  STD_LOGIC_VECTOR(3 DOWNTO 0)
9      );
10 END ring_counter;
11
12 ARCHITECTURE behav OF ring_counter IS
13     SIGNAL state : STD_LOGIC_VECTOR(3 DOWNTO 0);
14 BEGIN
15     PROCESS(clk, rst)
16     BEGIN
17         IF rst = '1' THEN
18             state <= "1000";
19         ELSIF rising_edge(clk) THEN
20             state <= state(0) & state(3 DOWNTO 1);
21         END IF;
22     END PROCESS;
23     q <= state;
24 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？
- (2) (2 分) 从复位释放后开始，请依次写出连续 8 个时钟上升沿后信号 q （即 $state$ ）的值。设初始复位值已载入。
- (3) (1 分) 如果第 16 行右移拼接改为 $state <= state(2 DOWNTO 0) \& state(3);$ （即循环左移），该电路将变成什么类型？输出序列将如何变化？

- (4) (1 分) 该电路中输出信号 $q(0)$ 的频率与输入时钟 clk 的频率之比是多少? 请说明推导过程。

五、综合设计题（共 3 题，共 60 分）

设计题 1：8 位算术逻辑单元（ALU）（20 分）

设计一个 8 位算术逻辑单元（ALU），能够对两个 8 位输入操作数执行 8 种基本运算。要求使用 VHDL 的行为描述风格（在 PROCESS 中使用 CASE 语句）实现。

端口定义：

- a(7 downto 0)（输入）：操作数 A
- b(7 downto 0)（输入）：操作数 B
- op(2 downto 0)（输入）：操作选择信号
- y(7 downto 0)（输出）：运算结果
- zero（输出）：零标志，当 $y = "00000000"$ 时输出'1'
- sign（输出）：符号标志，直接输出 y(7) 的值

操作码定义（op 编码）：

op(2:0)	操作	说明
"000"	PASS_A	$y \leq a$ （直通 A）
"001"	ADD	$y \leq a + b$ （加法）
"010"	SUB	$y \leq a - b$ （减法）
"011"	AND_OP	$y \leq a \text{ AND } b$ （按位与）
"100"	OR_OP	$y \leq a \text{ OR } b$ （按位或）
"101"	XOR_OP	$y \leq a \text{ XOR } b$ （按位异或）
"110"	NOT_A	$y \leq \text{NOT } a$ （按位取反 A）
"111"	SHL_A	$y \leq a(6 \text{ downto } 0) \& '0'$ （逻辑左移一位）

要求：

(1)（5 分）画出该 ALU 的顶层模块端口框图，标注所有输入输出信号、方向和位宽。

(2)（12 分）编写完整的 VHDL 代码，包括库声明、实体定义和结构体。要求：

- 引用 IEEE.NUMERIC_STD.ALL 包以支持算术运算；
- 在 PROCESS 中使用 CASE 语句根据 op 选择运算；

- 算术运算时需将 `STD_LOGIC_VECTOR` 转换为 `UNSIGNED`，运算结果再转换回 `STD_LOGIC_VECTOR`；
- 除 `zero` 和 `sign` 外，CASE 语句须覆盖所有 `op` 取值（含 `OTHERS` 分支）。

(3) (3 分) 在代码中添加注释：说明数据流方向、各 OP 分支的功能，以及 `zero/sign` 标志的生成逻辑。

提示：算术运算的类型转换示例：`y <= STD_LOGIC_VECTOR(UNSIGNED(a) + UNSIGNED(b))`；

设计题 2：N 级二分频器链（20 分）

使用结构化 VHDL 描述方法设计一个 N 级二分频器链（由 N 个 T 触发器级联而成）。每级 T 触发器将输入时钟二分频后输出至下一级，最终输出频率为 $f_{out} = f_{clk_in}/2^N$ 。级数 N 通过 GENERIC 参数设定。

基本单元——T 触发器（TFF）： T 触发器在每个时钟上升沿将输出翻转。其端口定义如下：

- clk（输入）：时钟信号
- rst（输入）：异步复位，高电平有效，复位时 $q \leq '0'$
- q（输出）：当前状态输出

顶层模块端口：

- clk_in（输入）：输入时钟
- rst（输入）：异步复位，高电平有效
- clk_out（输出）：第 N 级 T 触发器的输出（即 N 级分频后的时钟）
- chain(0 to N-1)（可选，中间节点输出，便于观察各级分频结果）：各级 TFF 的输出

要求：

- (1)（4 分）画出 N=3 时该分频器链的电路框图，标注各级输入/输出信号及时钟频率关系（设 $clk_in=8MHz$ ）。
- (2)（3 分）编写单个 T 触发器（TFF）的完整 VHDL 代码（ENTITY + ARCHITECTURE），使用变量 `toggle` 在 PROCESS 中实现翻转逻辑。
- (3)（10 分）编写顶层 N 级分频器链的完整 VHDL 代码。要求：

- 使用 GENERIC 定义级数参数 N（默认值 $N:=4$ ）；
- 在结构体中使用 COMPONENT 声明 T 触发器；
- 使用 FOR...GENERATE（必须）循环实例化 N 个 TFF；
- 使用内部信号 `chain(0 TO N-1)`（类型 `STD_LOGIC_VECTOR`）连接各级 TFF 的输出；

- 第 0 级 TFF 的时钟连接至 `clk_in`，第 i 级 ($i > 0$) TFF 的时钟连接至 `chain(i-1)`;
- 最终输出 `clk_out <= chain(N-1)`。

(4) (3 分) 在代码中添加清晰的注释，说明各模块的功能、GENERIC 参数的用途，以及 GENERATE 循环中各级的时钟连接关系。

提示：

- TFF 内部实现要点：VARIABLE `toggle : STD_LOGIC := '0'`; ——在时钟上升沿执行 `toggle := NOT toggle; q <= toggle;`。
- GENERIC 声明示例：GENERIC ($N : \text{INTEGER} := 4$); (写在 ENTITY 的 PORT 之前)。
- GENERATE 用法示例：G1: FOR i IN 0 TO $N-1$ GENERATE ... END GENERATE G1;
- 第 0 级与其余级的连接方式不同，可使用 IF GENERATE 区分：
first: IF $i = 0$ GENERATE ... END GENERATE first;
others: IF $i > 0$ GENERATE ... END GENERATE others;

设计题 3：数字密码锁控制器（20 分）

设计一个数字密码锁控制器（Finite State Machine）。该密码锁需要用户通过串行输入方式逐位输入 4 位二进制密码。预设正确密码为“1011”（即先输入 1，再输入 0，再输入 1，再输入 1）。当检测到完整的正确密码序列时，开锁信号有效；若输入错误位，则返回初始状态等待重新输入。允许序列重叠检测。

功能说明：

- 每个时钟周期采样一位输入信号 `key_in`（1 位），与预设密码逐位比对。
- 正确密码序列为 4 位：“1” → “0” → “1” → “1”。当连续 4 个时钟周期输入与此序列完全匹配时，下一个时钟周期 `unlock` 输出高电平‘1’（持续一个时钟周期），然后状态机返回初始状态。
- 若在匹配过程中某一位输入与密码不符，状态机立即返回初始状态，之前的部分匹配作废。
- 允许重叠检测：例如输入序列“1 0 1 1 0 1 1”（共 7 位），前四位“1011”触发第一次开锁（第 4 位之后），之后状态机返回 S0，从第 5 位‘0’开始重新匹配；第 5–7 位为 0、1、1，无法构成完整密码“1011”，因此仅开锁一次。若输入为“1 0 1 1 0 1 1 0 1 1 0 1 1”，则第 4 位后开锁一次，第 10 位后再次开锁（重叠发生在第 3–4–5–6 位“1–1–0–1”和第 7–8–9–10 位“1–0–1–1”之间）... 实际上重叠检测的重点在于：密码的末尾若干位可能同时构成新密码的开头。例如输入“1 0 1 1 0 1 1”：位序列 1-0-1-1 开锁（位 4 后），位 5=0 无法匹配 S0→S1，位 6=1 触发 S0→S1，位 7=1 触发 S1→S1（重叠利用）。

端口定义：

- `clk`（输入）：时钟信号，上升沿触发
- `rst`（输入）：异步复位，高电平有效，复位后回到初始状态
- `key_in`（输入）：串行密码输入（1 位）
- `unlock`（输出）：开锁信号（1 位），高电平有效，持续一个时钟周期
- `state_disp(2:0)`（输出，可选）：当前状态的编码输出，便于调试观察

要求：

- (1)（8 分）画出该密码锁控制器的状态转移图（采用 Moore 型状态机）。要求：

- 至少包含 5 个状态：S0（初始/未匹配）、S1（已匹配第一位“1”）、S2（已匹配“10”）、S3（已匹配“101”）、S4（已匹配“1011”，即开锁状态）；
- 标注每个状态的名称、含义及其输出值（unlock）；
- 标注所有状态之间的转移条件（基于 key_in 的值）。

(2) (3 分) 列出状态转移表，包含当前状态、输入（key_in）、次态和输出（unlock）。

(3) (9 分) 编写完整的 VHDL 代码实现该密码锁控制器。要求：

- 使用用户自定义枚举类型定义状态（`TYPE lock_state IS (S0, S1, S2, S3, S4);`）；
- 采用双进程（或三进程）描述风格：一个时序进程（状态寄存器和异步复位），一个组合进程（次态逻辑和输出逻辑）；
- 异步复位：rst 为高电平时立即回到 S0 状态；
- 使用二进制编码或独热码均可，但需在注释中说明编码方案；
- 代码中包含清晰的功能注释。

提示：

- Moore 型状态机中，unlock 输出仅取决于当前状态（S4 时输出 1，其余状态输出 0）。
- 状态转移逻辑：
 - S0: key_in='1' → S1; key_in='0' → 保持 S0
 - S1: key_in='0' → S2; key_in='1' → S1（注意重叠：已完成的第一位'1'可作为新序列的开头）
 - S2: key_in='1' → S3; key_in='0' → S0
 - S3: key_in='1' → S4; key_in='0' → S2（注意：“1010”中，后两位“10”恰好匹配 S2 的前缀）
 - S4: key_in='1' → S1（重叠：S4 的最后一个'1'可作为新序列的第一位）；key_in='0' → S0
- 三进程风格：一个时钟进程（状态寄存器 + 复位），一个组合进程（次态逻辑），一个组合进程（输出逻辑——Moore 型只依赖当前状态）。

——试卷结束——

第九章 第 9 套试卷

FPGA 与 VHDL 基础试卷（九）

（满分：100 分 考试时间：120 分钟）

一、选择题（共 6 题，每题 3 分，共 18 分）

1. 下列关于 CPLD 与 FPGA 内部互连结构的说法，正确的是（ ）。
 - A. CPLD 的互连资源是分段式、通过可编程开关矩阵连接的，延时可预测性差
 - B. FPGA 的互连资源是连续式布线池结构的，任意两个逻辑单元之间延时固定
 - C. CPLD 采用连续式互连结构（基于与或阵列），信号传输延时可预测性较好
 - D. CPLD 和 FPGA 的互连结构完全相同，均由 SRAM 工艺实现

2. 现代 FPGA 器件最主流的配置（编程）方式是（ ）。
 - A. 仅支持 JTAG 一种配置模式
 - B. 基于 SRAM 工艺，上电时需从外部配置存储器加载比特流
 - C. 采用反熔丝（Anti-fuse）工艺，只能一次性编程
 - D. 基于 Flash 工艺，配置文件掉电不丢失且不可在线更新

3. 关于 VHDL 中并发（Concurrent）语句与顺序（Sequential）语句的描述，错误的是（ ）。
 - A. 结构体（ARCHITECTURE）中直接出现的语句大多是并发语句
 - B. PROCESS 内部的语句是顺序执行的
 - C. 并发信号赋值语句（如 `y <= a AND b`）与进程是并发关系
 - D. 进程内部的语句在仿真时按书写顺序逐条立即生效，与实际硬件延迟无关

4. Moore 型状态机与 Mealy 型状态机的主要区别在于 ()。
- A. Moore 型的输出仅与当前状态有关, Mealy 型的输出与当前状态和输入均有关
- B. Moore 型只能检测序列, Mealy 型只能产生序列
- C. Moore 型状态机需要更多输入端口
- D. Mealy 型的输出在时钟边沿变化, Moore 型的输出在任何时候都可以变化
5. VHDL 中, 以下运算符的优先级从高到低排列**正确**的是 ()。
- A. AND > NOT > OR > XOR
- B. NOT > AND > OR > XOR
- C. NOT > AND > XOR > OR
- D. AND > OR > NOT > XOR
6. 在 VHDL 中, FOR...GENERATE 语句的主要用途是 ()。
- A. 在进程中实现循环控制
- B. 在仿真时产生周期性测试激励
- C. 在结构体中按规律重复生成多个相同或相似的硬件模块实例
- D. 替代 IF 语句实现多分支条件判断

二、填空题（共 4 题，每题 3 分，共 12 分）

1. 在 VHDL 中：

- **SIGNAL**（信号）适用于元件之间的互连和_____通信；
- **VARIABLE**（变量）只能在_____内部或子程序（FUNCTION/PROCEDURE）内部声明和使用；
- **CONSTANT**（常量）的值在_____（填“仿真前”或“仿真运行时”）确定，运行期间不可更改。

2. 在 VHDL 的结构化描述中：

- 使用关键字_____声明一个待实例化的子模块接口；
- 使用关键字_____将子模块的端口与上层信号连接起来；
- **GENERIC** 关键字用于向子模块传递_____（如位宽、延时等），提高代码复用性。

3. VHDL 中有两种 GENERATE 语句：

- _____ GENERATE 语句按固定次数重复生成硬件结构；
- _____ GENERATE 语句根据条件选择性地生成某些硬件结构。

4. 关于 PROCESS 的敏感信号列表：

- 组合逻辑进程的敏感信号列表中应包含_____（填“所有输入信号”或“仅时钟信号”）；
- 若漏写敏感信号列表中的某个输入信号，综合工具通常会推断出_____（填一种不希望出现的电路结构），导致综合前后仿真结果不一致。

三、程序改错题（共 5 分）

题目：以下 VHDL 程序意图设计一个带异步复位和时钟使能的 D 触发器，但代码中存在 5 处错误。请逐一指出错误位置并写出正确的修改方案。

Listing 9.1: 带错误的 D 触发器程序（找出 5 处错误，每处 1 分）

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY dff_ce IS
5      PORT(
6          clk  : IN  STD_LOGIC;
7          rst  : IN  STD_LOGIC;
8          ce   : IN  STD_LOGIC;
9          d    : IN  STD_LOGIC;
10         q    : OUT STD_LOGIC
11     );
12 END dff_ce;
13
14 ARCHITECTURE behav OF dff_ce IS
15 BEGIN
16     PROCESS(clk)
17     BEGIN
18         IF rst = '1' THEN
19             q <= '0';
20         ELSIF clk'EVENT AND clk = '1' THEN
21             IF ce = '1' THEN
22                 q := d;
23             ELSE
24                 q <= q;
25             END IF;
26         END IF;
27     END PROCESS;
28 END behav;
```

答题要求：在答题纸上按以下格式依次写出各错误：

错误 1（第 ____ 行）：_____ 改为 _____（原因：_____）

错误 2(第 ____ 行):_____ 改为 _____(原因:_____)
错误 3(第 ____ 行):_____ 改为 _____(原因:_____)
错误 4(第 ____ 行):_____ 改为 _____(原因:_____)
错误 5(第 ____ 行):_____ 改为 _____(原因:_____)

四、程序阅读分析题（共 5 分）

题目：阅读以下 VHDL 程序，回答下列问题。

Listing 9.2: 待分析 VHDL 程序——分频器

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY clk_div IS
6      GENERIC (N : INTEGER := 4);
7      PORT(
8          clk_in  : IN  STD_LOGIC;
9          rst     : IN  STD_LOGIC;
10         clk_out : OUT STD_LOGIC
11     );
12 END clk_div;
13
14 ARCHITECTURE behav OF clk_div IS
15     SIGNAL count : INTEGER RANGE 0 TO 15 := 0;
16     SIGNAL temp  : STD_LOGIC := '0';
17 BEGIN
18     PROCESS(clk_in, rst)
19     BEGIN
20         IF rst = '1' THEN
21             count <= 0;
22             temp  <= '0';
23         ELSIF rising_edge(clk_in) THEN
24             IF count = N-1 THEN
25                 count <= 0;
26                 temp  <= NOT temp;
27             ELSE
28                 count <= count + 1;
29             END IF;
30         END IF;
31     END PROCESS;
32     clk_out <= temp;
33 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？
- (2) (1 分) GENERIC 参数 N 的作用是什么？当 N=4 时，输出信号 `clk_out` 的频率与输入时钟 `clk_in` 的频率之间存在怎样的关系？
- (3) (1 分) 信号 `count` 和 `temp` 的作用分别是什么？
- (4) (1 分) 该电路中的复位 `rst` 是同步复位还是异步复位？请结合代码说明判断依据。
- (5) (1 分) 若将第 24 行的 `count <= count + 1` 改为 `count <= count + 2`，且 N=4，输出的 `clk_out` 频率将如何变化？请说明理由。

五、综合设计题（共 3 题，共 60 分）

设计题 1：BCD 码—七段数码管译码器（20 分）

设计一个 BCD 码（0-9）到七段共阳极数码管的译码器。输入为 4 位 BCD 码 `bcd(3 downto 0)`，输出为七段控制信号 `seg(6 downto 0)`，分别对应数码管的 a、b、c、d、e、f、g 段（`seg(6)` 对应 a 段，`seg(0)` 对应 g 段）。共阳极数码管的段码为低电平点亮（0=亮，1=灭）。要求非法输入（10-15）时所有段熄灭。

十进制	bcd(3:0)	a	b	c	d	e	f	g
0	0000	亮	亮	亮	亮	亮	亮	灭
1	0001	灭	亮	亮	灭	灭	灭	灭
2	0010	亮	亮	灭	亮	亮	灭	亮
3	0011	亮	亮	亮	亮	灭	灭	亮
4	0100	灭	亮	亮	灭	灭	亮	亮
5	0101	亮	灭	亮	亮	灭	亮	亮
6	0110	亮	灭	亮	亮	亮	亮	亮
7	0111	亮	亮	亮	灭	灭	灭	灭
8	1000	亮	亮	亮	亮	亮	亮	亮
9	1001	亮	亮	亮	亮	灭	亮	亮

图 9.1: 共阳极数码管段码显示对照表（亮 = 段点亮，灭 = 段熄灭）

要求：

- (1)（5 分）根据上述段码显示对照表，写出完整的七段码真值表（bcd 输入对应 seg 输出的二进制值，0 表示点亮，1 表示熄灭）。
- (2)（15 分）使用数据流描述风格编写该译码器的完整 VHDL 代码。要求：
 - 使用 WITH...SELECT 语句或 WHEN...ELSE 条件信号赋值语句实现译码逻辑；
 - 代码包含完整的库声明、实体定义和结构体；
 - 对非法输入（10-15）做安全处理，使所有段熄灭（`seg <= "1111111"`）。

设计题 2：4 位双向移位寄存器（20 分）

设计一个 4 位双向移位寄存器，支持左移、右移、并行置数和保持四种操作，采用结构化 VHDL 描述（即底层使用 D 触发器作为基本单元，通过 COMPONENT 与 PORT MAP 搭建顶层电路）。

端口定义：

- `clk`（输入）：时钟信号，上升沿触发
- `rst`（输入）：异步复位，高电平有效，复位时 `q <= "0000"`
- `mode(1 downto 0)`（输入）：操作模式选择
 - "00"：保持（数据不变）
 - "01"：右移（数据从高位向低位移动，`q(3)` 移入串行输入 `si_r`）
 - "10"：左移（数据从低位向高位移动，`q(0)` 移入串行输入 `si_l`）
 - "11"：并行置数（`q <= din`）
- `din(3 downto 0)`（输入）：并行置数数据
- `si_r`（输入）：右移时的串行输入（移入最高位）
- `si_l`（输入）：左移时的串行输入（移入最低位）
- `q(3 downto 0)`（输出）：当前移位寄存器的值

要求：

- (1)（4 分）画出顶层模块的端口框图，标注所有输入输出信号及位宽。
- (2)（3 分）画出单个 D 触发器（带时钟使能和异步复位）的符号框图，作为底层基本单元。该 D 触发器的端口为：`clk`、`rst`、`d`、`q`（为使能端设计留作后续步骤处理）。
- (3)（10 分）编写完整的 VHDL 代码，采用结构化描述风格：
 - 首先编写单个 D 触发器（带异步复位）的 ENTITY 和 ARCHITECTURE；
 - 在顶层模块中使用 COMPONENT 声明 D 触发器，并使用 FOR...GENERATE 语句实例化 4 个 D 触发器；

- 每个 D 触发器的 d 输入端由一个 4 选 1 多路选择器 (MUX) 驱动, 根据 mode 信号选择输入源: 保持时选自身 q、右移时选左侧触发器输出或 si_r、左移时选右侧触发器输出或 si_l、置数时选 din 对应位;
- MUX 逻辑可以在顶层用组合逻辑描述 (如条件信号赋值语句), 不必也做成 COMPONENT。

(4) (3 分) 在代码中添加清晰的注释, 说明各模块的功能和数据流向。

提示:

- 4 位寄存器从高位到低位编号为 q(3) (最左)、q(2)、q(1)、q(0) (最右)。
- 右移 ("01") 时: q(3) <= si_r, q(2) <= q(3), q(1) <= q(2), q(0) <= q(1)。
- 左移 ("10") 时: q(0) <= si_l, q(1) <= q(0), q(2) <= q(1), q(3) <= q(2)。
- GENERATE 语句示例: G1: FOR i IN 0 TO 3 GENERATE ... END GENERATE;

设计题 3：自动售货机控制器（20 分）

设计一个简单的自动售货机控制器状态机。该售货机只接收 1 元和 2 元硬币（每次只能投入一枚），商品价格为 3 元，不设找零（多投不退）。

功能说明：

- 硬币用两位输入信号 `coin(1:0)` 表示："00" 表示无硬币投入，"01" 表示投入 1 元硬币，"10" 表示投入 2 元硬币（"11" 为非法，按无投入处理）。每个时钟周期最多检测一枚硬币的投入。
- 状态机追踪已投入的累计金额（0、1、2、3 元）。
- 当累计金额达到或超过 3 元时，在下一个时钟周期输出 `dispense = '1'`（出货，持续一个时钟周期），同时状态机返回初始状态（0 元）。
- 输出信号 `dispense`（1 位）：出货信号，高电平有效。
- 输出信号 `state_disp(1:0)`（2 位）：当前状态的编码输出（可选，便于调试）。

端口定义：

- `clk`（输入）：时钟信号
- `rst`（输入）：异步复位，高电平有效，复位后状态回到初始状态（累计金额 = 0 元）
- `coin(1:0)`（输入）：硬币检测信号
- `dispense`（输出）：出货信号
- `state_disp(1:0)`（输出）：当前累计金额的状态编码（0 元 = "00"，1 元 = "01"，2 元 = "10"，3 元 = "11"）

要求：

(1)（7 分）画出该售货机控制器的状态转移图（Mealy 型或 Moore 型均可，但须标明类型）。要求：

- 明确标注所有状态名称及含义；
- 标注状态转移条件（输入）和输出值；
- 说明采用的是 Mealy 型还是 Moore 型状态机及其原因。

(2)（3 分）列出状态转移表，包含当前状态、输入（`coin`）、次态和输出（`dispense`）。

(3) (10 分) 编写完整的 VHDL 代码实现该控制器。要求:

- 使用用户自定义枚举类型定义状态(如 `TYPE state_type IS (S0, S1, S2, S3)`);
- 采用双进程描述风格: 一个时序进程(状态寄存器和复位), 一个组合进程(次态逻辑和输出逻辑);
- 异步复位(`rst` 为高电平时立即回到初始状态);
- 代码中包含必要的注释。

提示: 投币行为示例(假设每个时钟周期投一枚硬币): 时钟周期 1: `coin = "01"` (投 1 元) → 累计 1 元; 时钟周期 2: `coin = "10"` (投 2 元) → 累计 3 元 → 触发出货, 下一周期回到 0 元; 时钟周期 3: `coin = "01"` (投 1 元) → 累计 1 元; ...

——试卷结束——

第十章 第 10 套试卷

FPGA 与 VHDL 基础试卷（十）

（满分：100 分 考试时间：120 分钟 难度：中等）

一、选择题（每题 3 分，共 18 分）

1. FPGA 上电后需要从外部存储器加载配置数据（比特流）。以下哪一项**不是**常见的 FPGA 配置模式？
 - A. 主串模式（Master Serial）
 - B. 从串模式（Slave Serial）
 - C. JTAG 模式
 - D. UART 串口通信模式
2. 关于 CPLD 与 FPGA 在工艺方面的典型区别，以下描述正确的是？
 - A. CPLD 通常采用 Flash/EEPROM 工艺（非易失性），FPGA 通常采用 SRAM 工艺（易失性）
 - B. CPLD 通常采用 SRAM 工艺（易失性），FPGA 通常采用 Flash/EEPROM 工艺（非易失性）
 - C. 两者均采用 SRAM 工艺
 - D. 两者均采用反熔丝工艺
3. 在 VHDL 中，关于并发语句与顺序语句的描述，**正确**的是？
 - A. PROCESS 内部的语句是并发执行的
 - B. 并发信号赋值语句可以放在 PROCESS 内部

- C. 顺序语句（如 IF、CASE）只能出现在 PROCESS 或子程序内部
 - D. 结构体的 BEGIN...END 之间可以直接使用 IF 语句
4. 关于 Moore 型与 Mealy 型有限状态机，以下说法正确的是？
- A. Moore 型状态机的输出仅与当前状态有关
 - B. Mealy 型状态机的输出仅与当前状态有关
 - C. Moore 型与 Mealy 型的输出逻辑完全相同
 - D. Moore 型状态机的输出与当前输入和状态均有关
5. VHDL 中，逻辑运算符具有不同的优先级。表达式 `A AND B OR C` 的运算结果是？
- A. `(A AND B) OR C`（从左到右结合）
 - B. `A AND (B OR C)`（OR 优先级高于 AND）
 - C. 取决于综合工具的实现
 - D. 该表达式在 VHDL 中语法错误，必须使用括号
6. 在 VHDL 中，关于 GENERIC 关键字的描述，错误的是？
- A. GENERIC 用于向实体传递静态参数（如位宽、计数值等）
 - B. GENERIC 在 ENTITY 声明中定义，位于 PORT 声明之前
 - C. GENERIC 的值在仿真运行过程中可以被动态修改
 - D. 使用 GENERIC 可以实现参数化设计，提高代码复用性

二、填空题（每题 3 分，共 12 分）

1. VHDL 中三种重要对象的声明位置不同：SIGNAL 在_____（结构体的声明区）中声明；VARIABLE 只能在_____（PROCESS 或子程序内部）声明；CONSTANT 可在_____中声明。
2. 使用 GENERIC 实现参数化设计时，在 ENTITY 中声明 GENERIC 的语法为：
GENERIC (参数名 : 数据类型 _____ 默认值);
(填一个赋值符号，由冒号和等号组成)。
3. 在结构体 (ARCHITECTURE) 中，实例化一个已声明的元件 (COMPONENT) 使用的语句是_____。
4. VHDL 中两种并发生成语句分别是_____ GENERATE 语句 (用于重复实例化) 和_____ GENERATE 语句 (用于条件实例化)。

三、程序改错题（共 5 分）

下面的 VHDL 代码意图设计一个带异步复位的 4 位二进制加法计数器，但代码中存在 5 处错误。请指出每处错误的位置（行号）并写出正确的修改方案。

Listing 10.1: 带错误的 4 位计数器

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3                                     -- 缺少算术运算库声明
4  ENTITY cnt4_err IS
5      PORT(
6          clk, rst : IN  STD_LOGIC;          -- rst: 低电平异步复位
7          q       : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
8      );
9  END cnt4_err;
10
11 ARCHITECTURE behav OF cnt4_err IS
12     SIGNAL count : STD_LOGIC_VECTOR(3 DOWNTO 0);
13 BEGIN
14     PROCESS(clk)                          -- 敏感信号表不完整
15     BEGIN
16         IF rst = '0' THEN                -- 异步复位逻辑
17             count := "0000";              -- 赋值符号错误
18         ELSIF rising_edge(clk) THEN
19             count <= count + "0001";      -- 算术运算符使用不当
20         END IF;
21     END PROCESS;
22
23     q := count;                            -- 并发赋值符号错误
24 END behav;

```

答题要求：按以下格式逐条写出每个错误的位置、原因和正确写法（每处 1 分，共 5 分）。

错误 1（第 ____ 行）：_____ 改为 _____

错误 2（第 ____ 行）：_____ 改为 _____

错误 3（第 ____ 行）：_____ 改为 _____

错误 4（第 ____ 行）：_____ 改为 _____

错误 5（第 ____ 行）：_____ 改为 _____

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 10.2: 分析用 VHDL 代码

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY freq_div8 IS
6      PORT(
7          clk      : IN  STD_LOGIC;
8          rst      : IN  STD_LOGIC;
9          clk_out   : OUT STD_LOGIC
10     );
11 END freq_div8;
12
13 ARCHITECTURE rtl OF freq_div8 IS
14     SIGNAL cnt : UNSIGNED(2 DOWNT0 0);
15 BEGIN
16     PROCESS(clk, rst)
17     BEGIN
18         IF rst = '1' THEN
19             cnt <= (OTHERS => '0');
20         ELSIF rising_edge(clk) THEN
21             cnt <= cnt + 1;
22         END IF;
23     END PROCESS;
24
25     clk_out <= cnt(2);
26 END rtl;
```

问题：

- (1) 该 VHDL 代码实现的是什么电路？其功能是什么？（1 分）
- (2) 若输入时钟clk的频率为 8 MHz，输出clk_out的频率是多少？请说明计算依据。（1 分）

- (3) 信号cnt的位宽是多少？计数范围是多少？（1 分）
- (4) 该电路的复位信号是同步复位还是异步复位？请从代码中找到判断依据并说明。（1 分）
- (5) 若要将分频比从 8 改为 32（即 32 分频），代码需要做哪些修改？（1 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——BCD 码至七段数码管译码器（20 分）

设计一个 BCD 码至七段数码管译码器。输入为 4 位 BCD 码 `bcd(3 DOWNTO 0)`（仅 0–9 有效，10–15 不会出现），输出为七段数码管驱动信号 `seg(6 DOWNTO 0)`，对应段位分配为：`seg(6)=a`、`seg(5)=b`、`seg(4)=c`、`seg(3)=d`、`seg(2)=e`、`seg(1)=f`、`seg(0)=g`。输出低电平有效（共阳极数码管），即段亮时为'0'，灭时为'1'。

要求：

- (1)（5 分）列出完整的真值表（输入 0–9，输出 7 位段码）。
- (2)（3 分）画出顶层模块的端口框图，标注所有输入输出端口名称、方向和位宽。
- (3)（10 分）用 VHDL 编写完整的实体和结构体代码，要求使用选择信号赋值语句（WITH-SELECT-WHEN）实现组合逻辑功能。当输入为无效 BCD 码（10–15）时，所有段均熄灭（输出全'1'）。
- (4)（2 分）代码规范：命名合理、缩进清晰、注释完整。

提示：共阳极数码管段码（低有效）参考值——显示“0”：“1000000”（a–f 亮，g 灭），“1”：“1111001”，“2”：“0100100”，以此类推。`seg(6 DOWNTO 0) = {a, b, c, d, e, f, g}`。

设计题 2：时序逻辑设计——带预置数的 8 位双向移位寄存器（20 分）

设计一个 8 位双向移位寄存器，支持左移（向高位移动）和右移（向低位移动），并具有同步预置数功能。

端口定义：

- `clk`（输入）：时钟信号，上升沿触发
- `rst`（输入）：同步复位，高电平有效，复位时所有输出位清零
- `dir`（输入）：移位方向控制，`dir='1'`时左移，`dir='0'`时右移
- `load`（输入）：预置数使能，高电平有效
- `data_in(7 DOWNTO 0)`（输入）：预置数并行输入
- `si`（输入）：串行移位输入（右移时移入最高位 MSB，左移时移入最低位 LSB）

- `q(7 DOWNT0 0)` (输出): 当前移位寄存器的状态

要求:

- (1) (5 分) 画出该移位寄存器的功能框图 (端口图), 并写出其简化功能表 (含复位、预置、左移、右移四种操作)。
- (2) (10 分) 用 VHDL 编写完整的实体和结构体代码, 使用 PROCESS 实现。各控制信号的优先级为: 同步复位 > 预置数 > 移位操作。
- (3) (5 分) 若将此 8 位移位寄存器的每一位用一个 1 位 D 触发器 (带使能端) 元件级联实现, 请使用 FOR GENERATE 语句写出结构描述风格的 VHDL 代码 (要求: 先声明一个 DFF_E 的 COMPONENT, 然后使用 FOR GENERATE 按照移位方向连接各触发器)。

提示: 右移时, `q(6 DOWNT0 0)` 移入 `q(7 DOWNT0 1)`, `q(7)` 移入 `si`; 左移时, `q(7 DOWNT0 1)` 移入 `q(6 DOWNT0 0)`, `q(0)` 移入 `si`。

设计题 3: 状态机设计——“1011”序列检测器 (Moore 型) (20 分)

设计一个 Moore 型有限状态机, 用于检测串行输入序列 “1011”。每个时钟上升沿采样一位输入 `din`。当连续检测到 “1011” 序列时, 输出 `found` 输出 ‘1’ (持续一个时钟周期), 其余时间输出 ‘0’。允许序列重叠检测 (即 “101011” 中应检测出一次 “1011”——最后四位)。

示例: 若输入序列为 `...0 1 0 1 1 0 1 1 0...`, 则第 5 个时钟周期 (检测到完整的 “1011”) `found=1`, 第 8 个时钟周期再次 `found=1`。

要求:

- (1) (6 分) 画出完整的 Moore 型状态转移图, 标注每个状态的名称 (及含义)、状态编码 (自选编码方案)、状态转移条件和各状态的输出值 (`found`)。
建议使用 5 个状态: `S0` (初始/未匹配)、`S1` (已匹配 “1”)、`S2` (已匹配 “10”)、`S3` (已匹配 “101”)、`S4` (已匹配 “1011”——检测成功)。
- (2) (3 分) 列出状态转移表, 包含: 当前状态、输入 `din`、次态、输出 `found`。
- (3) (9 分) 用 VHDL 编写完整的实体和结构体代码。要求:
 - 使用用户自定义枚举类型定义状态 (`TYPE state_type IS (S0, S1, S2, S3, S4);`);
 - 采用双进程 (或三进程) 描述风格: 一个时序进程描述状态寄存器, 一个组合进程描述次态逻辑和输出逻辑;

- 时钟上升沿触发，复位`rst`为同步复位（高电平有效），复位后回到 S0 状态。

(4) (2 分) 代码规范：状态编码明确、注释清晰、进程划分合理。

提示： Moore 型状态机的输出仅取决于当前状态，与输入无关。注意处理重叠检测：例如在 S3（已匹配“101”）状态下，若输入 `din=1` 则进入 S4（检测成功）；若输入 `din=0`，则已匹配的后缀“10”可视为新检测序列的前缀，应回到 S2 状态。

——试卷结束——

第十一章 第 11 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列关于 CPLD 宏单元（Macrocell）结构的描述，正确的是？
 - A. CPLD 宏单元仅包含一个 D 触发器，不含任何组合逻辑
 - B. CPLD 宏单元中的乘积项阵列用于产生组合逻辑，输出可配置为组合或寄存器输出
 - C. CPLD 宏单元和 FPGA 的逻辑单元（LE）结构完全相同
 - D. CPLD 宏单元只能实现纯组合逻辑，无法实现时序逻辑
2. 目前主流的 SRAM 型 FPGA 在上电后，其配置数据通常存储在何处？
 - A. FPGA 内部的 Flash 存储器中
 - B. FPGA 内部的 SRAM 配置存储器中（掉电后丢失，需外部配置芯片重新加载）
 - C. FPGA 内部的 EEPROM 中
 - D. 无需配置，FPGA 上电后自动生成逻辑功能
3. 在 VHDL 中，以下哪种语句**不属于**并发语句（Concurrent Statement）？
 - A. 进程语句（PROCESS）
 - B. 并发信号赋值语句（Concurrent Signal Assignment）
 - C. 元件例化语句（PORT MAP）
 - D. 变量赋值语句（Variable Assignment — :=）

4. Moore 型状态机与 Mealy 型状态机的主要区别是？
- A. Moore 型状态机的输出只与当前状态有关；Mealy 型的输出与当前状态和输入均有关
 - B. Moore 型状态机需要更多状态；Mealy 型一定更简单
 - C. Mealy 型状态机只能检测序列，Moore 型只能控制输出
 - D. Moore 型状态机的输出比 Mealy 型延迟一个时钟周期，这是二者的定义性区别
5. 在 VHDL 中，逻辑运算符 AND、OR、NOT 三者的优先级从高到低依次为？
- A. AND > OR > NOT
 - B. NOT > AND > OR
 - C. NOT > OR > AND
 - D. OR > AND > NOT
6. 一个 n 输入的查找表（LUT），其内部 SRAM 存储单元的数量为？
- A. n 个
 - B. $2n$ 个
 - C. 2^n 个
 - D. n^2 个

二、填空题（每空 3 分，共 12 分）

1. VHDL 中，类属参数使用关键字_____声明；在结构体中例化一个已定义的元件时，使用_____和_____两条语句。
2. 在 VHDL 中，SIGNAL 可以在_____和_____中声明，但不能在 PROCESS 语句体内部声明；VARIABLE 仅允许在_____或子程序内部声明。
3. VHDL 中，用于生成重复结构的语句是_____，它分为两种形式：_____GENERATE（按条件生成）和_____GENERATE（按循环生成）。

4. 在 VHDL 中, SIGNAL 的信号赋值使用符号_____, VARIABLE 的变量赋值使用符号_____。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码（意图为实现一个带异步复位的 4 位加法计数器），找出其中的 5 处错误，并写出正确的修改方案。

Listing 11.1: 存在错误的 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY counter4 IS
5      PORT(
6          clk    : IN    STD_LOGIC;
7          rst    : IN    STD_LOGIC;
8          q      : OUT  STD_LOGIC_VECTOR(3 DOWNT0 0)
9      );
10 END counter4;
11
12 ARCHITECTURE behav OF counter4 IS
13     SIGNAL cnt : STD_LOGIC_VECTOR(3 DOWNT0 0);
14 BEGIN
15     PROCESS(clk)
16     BEGIN
17         IF rst = '0' THEN
18             cnt := (OTHERS => '0');
19         ELSIF rising_edge(clk) THEN
20             cnt <= cnt + 1;
21         END IF;
22     END PROCESS;
23     q <= cnt;
24 END behav;
```

答题要求：逐行列出错误位置、错误原因及正确写法（每处 1 分，共 5 分）。
提示：错误涉及：全角分号（；→;）、敏感信号表不完整（异步复位信号未列入）、信号-变量赋值混用（:= 与 <=）、包引用缺失（STD_LOGIC_VECTOR 算术运算）、端口声明语法。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 11.2: 分析用 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY ring_counter IS
5      PORT(
6          clk, rst : IN  STD_LOGIC;
7          q       : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)
8      );
9  END ring_counter;
10
11 ARCHITECTURE behavioral OF ring_counter IS
12     SIGNAL sr : STD_LOGIC_VECTOR(7 DOWNT0 0);
13 BEGIN
14     PROCESS(clk, rst)
15     BEGIN
16         IF rst = '0' THEN
17             sr <= "00000001";
18         ELSIF rising_edge(clk) THEN
19             sr <= sr(0) & sr(7 DOWNT0 1);
20         END IF;
21     END PROCESS;
22     q <= sr;
23 END behavioral;
```

问题：

- (1) 该 VHDL 代码描述的是什么电路？（1 分）
- (2) 复位后输出 q 的初始值是什么？经过 3 个时钟上升沿后，q 的值是什么？（2 分）
- (3) 该电路经过几个时钟周期后输出会回到初始值？（即周期是多少）（1 分）
- (4) 该电路是组合逻辑还是时序逻辑？判断依据是什么？（1 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——BCD 码转 7 段数码管译码器（20 分）

设计一个 BCD 码转 7 段数码管译码器，用于将 4 位 BCD 码输入转换为共阴极 7 段数码管的段选信号。要求：

- (1) **端口定义（4 分）**：输入 `bcd(3 downto 0)`，输出 `seg(6 downto 0)`（分别对应 a~g 段，`seg(6)=a`, `seg(5)=b`, ..., `seg(0)=g`，高电平点亮）。还包括一个使能输入 `en`：当 `en='0'` 时所有段熄灭（全 0）；当 `en='1'` 时正常译码显示。
- (2) **真值表（5 分）**：列出输入 BCD 码（0~9）与 7 段输出之间的对应关系真值表（段码 a~g 按顺序列出，无效输入 10~15 时全部熄灭）。
- (3) **VHDL 代码（8 分）**：编写完整的 VHDL 代码（实体 + 结构体），使用数据流描述方式（条件信号赋值语句 `WHEN-ELSE` 或选择信号赋值语句 `WITH-SELECT-WHEN`）实现译码逻辑。
- (4) **代码规范（3 分）**：端口类型选择合理，信号命名规范，注释清晰。

提示：共阴极 7 段数码管编码（高电平点亮，`seg(6 downto 0)` 对应 a b c d e f g）：

数字 0→abcdef 亮 →"1111110"，数字 1→bc 亮 →"0110000"，数字 2→abdeg 亮 →"1101101"，

数字 3→abcdg 亮 →"1111001"，数字 4→bcfg 亮 →"0110011"，数字 5→acdfg 亮 →"1011011"，

数字 6→acdefg 亮 →"1011111"，数字 7→abc 亮 →"1110000"，数字 8→全亮 →"1111111"，

数字 9→abcdfg 亮 →"1111011"。以 a 为最高位 (`seg(6)`)，g 为最低位 (`seg(0)`)。

设计题 2：时序逻辑设计——级联分频器（20 分）

设计一个基于 D 触发器的级联 2 分频器链，将输入时钟进行 2^N 分频。具体而言，设计一个由 8 个 D 触发器级联构成的 8 级 2 分频器，每级输出频率为前一级的一半。要求使用结构体描述方式（元件例化 `COMPONENT + PORT MAP`）配合 `FOR-GENERATE` 语句实现。

- (1) **底层模块设计（6 分）**：首先设计一个带异步复位的 D 触发器 (`dff_div`)：输入 `clk`、`rst`（低电平有效），输出 `q` 和 `qn`（反相输出）。在时钟上升沿， $q \leq qn$ （即 $Q_{n+1} = \overline{Q_n}$ ），形成 2 分频。编写该底层模块的完整 VHDL 代码。

- (2) **顶层模块设计 (10 分)**: 设计顶层模块 `div_chain`, 使用 `GENERIC` 定义级数 $N=8$ 。采用 `COMPONENT` 声明 `dff_div`, 然后使用 `FOR-GENERATE` 语句生成 8 级级联分频链。每一级的 `q` 输出连接到下一级的 `clk` 输入; 第一级的 `clk` 接外部时钟。所有级的 `rst` 并联接外部复位。输出端口 `q_out(7 downto 0)` 分别输出每一级的 `q` 信号。
- (3) **设计思路 (4 分)**: 在代码注释中说明分频原理, 以及为什么选择 `GENERATE` 语句来实现该设计。

提示: D 触发器的 2 分频原理: $Q_{n+1} = \overline{Q_n}$, 即每来一个时钟上升沿输出翻转一次, 输出频率为输入时钟的 $1/2$ 。8 级级联后, 第 k 级输出频率为输入时钟的 $1/2^k$ 。请使用 `STD_LOGIC` 和 `STD_LOGIC_VECTOR` 类型。

设计题 3: 状态机设计——自动售货机控制器 (20 分)

设计一个 Moore 型状态机, 控制一个简易自动售货机。该售货机只接受 1 元和 2 元两种硬币 (不考虑找零), 商品单价为 3 元。当投入的硬币总额达到或超过 3 元时, 售货机输出出货信号并退回多余金额 (但不找零, 即投入 4 元时出货但不找零)。

- (1) **功能说明 (3 分)**: 输入信号: `coin_1` (投入 1 元的脉冲信号, 高有效, 持续 1 个时钟周期)、`coin_2` (投入 2 元的脉冲信号, 高有效, 持续 1 个时钟周期)。两个信号不会同时有效。输出信号: `dispense` (出货, 高有效, 持续 1 个时钟周期)。时钟 `clk` 上升沿触发, 异步复位 `rst` 高电平有效。
- (2) **状态转移图 (6 分)**: 画出 Moore 型状态机的状态转移图, 标注所有状态的名称、含义、当前已投金额及输出值 (`dispense`)。状态定义应清晰表示已投入的金额 (0 元、1 元、2 元、3 元及以上)。状态总数应不超过 5 个。
- (3) **状态编码与状态转移表 (3 分)**: 给出状态编码方案, 并列出状态转移表 (当前状态、输入、次态、输出)。
- (4) **VHDL 代码 (8 分)**: 编写完整的 VHDL 代码 (实体 + 结构体), 采用双进程 (一个时序进程描述状态寄存器, 一个组合进程描述次态逻辑和输出逻辑) 或三进程描述风格。使用用户自定义枚举类型定义状态。

提示: 注意: 投入 2 元硬币后, 已投金额 3 元 (假设此前至少已投 1 元), 应立即出货并回到初始状态。投入两个 2 元硬币 (共 4 元) 也应出货, 但不找零。务必考虑 `coin_1` 和 `coin_2` 不会同时为 '1' 的前提。

第十二章 第 12 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列关于 FPGA 配置方式的描述，正确的是？

- A. 基于 SRAM 工艺的 FPGA，配置数据存储在内部 Flash 中，掉电不丢失
- B. 基于 SRAM 工艺的 FPGA 上电后需从外部配置存储器（如 SPI Flash）加载比特流，配置数据掉电丢失
- C. FPGA 只支持 JTAG 一种配置方式
- D. CPLD 和 FPGA 的配置原理完全相同，均基于 SRAM 工艺

2. 以下关于 FPGA 主流配置模式的描述，错误的是？

- A. 主串模式（Master Serial）由 FPGA 主动从外部串行 PROM 读取配置数据
- B. 从串模式（Slave Serial）由外部控制器向 FPGA 串行传输配置数据
- C. JTAG 模式主要用于调试和在线编程，也可用于配置 FPGA
- D. USB 配置模式是 FPGA 最基础、最常用的配置模式，所有 FPGA 均支持

3. 在 VHDL 中，下列关于并发语句与顺序语句的说法，正确的是？

- A. PROCESS 内部的 IF 语句和 CASE 语句是并发执行的
- B. 结构体（ARCHITECTURE）中，PROCESS 外部的信号赋值语句（如 $y \leq a \text{ AND } b$ ）是顺序执行的
- C. PROCESS 内部的语句按书写顺序依次执行，多个 PROCESS 之间是并发关系
- D. 变量赋值语句（:=）可以出现在结构体中的任何位置

4. Moore 型与 Mealy 型状态机在输出时序上的典型差异是？

- A. Moore 型输出比 Mealy 型输出晚一个时钟周期, 这是由两者的输出逻辑结构决定的
- B. Mealy 型输出总是比 Moore 型输出晚一个时钟周期
- C. 两者输出时序完全相同, 仅在状态编码方式上有差异
- D. Moore 型输出与时钟无关, Mealy 型输出与时钟同步
5. 在 VHDL 中, 逻辑运算符 NOT、AND、OR、XOR 四者的优先级从高到低排列正确的是?
- A. AND > OR > NOT > XOR
- B. NOT > AND > OR > XOR
- C. NOT 最高, AND/OR/XOR 三者优先级相同且低于 NOT
- D. NOT > AND > XOR > OR
6. 在 VHDL 结构描述风格 (Structural Description) 中, 以下哪组关键字是必不可少的?
- A. VARIABLE 和 SIGNAL
- B. COMPONENT 和 PORT MAP
- C. FUNCTION 和 PROCEDURE
- D. TYPE 和 SUBTYPE

二、填空题（每空 3 分，共 12 分）

1. 在 VHDL 中，**SIGNAL**（信号）通常在_____中声明，用于模块间的互连与通信，其赋值符号为_____。
VARIABLE（变量）只能在_____内部或子程序（FUNCTION/PROCEDURE）内部声明，其赋值符号为_____。
CONSTANT（常量）的值在仿真运行期间_____（填“可以”或“不可以”）被修改。
2. 在 VHDL 的实体（ENTITY）声明中，_____关键字用于定义可配置的静态参数（如数据位宽 N 、延时值 T_{pd} 等），使同一设计单元可用于不同参数配置。在顶层模块中通过_____将这些参数的实际值传递给被例化的子模块。
3. 在 VHDL 的结构化描述中，需先在结构体的声明区使用_____语句声明待例化的子模块接口，然后使用_____语句将子模块的端口与顶层信号实际连接。
4. VHDL 中的_____语句用于在结构体中按规律重复生成一组相同或相似的硬件结构。该语句的两种形式分别为_____（循环生成）和 IF（条件生成），它们均属于_____语句（填“并发”或“顺序”），只能出现在结构体内部而不能出现在 PROCESS 内部。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码（该代码意图实现一个组合逻辑二选一多路选择器和一个带异步复位的 4 位计数器，但代码中存在 5 处错误），找出错误并写出正确的修改方案。

Listing 12.1: 存在 5 处错误的 VHDL 代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY mixed_err IS
5      PORT(
6          clk, rst, sel : IN  STD_LOGIC;
7          a, b           : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
8          y              : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
9          q              : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
10     );
11 END mixed_err;
12
13 ARCHITECTURE behav OF mixed_err IS
14     SIGNAL cnt  : STD_LOGIC_VECTOR(3 DOWNTO 0);
15     SIGNAL temp : INTEGER RANGE 0 TO 15;
16 BEGIN
17     -- ===== 组合逻辑：二选一多路选择器 =====
18     PROCESS(a)                                     -- 敏感信号列表
19     BEGIN
20         IF sel = '1' THEN
21             y <= a;
22             -- 此处缺少对 sel='0' 情况的处理
23         END IF;
24     END PROCESS;
25
26     -- ===== 时序逻辑：4 位计数器 =====
27     PROCESS(rst)                                     -- 敏感信号列表
28     BEGIN
29         IF rst = '1' THEN
30             cnt := (OTHERS => '0');                 -- 复位操作
31         ELSIF rising_edge(clk) THEN
32             cnt <= cnt + 1;
33         END IF;

```

```
34  END PROCESS;  
35  
36  temp <= cnt;           -- 类型转换  
37  q <= temp;  
38  END behav;
```

答题要求：按以下格式逐条列出 5 处错误（每处 1 分，共 5 分）：

错误 1（第 ____ 行）：_____ 改为 _____（原因：_____）
错误 2（第 ____ 行）：_____ 改为 _____（原因：_____）
错误 3（第 ____ 行）：_____ 改为 _____（原因：_____）
错误 4（第 ____ 行）：_____ 改为 _____（原因：_____）
错误 5（第 ____ 行）：_____ 改为 _____（原因：_____）

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 12.2: 分析用 VHDL 代码——移位寄存器

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY shift_reg IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;
8          din      : IN  STD_LOGIC;
9          q        : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
10     );
11 END shift_reg;
12
13 ARCHITECTURE behav OF shift_reg IS
14     SIGNAL sr : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0000";
15 BEGIN
16     PROCESS(clk)
17     BEGIN
18         IF rising_edge(clk) THEN
19             IF rst = '1' THEN
20                 sr <= (OTHERS => '0');
21             ELSE
22                 sr <= din & sr(3 DOWNTO 1);
23             END IF;
24         END IF;
25     END PROCESS;
26     q <= sr;
27 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？该电路属于组合逻辑还是时序逻辑？请说明判断依据。
- (2) (1 分) 信号 `rst` 是同步复位还是异步复位？请结合代码说明判断依据。

- (3) (2 分) 假设初始状态 `sr` 已清零。若 `din` 依次输入 '1'、'0'、'1'、'1' (每个时钟周期输入 1 位), 写出经过 1、2、3、4 个时钟上升沿后 `sr` 的值分别是多少? (以 4 位二进制形式表示)。
- (4) (1 分) 若将代码第 20 行的 `sr <= din & sr(3 DOWNTO 1)` 改为 `sr <= sr(2 DOWNTO 0) & din`, 移位方向发生什么变化? 此时若 `din` 固定为 '0', 经过 4 个时钟周期后 `sr` 的值将变成什么?

五、综合设计题（共 60 分）

设计题 1：4 位算术逻辑单元（ALU）设计（20 分）

设计一个 4 位算术逻辑单元（ALU），可对两个 4 位输入操作数执行 4 种运算。要求使用数据流描述风格（条件信号赋值语句或选择信号赋值语句）实现。

端口定义：

- `a(3 downto 0)`、`b(3 downto 0)`（输入）：两个 4 位操作数（均为无符号二进制数）。
- `op(1 downto 0)`（输入）：操作码，用于选择运算类型，定义如下：
 - "00" — 加法 ($a + b$)
 - "01" — 减法 ($a - b$)
 - "10" — 按位与 ($a \text{ AND } b$)
 - "11" — 按位或 ($a \text{ OR } b$)
- `result(4 downto 0)`（输出）：5 位运算结果。加法/减法时最高位为进位/借位标志；逻辑运算时最高位填 '0'。
- `zero`（输出）：零标志位，当 `result(3 downto 0) = "0000"` 时输出 '1'，否则输出 '0'。

要求：

- (1)（4 分）画出该 ALU 的顶层模块端口框图，标注所有端口名称、方向和位宽。
- (2)（3 分）列出该 ALU 的功能表（包含 `op` 的四种取值对应的运算功能、输出表达式以及 `result` 各位的来源）。
- (3)（10 分）使用数据流描述风格编写完整的 VHDL 代码（实体 + 结构体）。要求：
 - 使用 `WITH...SELECT` 语句或 `WHEN...ELSE` 条件信号赋值语句实现运算选择；
 - 使用 `IEEE.NUMERIC_STD.ALL` 包中的类型转换函数处理加减法（例如 `UNSIGNED` 与 `STD_LOGIC_VECTOR` 之间的转换）；
 - 单独用一个并发信号赋值语句生成 `zero` 标志位。

- (4) (3 分) 在代码中以注释方式说明：(1) 为什么算术运算和逻辑运算需要不同的处理方式；(2) `result` 的位宽为什么设为 5 位而非 4 位。

提示：

- 加法示例：`result <= STD_LOGIC_VECTOR( ('0' & UNSIGNED(a)) + ('0' & UNSIGNED(b)) );`
- 减法示例：`result <= STD_LOGIC_VECTOR( ('0' & UNSIGNED(a)) - ('0' & UNSIGNED(b)) );`
- 逻辑运算：将两操作数按位进行逻辑操作后，在最高位补 '0' 组成 5 位输出。

设计题 2：参数化模 N 计数器设计（20 分）

设计一个参数化的模 N 同步计数器（计数范围 $0 \sim N - 1$ ），使用 **GENERIC** 参数使计数器模值可配置。该计数器带有同步复位、计数使能和进位输出功能。要求以**行为描述风格**（**PROCESS**）实现。

端口定义：

- **clk**（输入）：时钟信号，上升沿触发。
- **rst**（输入）：同步复位信号，高电平有效。复位时计数值清零。
- **en**（输入）：计数使能信号，高电平有效。**en**='1' 时每个时钟上升沿计数值加 1；**en**='0' 时计数值保持不变。
- **q(3 downto 0)**（输出）：4 位计数值输出（假设 $N \leq 16$ ，若 $N > 16$ 可只输出低 4 位）。
- **tc**（输出）：进位/终端计数（Terminal Count）输出。当计数值为 $N - 1$ 且 **en**='1' 时，**tc** 输出 '1'（持续一个时钟周期）。

要求：

- (1)（3 分）在实体中使用 **GENERIC** 声明参数 N （默认值为 10）。说明 **GENERIC** 在参数化设计中的意义。
- (2)（5 分）画出该模 N 计数器的状态转移图（以 $N = 6$ 为例，画出所有 $0 \sim 5$ 的状态及其转移条件，标注各状态下的 **tc** 输出值）。
- (3)（9 分）编写完整的 VHDL 代码（实体 + 结构体）。要求：
 - **GENERIC** 定义模值 N ，类型为 **INTEGER**，默认值为 10；
 - 内部使用 **INTEGER** 类型的信号（或变量）存储计数值，取值范围 $0 \sim N - 1$ ；
 - 在 **PROCESS** 中描述同步计数逻辑：同步复位优先级最高，其次是使能控制；
 - 使用 **rising_edge(clk)** 检测时钟上升沿；
 - 计数值达到 $N - 1$ 后，下一个有效计数脉冲使其回到 0，同时 **tc** 输出 '1'。
- (4)（3 分）回答以下问题：
 - (a) 若将模值 N 改为 60（ $N = 60$ ），4 位输出的 **q** 端口是否还能完整表示计数值？如果不能，应如何修改端口位宽？

- (b) 同步复位与异步复位在此计数器设计中各有什么优缺点？如果改用异步复位，代码需要做哪些修改？

提示：

- 内部计数信号可使用 `INTEGER RANGE 0 TO N-1` 类型，输出时转换为 `STD_LOGIC_VECTOR`。
- 终端计数 `tc` 在计数值等于 $N - 1$ 时置 '1'，否则为 '0'。可在 `PROCESS` 内部用 `IF` 语句实现，也可在 `PROCESS` 外部用并发信号赋值语句实现。
- 同步复位仅在时钟上升沿到达时生效；异步复位不依赖时钟，一旦复位信号有效立即生效。

设计题 3：电子密码锁控制器（20 分）

设计一个电子密码锁控制器状态机。用户通过一个串行输入端口逐位输入 4 位二进制密码，当时钟上升沿采样到正确的 4 位密码序列时，开锁信号有效。密码为固定值“1101”（即依次输入 1、1、0、1）。

功能说明：

- 输入信号 `din`（1 位）：每个时钟周期输入密码的 1 位（‘0’ 或 ‘1’）。
- 系统在每个时钟上升沿采样 `din`，并与预设密码逐位比对。
- 若连续 4 位输入恰好为“1101”，则输出 `unlock = `1'`（开锁），持续一个时钟周期后回到初始状态。
- 若在输入过程中任何一位不匹配，状态机立即回到初始状态（IDLE），用户需重新输入。
- 支持非重叠检测：例如输入序列“1101101”中，第 1-4 位为“1101”（开锁一次），之后状态机回到初始状态重新检测。
- 输入信号 `rst` 为异步复位（高电平有效），复位后状态机回到初始状态，`unlock = `0'`。

端口定义：

- `clk`（输入）：时钟信号
- `rst`（输入）：异步复位，高电平有效
- `din`（输入）：1 位串行密码输入
- `unlock`（输出）：开锁信号，高电平有效

要求：

(1)（4 分）说明该密码锁控制器的状态定义。画出 **Moore 型** 状态机的状态转移图。标注：

- 每个状态的名称（建议 S0~S4）及含义（说明已正确匹配到密码的第几位）；
- 每个状态下的 `unlock` 输出值；
- 所有状态转移条件（即输入 `din` 的值）和转移方向。

(2) (4 分) 列出该 Moore 型状态机的状态转移表, 包含以下列: 当前状态、输入 `din`、次态、输出 `unlock`。

(3) (9 分) 编写完整的 VHDL 代码实现该密码锁控制器。要求:

- 使用用户自定义枚举类型定义所有状态(如 `TYPE lock_state IS (S0, S1, S2, S3, S4)`); (1 分)
- 采用双进程描述风格: 一个时序进程描述状态寄存器和异步复位, 一个组合进程描述次态逻辑和输出逻辑; (3 分)
- 异步复位 `rst` 为高电平时, 状态立即回到 `S0`, `unlock` 输出 '0'; (1 分)
- 代码中包含清晰的注释, 说明各状态含义和关键转换条件。(2 分)
- 代码规范: 缩进一致、信号命名合理。(2 分)

(4) (3 分) 回答以下问题:

- (a) 简述 Moore 型状态机与 Mealy 型状态机在该密码锁应用中的主要区别。如果改用 Mealy 型实现, `unlock` 信号的输出时序会有什么变化?
- (b) 若需要支持密码可修改(即密码不固定为“1101”, 而是由外部输入动态设定), 现有设计需要增加哪些输入端口? 状态机设计需要做哪些主要改动?(仅需文字说明, 不要求写代码)

提示:

- 密码“1101”的匹配过程: `S0` (初始) $\xrightarrow{\text{din}=1}$ `S1` (匹配“1”) $\xrightarrow{\text{din}=1}$ `S2` (匹配“11”) $\xrightarrow{\text{din}=0}$ `S3` (匹配“110”) $\xrightarrow{\text{din}=1}$ `S4` (匹配“1101”, 开锁)。任何不匹配的输入均使状态回到 `S0`。
- Moore 型状态机的 `unlock` 输出仅取决于当前状态: 仅 `S4` 状态下 `unlock` = '1', 其余状态均为 '0'。
- 非重叠检测示例: 输入“11011101”可检测出两次“1101”。

——试卷结束——

第十三章 第 13 套试卷

一、选择题（每题 3 分，共 18 分）

1. 基于 SRAM 工艺的 FPGA 芯片，其配置数据通常存储于（ ）。
 - A. FPGA 内部 EEPROM 中，断电后配置不丢失
 - B. 外部配置存储器（如 SPI Flash）中，上电后自动加载
 - C. FPGA 内部熔丝中，一次编程永久固化
 - D. 不需要配置数据，FPGA 功能由硬件连线固定
2. 在 VHDL 结构体中，以下哪条语句属于并发语句（Concurrent Statement）？
 - A. IF ... THEN ... END IF;（在 PROCESS 内部）
 - B. FOR i IN 0 TO 7 LOOP ... END LOOP;（在 PROCESS 内部）
 - C. y <= a AND b;（在 PROCESS 外部）
 - D. VARIABLE temp : STD_LOGIC;（在 PROCESS 内部）
3. Moore 型状态机与 Mealy 型状态机的主要区别是（ ）。
 - A. Moore 型状态机的输出仅取决于当前状态，Mealy 型状态机的输出与当前状态和输入均有关
 - B. Moore 型状态机的状态数一定比 Mealy 型少
 - C. Mealy 型状态机不能检测序列，Moore 型可以
 - D. Moore 型状态机必须使用异步复位，Mealy 型必须使用同步复位

4. 在 VHDL 表达式中, 运算符 NOT、AND、OR、XOR 的优先级从高到低排列正确的是 ()。
- A. NOT > AND > OR > XOR
- B. NOT > AND > XOR > OR
- C. NOT > XOR > AND > OR
- D. NOT > AND = OR = XOR (三者优先级相同)
5. 下列关于 CPLD 宏单元 (Macrocell) 的描述, 错误的是 ()。
- A. 宏单元通常包含一个乘积项阵列和一个可配置的触发器
- B. 宏单元中的触发器可以被旁路 (Bypass), 实现纯组合逻辑输出
- C. 宏单元的输出极性通常不可配置, 固定为高电平有效
- D. 多个宏单元的乘积项可以通过乘积项共享 (Product Term Sharing) 进行扩展
6. VHDL 中, 以下关于 CONSTANT、SIGNAL 和 VARIABLE 的声明范围的说法, 正确的是 ()。
- A. CONSTANT 可以在结构体和进程内部声明, VARIABLE 只能在结构体中声明
- B. SIGNAL 只能在结构体中声明, VARIABLE 只能在进程或子程序内部声明
- C. VARIABLE 可以在结构体中声明, 用于进程间通信
- D. CONSTANT 只能在实体中声明, 不能在结构体或进程内部声明

二、填空题 (每空 3 分, 共 12 分)

1. 在 VHDL 中, 使用 _____ 关键字定义常量; 使用 _____ 关键字定义变量; 使用 _____ 关键字定义信号。
2. 在 VHDL 的结构化描述中, 使用关键字 _____ 声明一个待实例化的底层模块, 使用关键字 _____ 将其例化并连接端口。

3. 在 VHDL 中,GENERIC 关键字的作用是 _____ ;
FOR ... GENERATE 语句属于 _____ 语句 (填“顺序”或“并发”)。
4. 在 VHDL 组合逻辑进程中,敏感信号表必须包含所有 _____
的信号,否则综合时可能产生 _____ (即意外的存储单元)。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码，找出其中的 5 处错误，并写出正确的修改方案。

Listing 13.1: 存在错误的 VHDL 代码（描述一个组合逻辑电路）

```
1 LIBRARY ieee;  
2 USE ieee.std_logic_1164.ALL;  
3  
4 ENTITY comb_err IS  
5     PORT(  
6         a, b, c : IN  STD_LOGIC;  
7         y       : OUT STD_LOGIC  
8     );  
9 END comb_err;  
10  
11 ARCHITECTURE behav OF comb_err IS  
12     SIGNAL tmp : STD_LOGIC;  
13 BEGIN  
14     PROCESS(a, b)  
15     BEGIN  
16         IF a = '1' THEN  
17             tmp <= b;  
18         ELSIF b = '1' THEN  
19             tmp := c;  
20         END IF;  
21     END PROCESS  
22     y = tmp;  
23 END behav;
```

答题要求：请按以下格式逐行列出错误位置、错误原因及正确写法（每处 1 分，共 5 分）。

错误 1（第 ____ 行）：_____ 改为 _____

错误 2（第 ____ 行）：_____ 改为 _____

错误 3（第 ____ 行）：_____ 改为 _____

错误 4（第 ____ 行）：_____ 改为 _____

错误 5（第 ____ 行）：_____ 改为 _____

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 13.2: 分析用 VHDL 代码

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY shift_circuit IS
5     PORT(
6         clk, rst : IN  STD_LOGIC;
7         q        : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
8     );
9 END shift_circuit;
10
11 ARCHITECTURE behav OF shift_circuit IS
12     SIGNAL sreg : STD_LOGIC_VECTOR(3 DOWNTO 0);
13 BEGIN
14     PROCESS(clk, rst)
15     BEGIN
16         IF rst = '1' THEN
17             sreg <= "1000";
18         ELSIF rising_edge(clk) THEN
19             sreg <= sreg(0) & sreg(3 DOWNTO 1);
20         END IF;
21     END PROCESS;
22     q <= sreg;
23 END behav;
```

问题：

- (1) 该 VHDL 代码描述的是什么电路？请写出其名称。（1 分）
- (2) 该电路在复位后，从第一个有效时钟边沿开始，输出 q 的值依次是什么？请列出完整的一个循环序列（以二进制形式给出）。（2 分）
- (3) 若将第 19 行改为 `sreg <= sreg(2 DOWNTO 0) & NOT sreg(3);`，电路功能变成什么？请写出修改后电路的名称和输出循环序列。（2 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——BCD 码至 7 段数码管译码器（20 分）

设计一个 BCD 码至 7 段共阳极数码管译码器 (BCD-to-7-Segment Decoder)。输入为 4 位 BCD 码 `bcd(3 downto 0)` (范围 0000~1001, 即十进制 0~9), 输出为 7 段数码管段选信号 `seg(6 downto 0)`, 低电平点亮 (`seg(6)` 对应 a 段, `seg(5)` 对应 b 段, …… , `seg(0)` 对应 g 段)。当输入为非法值 (1010~1111) 时, 数码管全灭 (所有段输出高电平)。

- (1) (8 分) 列出 BCD 码至 7 段码译码器的真值表。要求: 列出所有有效输入 (0~9) 及非法输入 (10~15 的二进制) 对应的 `seg` 输出值。段排列顺序为 {a, b, c, d, e, f, g}。
- (2) (10 分) 写出完整的 VHDL 代码 (包含库声明、实体定义和结构体), 要求使用数据流描述方式——选择信号赋值语句 (WITH-SELECT-WHEN) 实现译码功能。实体名称为 `bcd2seg7`。
- (3) (2 分) 代码注释中说明: 为何选用 WITH-SELECT 而非 IF-ELSE 实现此功能? 两者在使用场景上的主要区别是什么?

提示: 共阳极数码管: 段选为 '0' 时该段点亮, '1' 时熄灭。例如显示数字 “0” 时, 除 g 段灭 ('1') 外其余 6 段均亮 ('0'), 即 `seg = “1000000”`。

设计题 2：时序逻辑设计——N 位双向移位寄存器（20 分）

设计一个参数化的 N 位双向移位寄存器 (默认 N=8), 要求使用结构化 VHDL 描述方式 (Component + Port Map + Generate 语句)。设计要求:

- 输入端口: `clk` (时钟)、`rst` (同步复位, 高电平有效)、`dir` (方向控制: '0' 左移、'1' 右移)、`din` (串行输入, 1 位)、`load` (并行加载使能, 高电平有效)、`pin(7 downto 0)` (并行加载数据)。
- 输出端口: `q(7 downto 0)` (当前寄存器值)、`dout` (串行输出, 1 位, 左移时取自最高位, 右移时取自最低位)。

- (1) (4 分) 画出顶层模块的端口框图, 标注所有端口名称、方向和位宽。
- (2) (2 分) 设计单个 D 触发器的底层元件 `dff_ce` (带时钟使能的 D 触发器), 写出其实体定义 (仅需实体, 无需结构体完整代码)。

(3) (10 分) 写出完整的顶层 VHDL 代码。要求:

- 使用 COMPONENT 声明 dff_ce 元件;
- 使用 FOR ... GENERATE 语句生成 8 个 D 触发器实例;
- 使用 PORT MAP 进行元件例化, 通过 2 选 1 多路选择逻辑 (可用条件信号赋值语句) 实现移位方向和并行加载的控制;
- 第 0 位和第 7 位需要特殊处理移位输入/输出连接。

(4) (4 分) 若需将位宽改为 16 位, 代码中需要修改哪些地方? 请在代码注释中标注参数化的关键位置, 并说明如何利用 GENERIC 语句实现位宽可配置。

提示: 结构体可声明内部信号数组 (如 TYPE wire_array IS ARRAY(0 TO 7) OF STD_LOGIC) 用于级联互连。每个 D 触发器实例的输入由移位方向、加载信号共同决定。可参考以下结构: q_int(i) <= pin(i) WHEN load = '1' ELSE ... (条件赋值决定每位 D 触发器的数据输入)。

设计题 3: 状态机设计——“1011”序列检测器 (20 分)

设计一个“1011”序列检测器。输入信号为 din (1 位串行输入), 输出信号为 found (1 位)。当连续检测到“1011”序列时 found 输出'1' (高电平有效), 其余情况输出'0'。允许序列重叠检测 (即“101011”中可检测出两次“1011”)。

(1) Moore 型状态机 (8 分)

- 画出 Moore 型状态机的状态转移图, 标注每个状态的名称、含义及该状态下 found 的输出值。需涵盖 5 个状态 (S0~S4)。
- 列出 Moore 型状态机的状态转移表 (当前状态、输入 din、次态、输出 found)。

(2) Mealy 型状态机 (6 分)

- 画出 Mealy 型状态机的状态转移图, 标注每个状态的名称、含义、转移条件及转移时的输出值。需涵盖 4 个状态 (S0~S3)。
- 列出 Mealy 型状态机的状态转移表 (当前状态、输入 din、次态、输出 found)。

(3) **VHDL 代码实现（6 分）** 选择 Moore 型或 Mealy 型状态机，编写完整的 VHDL 代码。要求：

- 使用用户自定义枚举类型定义状态；
- 采用双进程（或三进程）描述风格；
- 时钟 `clk` 上升沿触发，复位 `rst` 为异步复位（低电平有效）。

提示：“1011”序列需 5 个状态（Moore）或 4 个状态（Mealy）。Moore 型：S0（初始）、S1（已匹配“1”）、S2（已匹配“10”）、S3（已匹配“101”）、S4（已匹配“1011”，输出为 1）。Mealy 型：S0（初始）、S1（已匹配“1”）、S2（已匹配“10”）、S3（已匹配“101”），在 S3 状态下若输入为“1”则输出为 1 并转移到 S1（重叠）。注意序列重叠情况——例如“101011”中：第 1 次“1011”（位 0-3），第 2 次“1011”（位 2-5）。

第十四章 第 14 套试卷

一、选择题（每题 3 分，共 18 分）

1. 基于 SRAM 工艺的 FPGA 芯片，上电时配置数据通常从何处加载？
 - A. 片内掩膜 ROM
 - B. 片内 EEPROM
 - C. 外部非易失性存储器（PROM/Flash）
 - D. 无需加载，配置数据掉电不丢失
2. CPLD 的基本逻辑单元——宏单元（Macrocell）通常不包含以下哪一部分？
 - A. 乘积项阵列（Product Term Array）
 - B. 查找表（LUT）
 - C. 可配置触发器（Flip-Flop）
 - D. 输出极性选择与反馈通路
3. 下列 VHDL 语句中，只能在结构体的并发区域（PROCESS 外部）使用的是：
 - A. IF-ELSIF-ELSE
 - B. CASE-WHEN
 - C. 变量赋值（:=）
 - D. 选择信号赋值（WITH-SELECT-WHEN）
4. 关于 Moore 型与 Mealy 型有限状态机的描述，正确的是：
 - A. Moore 型输出仅与当前状态有关，Mealy 型输出与当前状态和输入均有关

- B. Moore 型比 Mealy 型所需状态数一定更少
 - C. Mealy 型输出仅由输入决定，与状态无关
 - D. Moore 型不能用于序列检测，Mealy 型可以
5. 在 VHDL 中，以下运算符优先级最高的是：
- A. AND
 - B. =（等于）
 - C. NOT
 - D. &（拼接）
6. 在 VHDL 中，CONSTANT（常量）可以在以下哪个位置声明？
- A. ENTITY 的 PORT 中
 - B. ARCHITECTURE 的声明区（BEGIN 之前）
 - C. 仅能在 PACKAGE（包）中
 - D. 仅能在 PROCESS 内部

二、填空题（每题 3 分，共 12 分）

1. VHDL 中，SIGNAL 通常在_____（ARCHITECTURE 声明区）中定义，用于进程间通信；VARIABLE 只能在_____（PROCESS 或子程序）内部声明，赋值后立即生效。
2. 关键字 _____ 用于向实体传递位宽等静态参数，实例化时通过 GENERIC MAP 赋值；关键字 _____ 用于声明可复用的子电路模板，实例化时使用 PORT MAP 连接端口。
3. 在组合逻辑进程的敏感信号表中，应列出所有_____的信号（填“被读取”或“被赋值”），否则综合工具可能推断出_____（填写非期望的存储单元类型）。
4. 生成语句中，_____ -GENERATE 按迭代次数重复生成硬件电路；_____ -GENERATE 按布尔条件选择性生成硬件电路。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码，找出其中的 5 处错误，并写出正确的修改方案。代码意图是设计一个带异步复位和使能端的 4 位加法计数器。

Listing 14.1: 存在错误的 VHDL 计数器代码

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY counter_bad IS
5      PORT(
6          clk : IN  STD_LOGIC;
7          rst : IN  STD_LOGIC;
8          en  : IN  STD_LOGIC;
9          q   : OUT STD_LOGIC_VECTOR(3 DOWNT0 0)
10     )
11 END ENTITY counter_bad;
12
13 ARCHITECTURE behav OF counter_bad IS
14     SIGNAL cnt : STD_LOGIC_VECTOR(3 DOWNT0 0);
15 BEGIN
16     PROCESS(clk)
17     BEGIN
18         IF rst = '1' THEN
19             cnt := "0000";
20         ELSIF rising_edge(clk) THEN
21             IF en = '1' THEN
22                 cnt <= cnt + 1;
23             END IF
24         END IF;
25     END PROCESS;
26
27     q <= cnt;
28 END behav;
```

答题要求：逐行指出错误位置、错误原因及正确写法（每处 1 分，找出 5 处即得满分 5 分）。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 14.2: 程序阅读分析

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY freq_div IS
6      PORT(
7          clk_in  : IN  STD_LOGIC;
8          rst     : IN  STD_LOGIC;
9          clk_out : OUT STD_LOGIC
10     );
11 END freq_div;
12
13 ARCHITECTURE behav OF freq_div IS
14     SIGNAL count : INTEGER RANGE 0 TO 7 := 0;
15     SIGNAL temp  : STD_LOGIC := '0';
16 BEGIN
17     PROCESS(clk_in, rst)
18     BEGIN
19         IF rst = '1' THEN
20             count <= 0;
21             temp  <= '0';
22         ELSIF rising_edge(clk_in) THEN
23             IF count = 3 THEN
24                 count <= 0;
25                 temp  <= NOT temp;
26             ELSE
27                 count <= count + 1;
28             END IF;
29         END IF;
30     END PROCESS;
31     clk_out <= temp;
32 END behav;
```

问题：

- (1) 该电路实现的是什么功能？分频比是多少？（2 分）
- (2) 信号 `count` 的作用是什么？它一共有几种取值？（1 分）
- (3) 若输入时钟 `clk_in` 的频率为 80 MHz，输出 `clk_out` 的频率是多少？占空比是多少？（2 分）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——BCD 码转 7 段数码管译码器（20 分）

设计一个 BCD 码转 7 段数码管译码器，将 4 位 BCD 码输入（0~9）转换为共阳极 7 段数码管的段选信号输出。输入为 `bcd(3 DOWNT0 0)`，输出为 `seg(6 DOWNT0 0)`（对应段 a、b、c、d、e、f、g，低电平点亮）。

- (1) 列出 BCD 码 0~9 对应的 7 段码真值表，包含输入 `bcd` 和输出 `seg` 的所有位（6 分）
- (2) 画出顶层模块的端口框图，标注所有端口名称、方向和位宽（4 分）
- (3) 编写完整的 VHDL 代码，使用 `WITH-SELECT` 或 `CASE` 语句实现译码逻辑。对于无效输入（10~15），输出全熄灭（全 '1'）。（10 分）

参考：共阳极数码管，段码为低电平时对应段点亮。各段位置约定：`seg(6)=a`，`seg(5)=b`，`seg(4)=c`，`seg(3)=d`，`seg(2)=e`，`seg(1)=f`，`seg(0)=g`。

设计题 2：时序逻辑设计——4 位双向移位寄存器（20 分）

设计一个 4 位双向移位寄存器，具备左移、右移、并行加载和保持功能。要求：

- (1) **端口定义（4 分）**：输入端口包括——`clk`（时钟）、`rst`（异步复位，高电平有效）、`data_in(3 DOWNTO 0)`（并行加载数据）、`load`（加载使能，高电平有效）、`dir`（移位方向，0= 右移，1= 左移）、`sin`（串行输入位）。输出端口——`q(3 DOWNTO 0)`（当前寄存器值）。
- (2) **功能描述（4 分）**：列出 4 种工作模式（复位、加载、右移、左移）及其对应的控制信号和操作。
- (3) **VHDL 代码（12 分）**：编写完整的 VHDL 实体和结构体。使用单个 PROCESS，在时钟上升沿处理。优先级：`rst > load > 移位操作`。右移时数据从高位向低位移动（`q(2:0) → q(3:1)`，`sin → q(0)`）；左移时数据从低位向高位移动（`q(3:1) → q(2:0)`，`sin → q(3)`）。

设计题 3：状态机设计——自动售货机控制器（20 分）

设计一个简单自动售货机的控制器。售货机只接受 1 元和 2 元硬币（信号 `coin1` 和 `coin2`，两者不会同时为‘1’），饮料售价为 3 元。控制器根据累计投币金额决定是否出货和找零。

端口信号：`clk`（时钟）、`rst`（异步复位，高有效，回初始状态）、`coin1`（投入 1 元）、`coin2`（投入 2 元）、`give`（出货，1 个时钟周期高电平）、`change`（找零 1 元，1 个时钟周期高电平）。

投币规则：每时钟周期检测一次投币信号（`coin1` 或 `coin2` 为‘1’时表示该周期投入对应面额硬币）；当累计金额 ≥ 3 元时，下一状态回到初始状态，同时输出 `give=1`；若累计金额 > 3 元（即投入 4 元），除 `give=1` 外还需输出 `change=1`（找零 1 元）。

状态定义建议：`S0`（累计 0 元）、`S1`（累计 1 元）、`S2`（累计 2 元）。

采用 **Mealy 型** 状态机设计（输出与当前状态和输入均有关）。

- (1) 画出状态转移图，标注所有状态名称、转移条件（含输入条件）和输出值（`give/change`）。（8 分）
- (2) 列出状态转移表，包含：当前状态、输入（`coin1 coin2`）、次态、输出（`give change`）。（4 分）
- (3) 编写完整的 VHDL 代码（实体 + 结构体），采用双进程描述风格：一个时序进程（状态寄存器 + 异步复位），一个组合进程（次态逻辑 + 输出逻辑）。状态使用用户自定义枚举类型。（8 分）

提示：Mealy 型状态机的输出在状态转移弧上标注，格式为“输入/输出”。当状态转移不满足出货条件时，`give` 和 `change` 均为‘0’。

第十五章 第 15 套试卷

一、选择题（每题 3 分，共 18 分）

1. 下列关于 CPLD 与 FPGA 内部基本逻辑单元实现方式的说法，正确的是？
 - A. CPLD 基于查找表（LUT）结构，FPGA 基于乘积项（Product Term）结构
 - B. CPLD 基于乘积项结构，FPGA 基于查找表结构
 - C. CPLD 和 FPGA 均基于查找表结构
 - D. CPLD 和 FPGA 均基于乘积项结构
2. 以下哪种**不是**常见的 FPGA 配置（编程）模式？
 - A. Master Serial（主串模式）
 - B. JTAG 模式
 - C. DMA 模式
 - D. SPI 模式
3. 在 VHDL 中，以下语句中属于**顺序语句**（只能出现在 PROCESS 或子程序内部）的是？
 - A. 简单信号赋值语句（如 $y \leq a$ ；位于结构体中）
 - B. PROCESS 语句本身
 - C. IF 语句
 - D. 元件例化语句（PORT MAP）
4. Moore 型与 Mealy 型有限状态机（FSM）的主要区别是？

- A. Moore 型的输出仅取决于当前状态；Mealy 型的输出取决于当前状态和输入
 - B. Moore 型的输出取决于当前状态和输入；Mealy 型的输出仅取决于当前状态
 - C. Moore 型只能用于序列检测，Mealy 型只能用于计数控制
 - D. Moore 型必须用双进程描述，Mealy 型必须用三进程描述
5. 在 VHDL 中，以下逻辑运算符按优先级从高到低排列，正确的是？
- A. AND > OR > NOT > XOR
 - B. NOT > AND > OR > XOR
 - C. NOT > XOR > AND > OR
 - D. 所有逻辑运算符优先级相同，按从左到右计算
6. 下列关于 PROCESS 敏感信号列表（Sensitivity List）的描述，错误的是？
- A. 组合逻辑进程的敏感信号列表应包含所有被读取的输入信号
 - B. 时序逻辑进程的敏感信号列表通常只包含时钟和异步复位/置位信号
 - C. 敏感信号列表不完整可能导致综合前仿真与综合后仿真结果不一致
 - D. 组合逻辑进程的敏感信号列表可以省略，综合工具会自动补全

二、填空题（每题 3 分，共 12 分）

1. 在 VHDL 中，信号（SIGNAL）使用_____（填符号）赋值，变量（VARIABLE）使用_____（填符号）赋值。常量的声明关键字是_____。
2. 在实体（ENTITY）声明中，关键字_____用于定义类属参数（如位宽、计数模值等），它必须声明在关键字_____之前。
3. 使用结构化描述风格时，需在结构体中用关键字_____声明待例化的子模块接口，再用关键字_____将其端口与顶层信号连接。
4. VHDL 中的生成语句有两种形式：_____ GENERATE 用于重复生成多个相同结构；_____ GENERATE 用于根据条件选择性地生成硬件结构。

三、程序改错题（共 5 分）

以下 VHDL 代码意图描述一个带异步复位的 4 位寄存器（含使能端），但存在 6 处错误。请找出任意 5 处，指出错误位置、原因及正确写法。

Listing 15.1: 含错误的 VHDL 代码（6 处错误，找出 5 处即满分）

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY reg4_err IS
5      PORT(
6          clk : IN  STD_LOGIC;
7          rst : IN  STD_LOGIC;
8          en  : IN  STD_LOGIC;
9          d   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
10         q    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
11     );
12 END reg4_err;
13
14 ARCHITECTURE behavioral OF reg4_err IS
15     SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNTO 0);
16 BEGIN
17     PROCESS(clk)
18         VARIABLE tmp : STD_LOGIC_VECTOR(3 DOWNTO 0);
19     BEGIN
20         IF rst = '1' THEN
21             q_int := (OTHERS => '0');
22             tmp <= (OTHERS => '0');
23         ELSIF rising_edge(clk) THEN
24             IF en = '1' THEN
25                 q_int <= d;
26                 tmp := d;
27             END IF
28         END IF
29     END PROCESS
30     q <= q_int;
31 END behavioral;
```

答题格式要求：逐行指出错误位置、错误原因及正确写法。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 15.2: 分析用 VHDL 代码

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY freq_divider IS
6      GENERIC(N : INTEGER := 8);
7      PORT(
8          clk      : IN  STD_LOGIC;
9          rst      : IN  STD_LOGIC;
10         clk_out   : OUT STD_LOGIC
11     );
12 END freq_divider;
13
14 ARCHITECTURE behav OF freq_divider IS
15     SIGNAL count : INTEGER RANGE 0 TO N-1 := 0;
16     SIGNAL temp  : STD_LOGIC := '0';
17 BEGIN
18     PROCESS(clk, rst)
19     BEGIN
20         IF rst = '1' THEN
21             count <= 0;
22             temp  <= '0';
23         ELSIF rising_edge(clk) THEN
24             IF count = N/2 - 1 THEN
25                 temp  <= NOT temp;
26                 count <= 0;
27             ELSE
28                 count <= count + 1;
29             END IF;
30         END IF;
31     END PROCESS;
32     clk_out <= temp;
33 END behav;
```

问题：

- (1) 该 VHDL 代码描述的是什么电路？其基本功能是什么？（1 分）
- (2) GENERIC 参数 N 的作用是什么？如果 clk 频率为 100MHz，N=8 时，输出信号 clk_out 的频率是多少？（2 分）
- (3) 信号 rst 的复位方式是同步复位还是异步复位？请结合代码说明判断依据。（1 分）
- (4) 若将条件 IF count = N/2 - 1 THEN 改为 IF count = N/2 THEN（即去掉 -1），对输出频率和占空比各有何影响？（1 分）

五、综合设计题（共 60 分）

设计题 1：BCD 码—7 段数码管译码器（20 分）

设计一个 BCD 码到 7 段共阳极数码管的译码电路。输入为 4 位 BCD 码 `bcd[3:0]`（表示十进制数 0~9），输出为 7 位段选信号 `seg[6:0]`，分别对应数码管的 a~g 段（`seg(6)` 对应 a 段，`seg(0)` 对应 g 段）。共阳极数码管：段选信号为 '0' 时对应段点亮，为 '1' 时熄灭。当输入为非法 BCD 码（10~15）时，所有段熄灭。要求：

- (1) 列出该译码器的真值表，至少包含输入 0~9 及非法输入 10~15 对应的输出段码。（5 分）
- (2) 分别写出 a 段、b 段和 g 段的最小化逻辑表达式（可用卡诺图化简）。（3 分）
- (3) 使用数据流描述方式（选择信号赋值语句 `WITH-SELECT-WHEN`）编写完整的 VHDL 代码，包含库声明、实体定义和结构体。（10 分）
- (4) 说明数据流描述与行为描述的差异，并指出本题选择数据流方式的理由。（2 分）

提示：共阳极数码管段码：数字 0 为"1000000"，数字 1 为"1111001"，其余自行推导。非法输入时输出"1111111"。

设计题 2：使用 GENERATE 语句设计 N 位双向移位寄存器（20 分）

设计一个参数化 N 位双向移位寄存器。要求使用 GENERATE 语句和结构化描述方式。寄存器功能如下：

- 输入端口：`clk`（时钟）、`rst`（同步复位，高电平有效）、`din`（串行输入，1 位）、`dir`（移位方向：'0' 右移，'1' 左移）、`load`（并行置数使能）、`pdata[N-1:0]`（并行置数数据）。
- 输出端口：`q[N-1:0]`（当前寄存器值）、`sout`（串行输出）。
- 工作模式（优先级从高到低）：复位 → 并行置数 → 移位（右移或左移）→ 保持。
- 右移时：最高位移入 `din`，最低位移出至 `sout`；左移时：最低位移入 `din`，最高位移出至 `sout`。

- (1) 画出单个位片（BitSlice）的内部逻辑框图，标注所有输入输出信号。（4 分）
- (2) 写出位片的 ENTITY 声明，将其定义为 COMPONENT。（3 分）

- (3) 使用 GENERIC (定义位宽) 和 FOR...GENERATE 语句, 编写顶层 N 位寄存器的完整 VHDL 代码 (实体 + 结构体)。要求例化 N 个位片并正确连接级联信号。(11 分)
- (4) 说明 FOR...GENERATE 与普通 FOR-LOOP 语句的本质区别。(2 分)

提示: 位片内部包含一个 D 触发器和选择逻辑 (复位/置数/移位/保持); 相邻位片之间需正确连接移位数据线。顶层可使用中间信号数组实现位间级联。

设计题 3: “1011” 序列检测器——Moore 型与 Mealy 型对比 (20 分)

设计一个有限状态机, 用于检测串行输入序列 “1011”。输入信号为 `din` (1 位, 每个时钟周期输入 1 位), 当检测到完整的 “1011” 序列时, 输出信号 `dout` 输出一个时钟周期的高电平。允许序列重叠检测 (例如序列 “1011011” 中, 前 4 位 “1011” 和后 4 位 “1011” 重叠, 应检测到两次)。

- (1) 画出 **Moore 型** 状态机的状态转换图, 标注每个状态的名称/含义、转换条件以及各状态的输出值。Moore 型输出仅取决于当前状态。(6 分)
- (2) 画出 **Mealy 型** 状态机的状态转换图, 标注每个状态的名称/含义、转换条件以及各转移边上的输出值。Mealy 型输出取决于当前状态和输入。(4 分)
- (3) 选择 **Moore 型**, 编写完整的 VHDL 代码 (实体 + 结构体)。要求: 使用用户自定义枚举类型定义状态, 采用三进程描述风格 (状态寄存器进程 + 次态逻辑进程 + 输出逻辑进程), 时钟上升沿触发, 复位 `rst` 为异步复位 (高电平有效)。(10 分)

提示: Moore 型需要 5 个状态: S0 (初始/未匹配)、S1 (已匹配 “1”)、S2 (已匹配 “10”)、S3 (已匹配 “101”)、S4 (已匹配 “1011”——输出 1)。注意重叠检测: S4 状态下若 `din=0` 则转到 S2 (因为尾部的 “10” 可作为下一个序列的前缀)。Mealy 型仅需 4 个状态。

第十六章 第 16 套试卷

FPGA 与 VHDL 基础试卷（十六）

（满分：100 分 考试时间：120 分钟 难度：中等）

一、选择题（每题 3 分，共 18 分）

1. 一个典型的 FPGA 可配置逻辑块（CLB/Slice）通常包含以下哪组基本资源？
 - A. 仅由查找表（LUT）构成，不含任何触发器
 - B. 查找表（LUT）、D 触发器、多路选择器和进位链
 - C. 完整的 CPU 处理器核和存储器
 - D. 模拟锁相环（PLL）和时钟管理模块
2. 在 VHDL 中，关于信号（SIGNAL）与变量（VARIABLE）的赋值和更新时机，以下描述正确的是？
 - A. 信号的赋值（<=）在语句执行时立即生效，变量的赋值（:=）在进程挂起时才更新
 - B. 变量的赋值（:=）在语句执行时立即生效，信号的赋值（<=）在进程挂起（或经过 Δ 延时）后才更新
 - C. 信号和变量的赋值均在语句执行时立即生效，没有区别
 - D. 信号和变量的赋值均在进程挂起时才生效，没有区别
3. 在 VHDL 的组合逻辑 PROCESS 中，如果敏感信号列表遗漏了某些被读取的输入信号，综合工具通常会产生什么后果？
 - A. 综合失败，无法生成任何电路
 - B. 综合工具自动补全敏感信号列表，综合前后仿真结果完全一致
 - C. 综合工具可能推断出锁存器（Latch），导致综合前仿真与综合后仿真结果不一致
 - D. 不会产生任何影响，因为敏感信号列表仅用于仿真

4. 对于**同一功能**（如相同的序列检测任务），Moore 型与 Mealy 型状态机相比，以下说法**正确**的是？
- A. Moore 型通常需要更多的状态，但其输出仅取决于当前状态，抗毛刺能力更强
 - B. Mealy 型通常需要更多的状态，但其输出响应更慢
 - C. 两种类型所需状态数完全相同，仅在编码方式上有差异
 - D. Moore 型输出与输入直接相关，Mealy 型输出仅取决于当前状态
5. 在 VHDL 中，IF 语句和 CASE 语句属于哪类语句？它们可以出现在什么位置？
- A. 并发语句（Concurrent Statement），可出现在结构体中的任何位置
 - B. 顺序语句（Sequential Statement），只能出现在 PROCESS 或子程序（FUNCTION/PROCEDURE）内部
 - C. 既可作为并发语句也可作为顺序语句，取决于上下文
 - D. 并发语句，但只能出现在结构体的声明区（BEGIN 之前）
6. 在 VHDL 中，关于逻辑运算符的优先级，以下描述**正确**的是？
- A. AND 优先级最高，OR 次之，NOT 最低
 - B. NOT 优先级最高，AND、OR、NAND、NOR、XOR、XNOR 等二元逻辑运算符的优先级相同且低于 NOT
 - C. 所有逻辑运算符（包括 NOT、AND、OR、XOR 等）优先级完全相同，按从左到右顺序计算
 - D. XOR 优先级最高，其次是 AND，最后是 OR 和 NOT

二、填空题（每空 3 分，共 12 分）

1. VHDL 中, IEEE 库的两个最常用的标准程序包是_____（定义了 STD_LOGIC 和 STD_LOGIC_VECTOR 类型）和_____（定义了 UNSIGNED 和 SIGNED 类型及相关的算术运算函数）。
2. 在 VHDL 中, 信号(SIGNAL)使用_____（填符号）进行赋值, 其值在进程_____（填“执行时”或“挂起时”）更新; 变量(VARIABLE)使用_____（填符号）进行赋值, 其值在语句_____（填“执行时”或“挂起时”）立即更新。
3. 在 VHDL 的实体(ENTITY)声明中, _____关键字用于定义可配置的静态参数（如数据位宽、模值等）, 它必须声明在_____关键字之前。在顶层例化子模块时, 通过_____语句将实际参数值传递给子模块。
4. 在 VHDL 的结构化描述中, 需先在结构体的声明区使用_____语句声明待例化子模块的接口, 再使用_____语句将该子模块的端口与顶层信号实际连接。

三、程序改错题（共 5 分）

以下 VHDL 代码意图设计一个带使能的 3-8 译码器。当使能信号 `en = '1'` 时，根据 3 位输入 `din` 将对应输出位设为 '1'（独热码），其余位为 '0'；当 `en = '0'` 时，所有输出为 '0'。但代码中存在至少 6 处错误。请找出任意 5 处（每处 1 分，共 5 分），指出错误位置（行号）、错误原因及正确写法。

Listing 16.1: 含错误的 3-8 译码器 VHDL 代码（6 处错误，找出 5 处即满分）

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY decoder38_err IS
5      PORT(
6          en    : IN  STD_LOGIC;
7          din   : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
8          dout  : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
9      )
10
11
12  END decoder38_err;
13
14  ARCHITECTURE behav OF decoder38_err IS
15  BEGIN
16      PROCESS(din)
17          -- 敏感信号列表不完整，缺少 en
18      BEGIN
19          IF en = '1' THEN
20              CASE din IS
21                  WHEN "000" => dout <= "00000001"
22                  WHEN "001" => dout <= "00000010"
23                  WHEN "010" => dout <= "00000100";
24                  WHEN "011" => dout <= "00001000";
25                  WHEN "100" => dout <= "00010000";
26                  WHEN "101" => dout <= "00100000";
27                  WHEN "110" => dout <= "01000000";
28                  WHEN "111" => dout <= "10000000";
29                  -- CASE语句缺少 OTHERS分支（不完整覆盖）
30              END CASE;
31              -- 缺少 en='0'时的 ELSE分支，将推断出锁存器（Latch）
32          END IF;
33      END PROCESS
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

-- 缺少分号

错 误 5 (第 行) : 原 因:

提示：错误类型包括但不限于——(1) 实体声明语法错误（缺少分号）；(2) 敏感信号列表不完整（组合逻辑进程遗漏输入信号）；(3) CASE 语句中分支语句缺少分号；(4) CASE 语句未覆盖所有可能取值（缺少 OTHERS 分支）；(5) 组合逻辑进程中 IF 语句缺少 ELSE 分支，推断出锁存器；(6) PROCESS 或 ARCHITECTURE 末尾缺少分号。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 16.2: 分析用 VHDL 代码——6 分频器

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY div6 IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;
8          clk_out  : OUT STD_LOGIC
9      );
10 END div6;
11
12 ARCHITECTURE behav OF div6 IS
13     SIGNAL count : INTEGER RANGE 0 TO 2 := 0;
14     SIGNAL temp  : STD_LOGIC := '0';
15 BEGIN
16     PROCESS(clk, rst)
17     BEGIN
18         IF rst = '1' THEN
19             count <= 0;
20             temp  <= '0';
21         ELSIF rising_edge(clk) THEN
22             IF count = 2 THEN
23                 count <= 0;
24                 temp  <= NOT temp;
25             ELSE
26                 count <= count + 1;
27             END IF;
28         END IF;
29     END PROCESS;
30     clk_out <= temp;
31 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？其基本功能是什么？
- (2) (2 分) 若输入时钟 `clk` 的频率为 12MHz，输出 `clk_out` 的频率是多少？请写出详细的推导计算过程。
- (3) (1 分) 输出信号 `clk_out` 的占空比是多少？请说明理由。
- (4) (1 分) 若要将分频比改为 10（即实现 10 分频），代码中的信号 `count` 的取值范围和比较条件需要如何修改？（仅写出修改方案，不要求完整代码）

五、综合设计题（共 60 分）

设计题 1：组合逻辑设计——带使能的 3-8 译码器（20 分）

设计一个带使能端的 3 线-8 线译码器（3-to-8 Decoder with Enable）。输入为 3 位二进制地址 `din(2 downto 0)` 和使能信号 `en`；输出为 8 位独热码 `dout(7 downto 0)`。

功能定义：

- 当 `en = '1'` 时，根据 `din` 的值将 `dout` 的对应位置为 '1'，其余位置为 '0'。例如：`din = "011"`（即十进制 3）时，`dout = "00001000"`（bit3 为 1）。
- 当 `en = '0'` 时，无论 `din` 为何值，`dout` 的所有位均输出 '0'。

要求：

- (1)（5 分）列出该译码器的完整真值表（共 8 行有效输入 +1 行使能无效的情况）。真值表应包含 `en`、`din(2:0)` 和 `dout(7:0)`。
 - (2)（4 分）画出顶层模块的端口框图，标注所有端口名称、方向和位宽。
 - (3)（9 分）使用行为描述风格（PROCESS + CASE 语句）编写完整的 VHDL 代码。
- 要求：

- 正确声明库和实体；
 - 在 PROCESS 中使用 IF 语句判断 `en`，在 CASE 语句中根据 `din` 为 `dout` 赋值；
 - CASE 语句必须覆盖 `din` 的所有可能取值（使用 WHEN OTHERS 分支）；
 - 当 `en = '0'` 时，所有输出复位为全 '0'（避免锁存器推断）。
- (4)（2 分）在代码中以注释方式说明：为什么组合逻辑的 PROCESS 敏感信号列表必须包含 `en`？如果遗漏了 `en`，综合结果会有什么问题？

提示：3-8 译码器的输出是“独热码”（One-Hot Code），即 8 位输出中仅有一位为 '1'，其余为 '0'。`din = "000"` 对应 `dout(0) = '1'`，`din = "001"` 对应 `dout(1) = '1'`，以此类推。

设计题 2：时序逻辑设计——模 6 计数器 (Mod-6 Counter) (20 分)

设计一个模 6 同步计数器 (计数范围: 0 5, 即 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 0 \rightarrow \dots$)。该计数器的模值 $N = 6$ 不是 2 的幂次, 需要使用复位逻辑或条件判断实现正确的循环计数。

端口定义:

- **clk** (输入): 时钟信号, 上升沿触发。
- **rst** (输入): 同步复位, 高电平有效。复位时计数值归零, **tc** 输出 '0'。
- **en** (输入): 计数使能信号, 高电平有效。**en** = '1' 时每个时钟上升沿计数值加 1; **en** = '0' 时保持当前计数值不变。
- **q(2 downto 0)** (输出): 3 位计数值输出 (因为计数值范围为 0 5, 需 3 位二进制表示)。
- **tc** (输出): 终端计数 (Terminal Count) 输出。当计数值为 5 且 **en** = '1' 时, **tc** 输出 '1' (持续一个时钟周期); 其余时间输出 '0'。

要求:

- (1) (4 分) 画出该模 6 计数器的状态转移图。标注 6 个状态 (0 5) 的名称 (以计数值表示)、各状态下的 **tc** 输出值, 以及所有状态转移条件 (需标注 **en** 的取值)。
- (2) (3 分) 画出顶层模块的端口框图, 标注所有端口名称、方向和位宽。
- (3) (10 分) 编写完整的 VHDL 代码 (实体 + 结构体)。要求:

- 使用行为描述风格 (PROCESS 实现);
- 内部使用 INTEGER RANGE 0 TO 5 类型的信号存储计数值;
- 在 PROCESS 中实现同步复位、使能控制和模 6 循环计数逻辑;
- 优先级: 同步复位 > 使能控制 > 计数;
- 输出 **tc** 和 **q** 需从内部计数值正确生成。

- (4) (3 分) 画出该计数器的简化测试平台 (Testbench) 草图, 包括:

- 时钟信号的生成方式 (周期和占空比自定, 标注参数);
- 复位信号的时序;

- 至少覆盖一个完整计数周期（0 5）的使能信号波形；
- 用波形示意图的形式画出关键信号的时序关系（clk、rst、en、q、tc）。

提示：

- 模 6 计数器的关键：当计数值达到 5 时，下一个有效计数脉冲使其回到 0（而非继续增加到 6）。
- 由于模值为 6（ $2^2 = 4 < 6 < 2^3 = 8$ ），输出端口 q 至少需要 3 位位宽。
- tc 信号在计数值 =5 且 en='1' 时为'1'（不必等到下一时钟沿），也可选择在 PROCESS 内部同步生成。

设计题 3：状态机设计——自动售货机控制器（20 分）

设计一个自动售货机控制器（Vending Machine Controller），使用 Moore 型有限状态机实现。售货机中某种商品的售价为 5 元，机器只接受 1 元和 2 元硬币（不接受纸币或其他面额）。用户逐次投入硬币，当累计投入金额恰好达到或超过 5 元时，售货机输出出货信号并自动找零（若超过 5 元则退还超出部分对应的硬币数）。为简化设计，本题目中暂不考虑找零——即要求投入金额恰好等于 5 元时出货，若某次投币会使金额超过 5 元，则忽略该次投币（金额保持不变）。

端口定义：

- `clk`（输入）：时钟信号，上升沿触发。
- `rst`（输入）：同步复位，高电平有效。复位后状态机回到初始状态，累计金额清零。
- `coin(1 downto 0)`（输入）：投币信号。编码定义如下：
 - "00"：本次无硬币投入。
 - "01"：投入 1 元硬币。
 - "10"：投入 2 元硬币。
 - "11"：无效编码，等同于无硬币投入。
- `dispense`（输出）：出货信号，高电平有效。当累计金额达到 5 元时，持续输出一个时钟周期的 '1'，随后自动复位为 '0'。

Moore 型状态机设计说明：

- 状态定义：每个状态表示当前累计已投入的金额（单位：元）。
- 状态设置：`S0`（0 元）、`S1`（1 元）、`S2`（2 元）、`S3`（3 元）、`S4`（4 元）、`S5`（5 元——出货状态）。
- 输出定义（Moore 型）：仅 `S5` 状态下 `dispense = '1'`，其余所有状态下 `dispense = '0'`。
- 状态转移规则：
 - 当前状态为 S_i （累计 i 元），若投入 1 元（`coin = "01"`），则次态为 S_{i+1} （若 $i+1 \leq 5$ ）；若投入 2 元（`coin = "10"`），则次态为 S_{i+2} （若 $i+2 \leq 5$ ）。
 - 若投入的硬币会使累计金额超过 5 元（即 $i+1 > 5$ 或 $i+2 > 5$ ），则状态保持不变（忽略该次投币）。

- 在 S5 状态下（已出货），无论输入为何值，次态均无条件回到 S0（准备下一次交易）。

示例：用户依次投入 2 元、2 元、1 元硬币。

$S0 \xrightarrow{2\text{元}} S2 \xrightarrow{2\text{元}} S4 \xrightarrow{1\text{元}} S5 \text{（出货）} \rightarrow S0$

若用户在 S4 状态下尝试投入 2 元（累计将达到 6 元 > 5 元），则该次投币被忽略，状态保持 S4。

要求：

(1) (6 分) 画出 **Moore 型状态转移图**。要求：

- 清晰标注全部 6 个状态（S0 S5）的名称和含义（以累计金额标注）；
- 标注每个状态的 `dispense` 输出值；
- 标注所有状态之间的转移条件（基于 `coin` 的不同取值），包括：投入 1 元、投入 2 元、无硬币投入、以及超额的无效投币。

(2) (4 分) 列出**状态转移表**，包含以下列：当前状态、输入 `coin`（区分“00”、“01”、“10”）、次态、输出 `dispense`。

(3) (8 分) 编写**完整的 VHDL 代码**（实体 + 结构体）实现该自动售货机控制器。要求：

- 使用用户自定义枚举类型定义所有状态：`TYPE vending_state IS (S0, S1, S2, S3, S4, S5);`
- 采用**双进程描述风格**（推荐）或三进程风格：
 - 进程 1（时序进程）：描述状态寄存器和同步复位逻辑。时钟上升沿触发，复位时回到 S0；
 - 进程 2（组合进程）：描述次态逻辑和输出逻辑。根据当前状态和输入 `coin` 确定次态，根据当前状态确定 `dispense` 输出。
- 正确使用 CASE 语句描述状态转移；
- 代码中必须包含清晰的注释，说明各状态的含义和关键转移逻辑。

(4) (2 分) 代码规范：缩进一致、信号命名合理、注释完整清晰。

提示：

- Moore 型状态机的输出只取决于当前状态，与输入无关——因此 `dispense` 信号仅在 S5 状态时为 '1'。
- 注意处理边界情况：S4 状态时，投入 2 元（总额 6 元 > 5 元）应忽略并保持 S4；投入 1 元（恰好 5 元）则进入 S5。
- S5 状态的下一个时钟周期无条件回到 S0（即每完成一次交易后，状态机自动准备好接受下一次交易）。
- 进程 2（组合逻辑）的敏感信号列表中应包含当前状态信号和 `coin` 信号。
- 若使用双进程风格，次态逻辑和输出逻辑可合并于同一个组合进程中。

——试卷结束——

第十七章 第 17 套试卷

FPGA 与 VHDL 基础试卷（十七）

（满分：100 分 考试时间：120 分钟 难度：中等）

一、选择题（每题 3 分，共 18 分）

1. 下列关于 CPLD 与 FPGA 架构差异的描述，正确的是？

- A. CPLD 基于查找表（LUT）结构实现逻辑功能；FPGA 基于乘积项（Product Term）结构实现逻辑功能
- B. CPLD 基于乘积项（AND-OR 阵列）结构，适合宽输入组合逻辑；FPGA 基于查找表（LUT）结构，适合寄存器密集型和复杂时序逻辑
- C. CPLD 的逻辑单元（宏单元）数量通常比 FPGA 多，且互连资源更丰富
- D. FPGA 的配置数据存储在非易失性 Flash 中，因此上电即可工作；CPLD 则需要外部配置芯片

2. 在 VHDL 中，下列关于 CASE 语句与 IF-ELSE 语句综合结果的说法，正确的是？

- A. CASE 语句综合后通常生成并行的多路选择器（MUX）结构；IF-ELSIF 语句综合后通常生成优先级链（Priority Chain）结构
- B. CASE 语句和 IF-ELSE 语句综合结果完全等价，综合工具会自动优化为相同电路
- C. CASE 语句只能用于组合逻辑描述，不能出现在 PROCESS 内部
- D. IF-ELSIF 语句综合后总是生成并行结构，与 CASE 语句无区别

3. 在 VHDL 结构化描述中，关于 COMPONENT（元件）声明和实例化的语法规则，以下说法错误的是？

- A. COMPONENT 声明位于 ARCHITECTURE 的声明区 (BEGIN 之前), 其 PORT 列表必须与被例化实体的 PORT 完全一致
- B. 元件实例化时使用 PORT MAP 关键字, 支持命名关联 (如 `a => sig_a`) 和位置关联两种方式
- C. 同一 COMPONENT 在一个 ARCHITECTURE 中只能被实例化一次, 多次实例化会导致综合错误
- D. COMPONENT 声明的作用是告知编译器待例化子模块的接口信息, 真正的实体-结构体绑定由综合工具通过搜索工程文件完成 (或通过 CONFIGURATION 显式指定)
4. VHDL 中 GENERIC (类属参数) 的使用规则, 正确的是?
- A. GENERIC 声明必须位于 ENTITY 的 PORT 声明之前; 实例化时使用 GENERIC MAP 为参数赋值
- B. GENERIC 只能在顶层 ENTITY 中声明, 底层模块不能拥有自己的 GENERIC 参数
- C. GENERIC MAP 的赋值只能使用位置关联方式, 不支持命名关联
- D. GENERIC 参数的值在综合后仍可通过外部引脚动态修改
5. 关于 VHDL 中的 GENERATE 语句, 下列描述错误的是?
- A. FOR ... GENERATE 用于重复生成多个相同的硬件结构实例 (如 N 位寄存器的 N 个位片); IF ... GENERATE 用于根据条件选择性地生成硬件
- B. GENERATE 语句属于并发语句 (Concurrent Statement), 必须放在 ARCHITECTURE 的 BEGIN 之后
- C. GENERATE 语句内部可以包含进程 (PROCESS)、信号赋值语句、元件实例化等并发语句
- D. GENERATE 语句的循环变量 (如 i) 需要在结构体声明区提前声明为 SIGNAL 类型, 不能在 GENERATE 内部隐式定义
6. 在有限状态机 (FSM) 设计中, 关于二进制编码与独热码 (One-Hot) 编码的比较, 正确的是?

- A. 二进制编码使用 $\lceil \log_2 N \rceil$ 个触发器表示 N 个状态，触发器用量最少；独热码使用 N 个触发器，但次态逻辑和输出译码更简单
- B. 独热码编码方式使用的触发器数量总是少于二进制编码，因此功耗更低
- C. 二进制编码的状态译码逻辑比独热码更简单，因为每一位直接对应一个状态
- D. 对于所有 FSM 设计，FPGA 综合工具默认强制使用二进制编码，无法更改

二、填空题（每空 3 分，共 12 分）

1. 在 VHDL 结构化描述中，使用关键字_____ 在 ARCHITECTURE 声明区定义待例化子模块的接口；在 BEGIN 之后使用关键字_____ 将子模块端口与顶层信号相关联。此过程称为**元件实例化**（Component Instantiation）。
2. 在 VHDL 中，GENERIC 参数必须声明在 ENTITY 的_____ 声明之前。实例化带有 GENERIC 参数的子模块时，需使用关键字_____ 为类属参数指定实际值。
3. VHDL 提供两种 GENERATE 语句：_____ GENERATE 用于按固定次数重复生成硬件结构（如 N 位全加器链）；_____ GENERATE 用于根据布尔条件选择性地生成或不生成某段硬件。
4. 在 FSM 状态编码方式中，对于具有 N 个状态的状态机：二进制编码需要_____ 个触发器（用表达式表示），独热码（One-Hot）编码需要_____ 个触发器。

三、程序改错题（共 5 分）

以下 VHDL 代码意图描述一个使用 COMPONENT 例化和 GENERATE 语句构建的 4 位行波进位加法器（Ripple Carry Adder），由一个全加器位片和一个顶层累加器组成。代码中存在 6 处错误。请找出任意 5 处，指出错误位置（行号）、错误原因及正确写法（每处 1 分，共 5 分）。

Listing 17.1: 含错误的 VHDL 代码——4 位行波进位加法器（6 处错误，找出 5 处即满分）

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  -- ===== 全加器位片（全加器 = Full Adder） =====
5  ENTITY full_adder IS
6      PORT(
7          a, b, cin : IN  STD_LOGIC;
8          sum, cout : OUT STD_LOGIC
9      );
10 END full_adder;
11
12 ARCHITECTURE dataflow OF full_adder IS
13 BEGIN
14     sum  <= a XOR b XOR cin;
15     cout <= (a AND b) OR (a AND cin) OR (b AND cin);
16 END dataflow;
17
18 -- ===== 4位行波进位加法器顶层 =====
19 LIBRARY ieee;
20 USE ieee.std_logic_1164.ALL;
21
22 ENTITY ripple_adder_4bit IS
23     PORT(
24         a, b : IN  STD_LOGIC_VECTOR(3 DOWNT0 0);
25         cin  : IN  STD_LOGIC;
26         sum  : OUT STD_LOGIC_VECTOR(3 DOWNT0 0);
27         cout : OUT STD_LOGIC
28     );
29 END ripple_adder_4bit;
30

```

```

31 ARCHITECTURE structural OF ripple_adder_4bit IS
32     COMPONENT full_add PORT(                -- 错误 1: COMPONENT 名称与实体不一
        致
33         a, b, cin : IN  STD_LOGIC;
34         sum, cout : OUT STD_LOGIC
35     );
36     END COMPONENT full_add;
37     SIGNAL carry : STD_LOGIC_VECTOR(4 DOWNT0 0); -- carry(0) = cin,
        carry(4) = cout
38 BEGIN
39     carry(0) <= cin;
40
41     gen_adders: FOR i IN 0 TO 3 GENERATE
42         u_full_adder: full_add PORT MAP(
43             a    => a(i),
44             b    => b(i),
45             cin => carry(i),      -- 正确
46             sum => sum(i),        -- 错误: PORT MAP 方向不匹配 (sum 在实体中是
                OUT, 实例化时 sum(i) 是顶层输出)
47             cout => carry(i+1)
48         );
49     END GENERATE;
50
51     cout <= carry(3);             -- 错误: 应为 carry(4) 而非 carry(3)
52
53 END structural;

```

答题格式要求: 按以下格式逐条列出 5 处错误 (每处 1 分, 共 5 分)。注意: 代码中的错误不止 5 处, 请选择 5 处作答。

错误 1 (第 ____ 行) : _____ 正确写法:

错误 2 (第 ____ 行) : _____ 正确写法:

错误 3 (第 ____ 行) : _____ 正确写法:

错误 4 (第 ____ 行) : _____ 正确写法:

错误 5 (第 ____ 行) : _____ 正确写法:

提示: 错误类型包括但不限于——(1) COMPONENT 名称与实体名不一致; (2) PORT MAP 中输出端口与顶层输出信号的连接写法问题; (3) 进位链的正确索引; (4) 实体声明段重复或位置错误; (5) 全加器端口列表中信号模式的写法。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 17.2: 分析用 VHDL 代码——模 60 BCD 计数器

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY bcd_counter_60 IS
6      PORT(
7          clk    : IN  STD_LOGIC;
8          rst    : IN  STD_LOGIC;
9          en     : IN  STD_LOGIC;
10         q      : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 高4位=十位，低4位=
                个位
11         tc     : OUT STD_LOGIC                      -- 计到59且en='1'时输
                出一个周期的高电平
12     );
13 END bcd_counter_60;
14
15 ARCHITECTURE behav OF bcd_counter_60 IS
16     SIGNAL ones : INTEGER RANGE 0 TO 9 := 0;  -- 个位 (0~9)
17     SIGNAL tens : INTEGER RANGE 0 TO 5 := 0;  -- 十位 (0~5)
18 BEGIN
19     PROCESS(clk, rst)
20     BEGIN
21         IF rst = '1' THEN
22             ones <= 0;
23             tens <= 0;
24             tc   <= '0';
25         ELSIF rising_edge(clk) THEN
26             tc <= '0';  -- 默认值
27             IF en = '1' THEN
28                 IF ones = 9 THEN
29                     ones <= 0;
30                     IF tens = 5 THEN
31                         tens <= 0;
32                         tc   <= '1';  -- 59 → 00时进位

```

```
33         ELSE
34             tens <= tens + 1;
35         END IF;
36     ELSE
37         ones <= ones + 1;
38     END IF;
39 END IF;
40 END IF;
41 END PROCESS;
42
43 -- BCD输出转换：整数 → 8位BCD码
44 q(3 DOWNTO 0) <= STD_LOGIC_VECTOR(TO_UNSIGNED(ones, 4));
45 q(7 DOWNTO 4) <= STD_LOGIC_VECTOR(TO_UNSIGNED(tens, 4));
46 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？其计数范围是多少？说明复位信号 `rst` 的复位方式是同步复位还是异步复位，并给出判断依据。
- (2) (2 分) 信号 `tc`（终端计数/Terminal Count）的有效条件是什么？在什么情况下 `tc` 输出高电平？若将该计数器级联为更高位宽的计数器（如模 600 计数器），`tc` 信号应如何连接？
- (3) (1 分) 代码第 34-35 行使用 `TO_UNSIGNED` 函数进行类型转换。请说明为什么需要此转换？若不进行此转换，直接将整数信号 `ones` 和 `tens` 赋值给 `STD_LOGIC_VECTOR` 类型的端口 `q`，会发生什么？
- (4) (1 分) 若需求改为**模 100 BCD 计数器**（计数范围 0~99，十位 0~9），需要对该代码做哪些修改？请简述修改点（不要求写完整代码）。

五、综合设计题（共 60 分）

设计题 1：4 位数值比较器设计（20 分）

设计一个 4 位数值比较器，对两个 4 位无符号二进制数 A 和 B 进行大小比较。要求使用行为描述方式（PROCESS）实现。

端口定义：

- A[3:0]（输入）：4 位无符号操作数 A。
- B[3:0]（输入）：4 位无符号操作数 B。
- GT（输出）：当 $A > B$ 时输出'1'，否则输出'0'。
- EQ（输出）：当 $A = B$ 时输出'1'，否则输出'0'。
- LT（输出）：当 $A < B$ 时输出'1'，否则输出'0'。

要求：

- (1) (4 分) 画出该比较器的顶层模块框图，标注所有端口名称、方向和位宽。列出该比较器的功能表（至少覆盖 $A > B$ 、 $A = B$ 、 $A < B$ 三种情况各一个示例）。
- (2) (10 分) 使用行为描述方式编写完整的 VHDL 代码（实体 + 结构体）。要求：
 - 在结构体中使用一个 **PROCESS**（敏感信号表包含 A 和 B）；
 - 在 PROCESS 内部使用 IF-ELSE 语句实现比较逻辑；
 - 注意：三个输出信号 GT、EQ、LT 的任何组合中有且仅有一个为'1'；
 - 所有输出信号在 PROCESS 的每条执行路径中均被赋值（避免锁存器推断）。
- (3) (4 分) 使用另外一种描述方式——选择信号赋值语句（WITH-SELECT-WHEN）——重新实现该比较器的输出逻辑。请完整写出该结构体（实体部分可省略），并说明 WITH-SELECT-WHEN 语句与 PROCESS 内 IF-ELSE 语句在综合结果上的主要差异。
- (4) (2 分) 若将该比较器扩展为 8 位比较器，使用 VHDL 的 GENERIC 参数化设计，应如何修改代码？请写出带 GENERIC 参数的 ENTITY 声明，并简要说明结构体中的修改要点（不要求写完整结构体）。

提示：

- 比较两个 STD_LOGIC_VECTOR 类型的数据时，可使用关系运算符 (>, =, <) 直接比较，需在库声明中加入 USE ieee.numeric_std.ALL 或 USE ieee.std_logic_unsigned.ALL。
- WITH-SELECT-WHEN 是并发语句，必须放在 ARCHITECTURE 的 BEGIN 之后 (PROCESS 外部)。
- WITH-SELECT-WHEN 要求所有选择值被覆盖，必要时使用 WHEN OTHERS。

设计题 2：模 60 BCD 计数器设计（20 分）

设计一个模 60 BCD 计数器，以两位 BCD 码格式（十位 0~5、个位 0~9）从 0 计数到 59，计满后返回 0。该计数器可作为数字钟的秒/分计数核心单元。

端口定义：

- **clk**（输入）：时钟信号，上升沿触发。
- **rst**（输入）：同步复位，高电平有效。复位后计数器值为 00。
- **en**（输入）：计数使能，高电平有效。为'1'时每个时钟上升沿计数值加 1。
- **load**（输入）：并行置数使能，高电平有效。为'1'时将 **data_in** 的值加载到计数器。
- **data_in[7:0]**（输入）：并行置数数据，高 4 位为十位 BCD 码（0~5），低 4 位为个位 BCD 码（0~9）。
- **q[7:0]**（输出）：当前计数值，高 4 位为十位 BCD 码，低 4 位为个位 BCD 码。
- **tc**（输出）：终端计数（Terminal Count），当计数值为 59 且 **en**='1' 时输出'1'（持续一个时钟周期），其余时间为'0'。

功能优先级（从高到低）：复位 > 并行置数 > 计数使能 > 保持。

要求：

(1)（3 分）画出该模 60 BCD 计数器的顶层模块框图，标注所有端口名称、方向和位宽。结合框图说明个位和十位之间的进位关系。

(2)（12 分）编写完整的 VHDL 代码（实体 + 结构体）。要求：

- 采用行为描述方式，使用单个 PROCESS 实现全部计数器逻辑；
- 个位和十位分别用内部信号（或变量）维护，个位计数范围 0~9，十位计数范围 0~5；
- 个位计到 9 时，下一个计数脉冲使个位归 0、十位加 1；
- 十位计到 5 且个位计到 9 时，下一个计数脉冲使两者均归 0，同时 **tc** 输出高电平；
- 并行置数时仅当 **data_in** 的十位 ≤ 5 且个位 ≤ 9 时才有效（否则保持原值）；

- 使用 `ieee.numeric_std.ALL` 包中的类型转换函数，将内部整数值转换为 `STD_LOGIC_VECTOR` 输出。

(3) (3 分) 使用 **COMPONENT** 例化方式，用两个独立的 BCD 计数器子模块（一个模 10 个位计数器、一个模 6 十位计数器）和顶层逻辑重新实现该模 60 计数器。要求：

- 画出两个子模块及顶层连接的结构框图，标注进位信号流向；
- 写出顶层模块的 COMPONENT 声明和各实例的 PORT MAP 语句（仅需写出关键连接，不要求完整 VHDL 代码）。

(4) (2 分) 回答问题：

- (a) 若将 `rst` 从同步复位改为异步复位，VHDL 代码中 PROCESS 的敏感信号表和复位判断逻辑应如何修改？
- (b) 若需要将两个模 60 计数器级联构成模 3600 计数器（用于计时秒数），简述级联方式。

提示：

- 个位和十位可使用 `INTEGER` 类型（范围约束）或 `STD_LOGIC_VECTOR` 类型，建议使用 `INTEGER` 以便算术运算。
- 并行置数时的合法性检查示例：若 `TO_INTEGER(UNSIGNED(data_in(7 DOWNTO 4))) > 5` 则十位数据非法。
- 终端计数逻辑：`tc <= '1' WHEN (tens = 5 AND ones = 9 AND en = '1') ELSE '0';`
- 子模块方案中，模 10 个位计数器的进位输出（`tc_ones`）作为模 6 十位计数器的使能输入（`en_tens`）。

设计题 3：数字密码锁——Moore 型状态机设计（20 分）

设计一个数字密码锁控制器。预设密码为 4 位数字“2-5-3-7”，用户通过两个按键以二进制方式串行输入：按键 0 输入比特‘0’，按键 1 输入比特‘1’。数字 0~9 各用 4 位二进制表示（不足 4 位高位补 0）。因此完整密码为 16 位二进制序列：

2 = “0010” → 5 = “0101” → 3 = “0011” → 7 = “0111”

完整输入序列：0,0,1,0, 0,1,0,1, 0,0,1,1, 0,1,1,1

系统描述：

- **输入信号：**clk（时钟，上升沿触发）、rst（异步复位，高电平有效）、din（1 位串行数据输入，取值为‘0’或‘1’）。
- **输出信号：**unlock（开锁信号，高电平有效。当连续正确输入 16 位密码序列后输出‘1’，持续一个时钟周期；任何一次输入错误后立即返回初始状态，unlock 保持为‘0’）。
- 用户在每个时钟周期输入 1 位（通过按键 0 或按键 1），状态机在每个时钟上升沿采样 din 并判断是否正确。
- **安全要求：**任何一位输入错误，状态机立即回到初始等待状态（IDLE），前面的正确输入全部作废。只有连续 16 位全部正确，才输出开锁信号。

要求：**(1)（5 分）Moore 型状态机——状态转移图与状态表：**

- 定义所有状态：IDLE（初始等待）以及对应 16 位密码序列的 16 个中间状态（S1~S16）和 UNLOCK 状态（共 18 个状态）。说明每个状态的含义（即当前已正确匹配到密码的第几位）。
- 画出**完整的状态转移图**（至少画出 IDLE、S1、S2、UNLOCK 及关键转移弧，中间状态可用省略号表示但需注明转移规律）。每条转移弧上标注转移条件（din=‘0’或 din=‘1’）。
- 列出**状态转移表**，包含：当前状态、输入 din、次态、输出 unlock。

(2)（3 分）One-Hot 编码定义：

- 写出所有 18 个状态的 One-Hot 编码（每个状态对应一个 18 位 STD_LOGIC_VECTOR，其中仅 1 位为‘1’）。

- (b) 若改用二进制编码, 需要多少位? 与 One-Hot 编码相比, 各有何优缺点 (从触发器数量、译码复杂度两个角度简要说明)?

(3) (10 分) 完整 VHDL 代码实现:

- 编写完整的实体和结构体 (Moore 型状态机);
- 使用 CONSTANT 定义所有状态的 One-Hot 编码值;
- 采用三进程描述风格:
 - 进程 1 (时序逻辑): 状态寄存器, 异步复位 (`rst='1'` 时进入 IDLE), 时钟上升沿状态更新;
 - 进程 2 (次态组合逻辑): 根据当前状态和输入 `din` 计算次态;
 - 进程 3 (输出组合逻辑): 根据当前状态生成输出 `unlock` (仅 UNLOCK 状态时 `unlock='1'`);
- 密码序列正确性判断: 在次态逻辑中, 根据当前所处状态和输入 `din`, 判断是否符合密码对应位的期望值;
- 代码中需有清晰的注释, 标注每个状态对应的密码位及期望输入值。

(4) (2 分) 分析与扩展:

- (a) 若密码改为 6 位数字 (如 “1-9-4-6-8-2”), 需要增加多少个状态? One-Hot 编码的位宽需要增加多少?
- (b) 为增强安全性, 可增加错误次数限制 (如连续 3 次输入错误则锁定 30 秒)。请简要说明: 需要在现有状态机基础上增加哪些状态和计数器逻辑? (不要求写代码, 仅说明设计思路)

提示:

- 密码序列共 16 位, 各位期望值依次为: 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1。
- 状态定义建议:
 - `S_IDLE` (初始/错误后返回)、`S_b01` (已正确输入第 1 位'0')、`S_b02` (已正确输入第 2 位'0')、`S_b03` (已正确输入第 3 位'1')、...、`S_b16` (已正确输入第 16 位'1')、`S_UNLOCK` (开锁状态)。

- 次态逻辑规律：当前处于状态 S_{bK} （已正确匹配前 K 位）时，若 din 等于第 $K + 1$ 位的期望值，则进入 $S_{b(K+1)}$ ；否则回到 S_IDLE 。
- One-Hot 编码示例： $IDLE = "000\dots001"$ （仅 $bit0$ 为'1'）， $S_{b01} = "000\dots010"$ （仅 $bit1$ 为'1'），依此类推（共 18 位，下标 0~17）。
- 进程 2（次态组合逻辑）中：建议使用 CASE 语句按当前状态分支，在每个分支中再用 IF 判断 din 的值。
- 进程 3（输出逻辑）中： $unlock \leq '1' \text{ WHEN } state = S_UNLOCK \text{ ELSE } '0';$ （简单的条件信号赋值即可）。

——试卷结束——

第十八章 第 18 套试卷

FPGA 与 VHDL 基础试卷（十八）

（满分：100 分 考试时间：120 分钟 难度：中等）

适用对象：正在学习 *FPGA/VHDL* 基础的本科生。重点考查 *VHDL* 基本语法、组合逻辑与时序逻辑设计、有限状态机设计方法。

一、选择题（每题 3 分，共 18 分）

1. 在 FPGA 中， n 输入的查找表（ n -input LUT）可以实现任意 n 变量组合逻辑函数，其根本原理是（ ）。
 - A. LUT 内部使用 SRAM 单元存储 2^n 个真值表值，通过 n 个输入作为地址选择对应存储值输出
 - B. LUT 内部使用若干个与门和或门动态组合实现任意逻辑函数
 - C. LUT 通过遍历所有可能的输入组合、实时计算输出值
 - D. LUT 实际上是一个 n 位计数器，通过计数遍历所有真值表行
2. 在 VHDL 进程（PROCESS）内部，下列关于信号（SIGNAL）与变量（VARIABLE）赋值语句执行时机的描述，**正确**的是（ ）。
 - A. 信号赋值（ $\leq$ ）在进程执行到该语句时立即生效；变量赋值（ $:=$ ）在进程挂起时才生效
 - B. 变量赋值（ $:=$ ）在进程执行到该语句时立即生效；信号赋值（ $\leq$ ）在进程执行完毕（挂起）后才更新目标信号的值
 - C. 信号赋值和变量赋值均在进程执行到对应语句时立即生效，无区别
 - D. 信号赋值和变量赋值均在进程挂起后才统一更新，无区别

3. 在 VHDL 中, 若组合逻辑进程 (PROCESS) 的敏感信号列表 (Sensitivity List) **不完整** (缺少某个被读取的输入信号), 综合工具通常会 ()。
- A. 给出错误信息并拒绝综合
 - B. 将缺少的信号自动补入敏感信号列表, 综合结果与完整列表完全相同
 - C. 综合出锁存器 (Latch), 因为综合工具认为该信号的变化无需触发进程重新计算——导致仿真与综合行为不一致
 - D. 忽略该问题, 仿真和综合行为完全一致
4. 下列关于 Moore 型和 Mealy 型有限状态机 (FSM) 的描述, **正确的是** ()。
- A. Moore 型的输出仅取决于当前状态, 输出变化与时钟边沿同步; Mealy 型的输出取决于当前状态和输入, 输入变化可直接影响输出
 - B. Moore 型和 Mealy 型在相同功能下状态数一定相同
 - C. Mealy 型的输出一定比 Moore 型慢一拍
 - D. Moore 型不能实现序列检测功能
5. 在 VHDL 中, 下列关于并行信号赋值语句 (Concurrent Signal Assignment) 和进程内顺序信号赋值语句的描述, **正确的是** ()。
- A. 并行信号赋值语句位于结构体 (ARCHITECTURE) 中, 对所有赋值目标同时更新; 进程内的信号赋值是顺序执行的, 但在进程结束时统一更新
 - B. 并行信号赋值和进程内信号赋值在实际电路行为上完全不同, 前者对应组合逻辑, 后者对应时序逻辑
 - C. 并行信号赋值语句只能用于描述时序逻辑
 - D. 进程内信号赋值在每条语句执行后立即更新信号值, 与并行赋值无区别

6. 在 VHDL 中，下列关于逻辑运算符优先级的排列，**正确**的是（ ）。

- A. AND 的优先级高于 OR
- B. OR 的优先级高于 AND
- C. **NOT > AND > OR > XOR**（即 NOT 优先级最高，然后依次为 AND、OR、XOR，XOR 优先级最低）
- D. 所有逻辑运算符优先级都相同，按从左到右的顺序计算

二、填空题（每题 3 分，共 12 分）

1. **变量声明与赋值：**在 VHDL 中，变量（VARIABLE）只能在_____（填“结构体”或“进程/子程序”）内部声明。变量赋值使用符号_____（填一个 2 字符符号），赋值后变量值_____（填“立即改变”或“进程结束时改变”）。
2. **信号赋值：**在 VHDL 中，信号（SIGNAL）可在结构体中声明。信号赋值使用符号_____（填一个 2 字符符号）。进程内的信号赋值语句在_____（填“语句执行时”或“进程挂起时”）才实际更新目标信号的值。
3. **进程敏感信号列表规则：**描述组合逻辑的 PROCESS,其敏感信号列表应包含_____（即所有在进程中被读取的输入信号）。若敏感信号列表不完整，综合工具可能推断出_____（填一种存储元件），导致综合前仿真与综合后仿真结果不一致。
4. **状态机三要素：**一个完整的有限状态机（FSM）通常由三个核心部分组成：(1) _____（用于保存当前状态的寄存器）；(2) _____（根据当前状态和输入决定次态的组合逻辑）；(3) _____（根据当前状态或当前状态 + 输入产生输出的逻辑）。

三、程序改错题（共 5 分）

阅读以下 VHDL 代码。该代码意图实现一个带使能的 4 位二进制加法计数器，并利用组合逻辑对计数值进行译码输出。代码中存在 6 处错误。请找出任意 5 处（每处 1 分，共 5 分），指出错误原因及正确写法。

Listing 18.1: 含错误的 VHDL 代码——找出 5 处即满分（共 6 处错误）

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  -- 错误1: 缺少 numeric_std包，却使用了加法运算
4
5  ENTITY counter_err IS
6      PORT(
7          clk      : IN  STD_LOGIC;
8          rst      : IN  STD_LOGIC;
9          en       : IN  STD_LOGIC;
10         q        : OUT STD_LOGIC_VECTOR(3 DOWNT0 0);
11         flag     : OUT STD_LOGIC
12     );
13 END counter_err;
14
15 ARCHITECTURE behav OF counter_err IS
16     SIGNAL count : STD_LOGIC_VECTOR(3 DOWNT0 0) := "0000";
17     -- 错误2: count信号在进程间共享，但声明为默认初始值（:=），综合时会
      被忽略
18 BEGIN
19     -- 计数器进程
20     PROCESS(clk)
21     BEGIN
22         IF rst = '1' THEN
23             count <= "0000";
24         ELSIF rising_edge(clk) THEN
25             IF en = '1' THEN
26                 count <= count + 1; -- 错误3: STD_LOGIC_VECTOR不能直接与整数
                                     1相加
27             END IF;
28         END IF;
29     END PROCESS;
30

```

```

31  -- 输出译码进程
32  PROCESS(count)
33  BEGIN
34      IF count = "1010" THEN
35          flag <= '1';
36      END IF;
37      -- 错误4: IF缺少ELSE分支, 且敏感信号列表中缺少flag读取的信号
38      -- 但flag未被读取, 主要问题是缺少ELSE导致锁存器推断
39  END PROCESS;
40
41  -- 错误5: 以下两个并发信号赋值语句均驱动q信号, 构成多驱动
42  q <= count;
43  q <= count WHEN rst = '0' ELSE "0000";
44
45  -- 错误6: 在PROCESS内部使用了变量但未声明
46  PROCESS(clk)
47      VARIABLE temp : STD_LOGIC;
48  BEGIN
49      IF rising_edge(clk) THEN
50          temp := en AND count(0); -- :=用于variable正确, 但en在时序逻辑
51          -- 中应同步
52          -- 此处temp声明了但未使用, 无实际功能错误, 但属于冗余代码
53      END IF;
54  END PROCESS;
55  END behav;

```

答题要求: 按以下格式逐条列出 5 处错误 (每处 1 分, 共 5 分)。注意: 代码中的错误不止 5 处, 请选择 5 处作答。

错 误 1 (第 ____ 行) : _____ 原 因:

错 误 2 (第 ____ 行) : _____ 原 因:

错 误 3 (第 ____ 行) : _____ 原 因:

错 误 4 (第 ____ 行) : _____ 原 因:

错误 5 (第 ____ 行) : _____ 原因:

提示：错误类型包括但不限于——(1) 缺少必要的 IEEE 标准包引用（如 `numeric_std`）；(2) `STD_LOGIC_VECTOR` 类型信号与整数直接进行算术运算；(3) 组合进程 IF 语句缺少 ELSE 分支导致锁存器推断；(4) 同一信号被多个并发语句驱动（多驱动冲突）；(5) 进程敏感信号列表中缺少被读取的信号。

四、程序阅读分析题（共 5 分）

阅读以下 VHDL 代码，回答问题。

Listing 18.2: 环形计数器/序列发生器——待分析 VHDL 程序

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY ring_counter IS
6      PORT(
7          clk      : IN  STD_LOGIC;
8          rst      : IN  STD_LOGIC;
9          en       : IN  STD_LOGIC;
10         q        : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
11     );
12 END ring_counter;
13
14 ARCHITECTURE behav OF ring_counter IS
15     SIGNAL sr : STD_LOGIC_VECTOR(3 DOWNTO 0);
16 BEGIN
17     PROCESS(clk)
18     BEGIN
19         IF rising_edge(clk) THEN
20             IF rst = '1' THEN
21                 sr <= "1000";
22             ELSIF en = '1' THEN
23                 sr <= sr(0) & sr(3 DOWNTO 1);
24             ELSE
25                 sr <= sr;
26             END IF;
27         END IF;
28     END PROCESS;
29
30     q <= sr;
31 END behav;
```

问题：

- (1) (1 分) 该 VHDL 代码描述的是什么电路？它在数字系统中通常有什么用途？
- (2) (1 分) 该电路属于组合逻辑还是时序逻辑？请结合代码中的 PROCESS 写法给出判断依据。
- (3) (2 分) 从复位释放后 (`rst` 从 '1' 变为 '0'、`en = '1'`)，请依次写出前 6 个时钟上升沿后输出 `q` 的值序列（以 4 位二进制形式）。该电路的一个完整循环周期包含几个时钟周期？
- (4) (1 分) 若将第 18 行的移位更新语句改为 `sr <= sr(2 DOWNT0 0) & sr(3);`，该电路的功能会发生什么变化？（仅文字描述即可）

五、综合设计题（共 60 分）

设计题 1：8-3 优先编码器设计（20 分）

设计一个 8-3 线优先编码器（Priority Encoder），输入为 8 位请求信号，输出为优先级最高的有效请求的 3 位二进制编码，同时提供群选通输出信号 GS（Group Select，当至少有一个输入有效时为'1'）。

端口定义：

- D(7 DOWNTO 0)（输入）：8 位请求输入信号，高电平有效。D(7) 优先级最高，D(0) 优先级最低。
- Y(2 DOWNTO 0)（输出）：优先级最高的有效请求对应的 3 位二进制编码。例如：若 D(5) = '1'（且 D(7)、D(6) 均为'0'），则 Y = "101"。
- GS（输出）：群选通信号。当至少有一个输入为'1' 时，GS = '1'；当所有输入均为'0' 时，GS = '0'。

功能表（部分）：

D(7..0)（有效位）	Y(2..0)	GS	说明
1xxx xxxx	111	1	D(7) 优先级最高
01xx xxxx	110	1	D(6) 有效
001x xxxx	101	1	D(5) 有效
0001 xxxx	100	1	D(4) 有效
0000 1xxx	011	1	D(3) 有效
0000 01xx	010	1	D(2) 有效
0000 001x	001	1	D(1) 有效
0000 0001	000	1	D(0) 有效
0000 0000	—	0	无请求

要求：

- (1)（3 分）画出 8-3 优先编码器的顶层端口框图，标注所有端口名称、方向和位宽。
- (2)（3 分）列出该优先编码器的完整功能表（简化形式：以 IF-ELSE 优先链顺序列出 8 种情况，从 D(7) 到 D(0)）。
- (3)（10 分）编写完整的 VHDL 代码（实体 + 结构体）。要求：

- 采用行为描述方式，使用 **PROCESS + IF-ELSIF-ELSE** 语句链实现优先级判断逻辑；
 - 当没有任何输入有效时，Y 输出为"000" 且 GS = '0'；
 - 代码中需要包含清晰的注释，说明优先级判定逻辑。
- (4) (2 分) 在代码注释中说明：为什么优先编码器的 PROCESS 敏感信号列表中应包含 D 的所有位？
- (5) (2 分) 回答：若改用条件信号赋值语句（WHEN-ELSE）来实现该优先编码器，与 PROCESS+IF-ELSIF 方式相比，两者在综合结果上有差异吗？简述理由。

提示：

- 使用 PROCESS 实现组合逻辑优先编码器的典型模板：

```
PROCESS(D)
BEGIN
    IF D(7) = '1' THEN
        Y <= "111"; GS <= '1';
    ELSIF D(6) = '1' THEN
        Y <= "110"; GS <= '1';
    ...
    ELSE
        Y <= "000"; GS <= '0';
    END IF;
END PROCESS;
```

- 注意：敏感信号列表必须包含所有 8 位输入 D，且在 IF-ELSE 链中**必须包含最终的 ELSE 分支**，否则会推断出锁存器。

设计题 2：8 位多功能移位寄存器设计（20 分）

设计一个带并行加载功能的 8 位多功能移位寄存器，支持 4 种操作模式。要求使用时序逻辑（PROCESS + 时钟）实现。

端口定义：

- `clk`（输入）：时钟信号，上升沿触发。
- `rst`（输入）：同步复位，高电平有效。复位后寄存器值为"00000000"。
- `data_in(7 DOWNTO 0)`（输入）：8 位并行输入数据（用于加载操作）。
- `mode(1 DOWNTO 0)`（输入）：2 位操作模式选择信号，定义如下：
 - "00" ——保持（Hold）：寄存器内容保持不变。
 - "01" ——逻辑左移（Shift Left）：寄存器内容整体左移 1 位，最高位移出丢弃，最低位补'0'。
 - "10" ——逻辑右移（Shift Right）：寄存器内容整体右移 1 位，最低位移出丢弃，最高位补'0'。
 - "11" ——并行加载（Parallel Load）：将 `data_in` 的值加载到寄存器中。
- `serial_in`（输入）：串行输入位（本题中左移和右移均补'0'，串行输入暂不使用——此端口保留给扩展用）。
- `q(7 DOWNTO 0)`（输出）：8 位寄存器当前值。

要求：

- (1)（3 分）画出该多功能移位寄存器的顶层端口框图，标注所有端口名称、方向和位宽。
- (2)（3 分）画出该移位寄存器的工作模式表（以 `mode` 为行、各操作模式的功能描述和次态表达式为列）。
- (3)（10 分）编写完整的 VHDL 代码（实体 + 结构体）。要求：
 - 使用一个时序 **PROCESS**（敏感信号列表包含 `clk` 即可，同步复位在 `IF rising_edge(clk)` 内部处理）；
 - 在进程内部使用 **CASE** 语句根据 `mode` 选择操作模式；

- 保持模式时寄存器值不变（自赋值或显式 NULL 处理）；
 - 代码中包含清晰的注释。
- (4) (2 分) 在代码注释中说明：为什么移位寄存器必须使用时序逻辑（边沿触发）而非组合逻辑来实现？
- (5) (2 分) 回答以下问题：若需要在该移位寄存器中增加“循环右移”（Rotate Right, 最低位移出的位回填到最高位）作为第 5 种模式，现有设计需要如何修改？（仅说明修改方案，不要求写出完整代码）

提示：

- 逻辑左移示例（ $q = "10110010"$ ）：输出"01100100"（左移 1 位，LSB 补'0'）。
- 逻辑右移示例（ $q = "10110010"$ ）：输出"01011001"（右移 1 位，MSB 补'0'）。
- 移位实现推荐使用 VHDL 拼接运算符 & 和切片：

– 左移： $q \leq q(6 \text{ DOWNTO } 0) \& '0'$ ；

– 右移： $q \leq '0' \& q(7 \text{ DOWNTO } 1)$ ；

- CASE 语句典型结构：

```
CASE mode IS
```

```
  WHEN "00" => q <= q;           -- 保持
```

```
  WHEN "01" => q <= q(6 DOWNTO 0) & '0'; -- 左移
```

```
  WHEN "10" => q <= '0' & q(7 DOWNTO 1); -- 右移
```

```
  WHEN "11" => q <= data_in;      -- 加载
```

```
  WHEN OTHERS => q <= q;
```

```
END CASE;
```

- 同步复位优先级最高： $\text{IF } rst = '1' \text{ THEN } q \leq (\text{OTHERS} \Rightarrow '0'); \text{ELSIF}$
...

设计题 3：十字路口交通灯控制器设计（20 分）

设计一个简化版十字路口交通灯控制器。路口有两条道路：主干道（Main Road, MR）和支路（Side Road, SR）。控制系统按照固定时序循环切换交通灯状态，使用 Moore 型状态机实现，内部自带计数器计时。

系统描述：

- 每条道路有红（R）、黄（Y）、绿（G）三盏灯，共 6 路控制信号：
 - 主干道：MR_R、MR_Y、MR_G
 - 支路：SR_R、SR_Y、SR_G
- 所有灯控信号均为高电平点亮（'1'= 亮，'0'= 灭）。
- 基本循环（固定周期）：

阶段	主干道（MR）	支路（SR）	持续时间	说明
S0	绿灯	红灯	30 秒	主干道通行
S1	黄灯	红灯	5 秒	主干道过渡
S2	红灯	绿灯	20 秒	支路通行
S3	红灯	黄灯	5 秒	支路过渡

- 状态循环：S0 → S1 → S2 → S3 → S0 → …。
- 系统时钟假设为 1Hz（即每时钟周期为 1 秒）。
- 安全互锁要求：任何时刻，主干道和支路的绿灯不能同时点亮（即 MR_G = '1' 时必有 SR_G = '0'，反之亦然）。

端口定义：

- clk（输入）：系统时钟（1Hz）。
- rst（输入）：异步复位，高电平有效。复位后进入 S0 状态（主干道绿灯、支路红灯），内部计数器清零。
- MR_R, MR_Y, MR_G（输出）：主干道红、黄、绿灯控制信号。
- SR_R, SR_Y, SR_G（输出）：支路红、黄、绿灯控制信号。

要求：

(1) (4 分) 状态图设计：

- (a) 定义所有状态 (S0~S3)，说明每个状态的含义以及该状态下的 6 路输出信号值 (以表格形式：每个状态一行，列分别为状态名、MR_R/Y/G、SR_R/Y/G、持续时间)。
- (b) 画出 Moore 型状态转移图，标注：每个状态的名称、状态下的输出值 (MR_R MR_Y MR_G / SR_R SR_Y SR_G)，以及状态间的转移条件 (计时到信号)。

(2) (4 分) 状态转移表：列出完整的状态转移表，格式如下：

当前状态	计时条件	次态	输出 (MR_R MR_Y MR_G, SR_R SR_Y SR_G)
S0	计时 < 30s	S0	
S0	计时 ≥ 30s	S1	
S1	...	...	
...	...	...	

(3) (10 分) 编写完整 VHDL 代码 (实体 + 结构体)：

- 使用用户自定义枚举类型定义状态:TYPE traffic_state IS (S0, S1, S2, S3);
- 采用三进程描述风格：
进程 1: 时序进程——状态寄存器 (state <= next_state) + 内部计数器 (进入新状态时清零，否则每个时钟上升沿 +1) +rst 异步复位；
进程 2: 组合进程——次态逻辑 (根据当前状态和计数器值决定次态)；
进程 3: 组合进程——输出逻辑 (根据当前状态驱动 6 路输出信号)。
• 内部信号要求：
 - state, next_state : traffic_state; (当前状态和次态信号)
 - timer : INTEGER RANGE 0 TO 29; (内部计数器，最大值只需到 29 即可，因为最长计时为 S0 的 30 秒，计数范围为 0~29)
- 各状态时间参数定义：
 - S0 持续时间: 30 秒 (timer 从 0 计数到 29，当 timer = 29 时计时到)

- S1 持续时间: 5 秒 (timer 从 0 计数到 4)
- S2 持续时间: 20 秒 (timer 从 0 计数到 19)
- S3 持续时间: 5 秒 (timer 从 0 计数到 4)
- 代码规范: 缩进清晰、信号命名合理、关键逻辑有中文注释。

(4) (2 分) 回答问题:

- (a) 本设计中为什么选择 Moore 型而非 Mealy 型状态机? 从输出时序稳定性和安全性角度给出理由。
- (b) 若将状态 S2 (支路绿灯) 的持续时间由 20 秒改为可配置 (通过输入端口 `green_time[5:0]` 设置, 范围为 1~63 秒), 状态机设计需要做哪些修改? (简要说明修改方案, 不要求写出完整代码)

提示:

- 计数器设计参考 (在进程 1 中):

```
PROCESS(clk, rst)
BEGIN
    IF rst = '1' THEN
        state <= S0;
        timer <= 0;
    ELSIF rising_edge(clk) THEN
        state <= next_state;
        IF state /= next_state THEN
            timer <= 0;    -- 状态切换时清零
        ELSE
            timer <= timer + 1;
        END IF;
    END IF;
END PROCESS;
```

- 次态逻辑参考 (在进程 2 中):

```
PROCESS(state, timer)
BEGIN
```

```
CASE state IS
  WHEN S0 =>
    IF timer >= 29 THEN next_state <= S1;
    ELSE next_state <= S0; END IF;
  WHEN S1 =>
    IF timer >= 4 THEN next_state <= S2;
    ELSE next_state <= S1; END IF;
  -- ... 其他状态类似
END CASE;
END PROCESS;
```

- 输出逻辑参考（在进程 3 中）：

```
PROCESS(state)
BEGIN
  CASE state IS
    WHEN S0 =>
      MR_R <= '0'; MR_Y <= '0'; MR_G <= '1';
      SR_R <= '1'; SR_Y <= '0'; SR_G <= '0';
    WHEN S1 =>
      MR_R <= '0'; MR_Y <= '1'; MR_G <= '0';
      SR_R <= '1'; SR_Y <= '0'; SR_G <= '0';
    -- ... 其他状态类似
  END CASE;
END PROCESS;
```

- 注意：每个状态均有完整输出赋值，避免推断出锁存器。所有信号在每个 CASE 分支中都必须被赋值。
- 安全互锁验证：检查所有状态的输出，确保不存在 $MR_G = '1'$ 和 $SR_G = '1'$ 同时发生的情况。

——试卷结束——

第十九章 第 19 套试卷

FPGA 与 VHDL 基础试卷（十九）

（满分：100 分 考试时间：120 分钟 难度：中等）

适用对象：正在学习 *FPGA/VHDL* 基础的学生。考查 *VHDL* 基本数据类型、并发语句、进程执行模型、*FPGA* 配置方式、子程序及基本数字系统设计能力。

一、选择题（共 6 题，每题 3 分，共 18 分）

1. 在 VHDL 中，STD_LOGIC 与 STD_ULOGIC 是两种常用的逻辑数据类型。关于两者的区别，以下描述正确的是（ ）。

- A. STD_LOGIC 有 9 种取值 ('U','X','0','1','Z','W','L','H','-')，而 STD_ULOGIC 只有 4 种取值 ('0','1','Z','X')
- B. STD_LOGIC 是决断类型 (Resolved Type)，包含决断函数，允许多个驱动源同时驱动同一信号；STD_ULOGIC 是非决断类型 (Unresolved Type)，若有多个驱动源则产生错误
- C. STD_LOGIC 只能用于内部信号声明，不能用于端口；STD_ULOGIC 只能用于端口声明
- D. 两者在综合实现上产生完全不同的硬件电路，不能互换使用

2. SIGNED 和 UNSIGNED 数据类型定义于 IEEE.NUMERIC_STD 包中，常用于算术运算。关于两者的描述，以下错误的是（ ）。

- A. SIGNED 以二进制补码 (Two's Complement) 形式表示有符号数, UNSIGNED 表示无符号整数
 - B. 两者本质上都是 STD_LOGIC 元素组成的一维数组, 但各自重载了不同的算术运算符 (+、-、* 等) 以匹配有符号/无符号语义
 - C. SIGNED 和 UNSIGNED 可以不经类型转换直接与 STD_LOGIC_VECTOR 进行算术运算
 - D. 使用 NUMERIC_STD 包替代过时的 STD_LOGIC_ARITH/STD_LOGIC_UNSIGNED 包, 是 IEEE 推荐的做法
3. VHDL 中, 条件信号赋值语句 (WHEN-ELSE) 和选择信号赋值语句 (WITH-SELECT) 是两种常用的并发语句。关于两者, 以下描述**正确**的是 ()。
- A. WHEN-ELSE 和 WITH-SELECT 都是顺序语句, 只能在 PROCESS 内部使用
 - B. WHEN-ELSE 中各条件按书写顺序依次判断 (具有优先级), 最先满足的条件对应的值被选中; 而 WITH-SELECT 中所有分支是平等 (并行) 判断的, 不允许有分支重叠
 - C. WITH-SELECT 语句中可以不使用 WHEN OTHERS 分支, 只要所有显式列出的分支覆盖了选择表达式的全部可能取值即可
 - D. WHEN-ELSE 和 WITH-SELECT 的综合结果总是完全相同的组合逻辑电路, 无任何区别
4. 下列关于 VHDL 中 PROCESS (进程) 执行模型的描述, **正确**的是 ()。

- A. 一个带敏感信号列表的 PROCESS 在仿真中, 当敏感列表中的任一信号发生事件(值变化)时被激活, 执行完内部语句后挂起等待下一次事件
 - B. PROCESS 内部对同一信号的多次赋值中, 只有第一次赋值生效, 后续赋值被忽略
 - C. 在 PROCESS 的敏感信号列表中只写时钟信号 `clk`, 但进程中使用了 `rising_edge(clk)`, 综合工具会自动推断出 `rst` 也为敏感信号
 - D. 不带敏感信号列表的 PROCESS (含 WAIT 语句) 与带敏感信号列表的 PROCESS 在综合时没有任何区别
5. FPGA 器件的配置(Configuration)是指将比特流数据加载到 FPGA 内部的 SRAM 配置存储器中的过程。以下关于 FPGA 常用配置模式的描述, 错误的是 ()。
- A. 主串模式 (Master Serial): FPGA 主动产生配置时钟 CCLK, 从外部 SPI Flash/PROM 逐位串行读取配置数据
 - B. 从串模式 (Slave Serial): 由外部处理器或下载器提供 CCLK, 将配置数据逐位送入 FPGA
 - C. JTAG 模式 (Boundary Scan): 仅能用于芯片的边界扫描测试和在线调试, 不能用于 FPGA 的初始配置下载
 - D. 主并模式 (Master Parallel/SelectMAP) : FPGA 产生控制信号, 从外部并行 NOR Flash 以 8 位或 16 位宽度读取配置数据, 速度较串行模式更快
6. VHDL 子程序包括 FUNCTION (函数) 和 PROCEDURE (过程)。关于两者的使用, 以下描述正确的是 ()。

- A. FUNCTION 可以有多个 RETURN 语句以返回不同值；PROCEDURE 必须至少包含一个 RETURN 语句
- B. FUNCTION 可以出现在表达式中（如赋值语句右侧），而 PROCEDURE 的调用是一条独立的顺序语句
- C. FUNCTION 和 PROCEDURE 都可以在内部包含 WAIT 语句，以暂停执行等待信号事件
- D. PROCEDURE 的参数模式默认为 IN，因此无须显式指定；FUNCTION 的参数模式默认为 OUT

二、填空题（共 4 题，每题 3 分，共 12 分）

1. `std_logic` 的 9 值系统：VHDL 中 `STD_LOGIC` 类型（定义于 `IEEE.STD_LOGIC_1164` 包）是一个 9 值逻辑系统，用于精确建模数字电路的各种物理状态。请根据下表中的取值字符，填写各字符对应的含义（中文或英文均可）：

字符	含义	字符	含义
'U'	_____（未初始化）	'X'	_____（强未知/冲突）
'0'	强逻辑 0	'1'	_____（强逻辑 1）
'Z'	_____（高阻态）	'W'	_____（弱未知）
'L'	_____（弱逻辑 0）	'H'	_____（弱逻辑 1）
'-'	_____（无关项）		

2. 函数（`FUNCTION`）与过程（`PROCEDURE`）的参数与返回：

- `FUNCTION` 的所有形式参数的模式只能为_____（若不写模式关键字，默认为此模式）；`FUNCTION` 体内**必须**包含_____语句，向调用者返回计算结果。
- `PROCEDURE` 的参数模式有三种：_____（输入）、_____（输出）和 `INOUT`（双向）。`PROCEDURE` 将运算结果通过_____模式参数传递给调用者（不需要 `RETURN` 语句）。
- 调用 `FUNCTION` 时，返回值可出现在_____（例如赋值语句的右侧、`IF` 条件表达式等）；调用 `PROCEDURE` 则是一条独立的_____语句（不能嵌套在表达式中）。

3. 程序包（`PACKAGE`）的结构：

- 一个完整的 VHDL 程序包由两个设计单元组成：第一部分称为_____（`PACKAGE` 声明），其中放置对外可见的_____（至少写出两种：如常量声明、类型声明）、元件声明以及子程序的**接口声明**（即 `FUNCTION/PROCEDURE` 的原型）；第二部分称为_____（`PACKAGE BODY`），其中包含第一部分所声明的子程序的_____（即具体实现代码）。

- 如果 PACKAGE 声明中只包含常量、类型和信号声明而没有声明任何子程序,则该 PACKAGE_____ (填“需要”或“不需要”)配套的 PACKAGE BODY。

4. 并发语句与顺序语句:

- 出现在 ARCHITECTURE(结构体)中、PROCESS 之外的语句属于_____ 语句(Concurrent Statement)。它们的执行顺序与代码中的书写顺序_____ (填“有关”或“无关”)。
- 出现在 PROCESS、FUNCTION 或 PROCEDURE 内部的语句属于_____ 语句 (Sequential Statement)。它们按照代码中的书写顺序_____ 执行。
- 常见的并发信号赋值语句包括: 简单并发信号赋值 ($\leq$)、条件信号赋值 (_____) 和选择信号赋值 (_____)。

三、程序改错题（共 1 题，共 5 分）

题目：以下 VHDL 代码意图实现一个带复位功能的 4 位计数器，并将计数值通过 7 段数码管译码输出。代码中存在 **6 处错误**，请找出任意 **5 处**（共 6 处错误，找出 5 处即满分），指出错误位置、错误原因及正确写法。

Listing 19.1: 含多类错误的 VHDL 代码——找出 5 处即满分（共 6 处错误）

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.STD_LOGIC_ARITH.ALL;
4  USE IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  ENTITY counter_seg_err IS
7      PORT(
8          clk      : IN  STD_LOGIC;
9          rst      : IN  STD_LOGIC;
10         en       : IN  STD_LOGIC;
11         seg      : OUT STD_LOGIC_VECTOR(6 DOWNT0 0)
12     );
13 END counter_seg_err;
14
15 ARCHITECTURE behav OF counter_seg_err IS
16     SIGNAL count : STD_LOGIC_VECTOR(3 DOWNT0 0);
17 BEGIN
18     -- Process 1: Counter
19     -- Error 1: 异步复位进程中敏感信号列表不全（缺少 clk），
20     --          且异步复位写法与 rising_edge 混用不规范
21     PROCESS(rst)
22     BEGIN
23         IF rst = '1' THEN
24             count <= "0000";
25         ELSIF rising_edge(clk) THEN
26             IF en = '1' THEN
27                 count <= count + '1';
28             END IF;
29         END IF;
30     END PROCESS;
31
32     -- Process 2: 7-segment decoder

```

```

33  -- Error 2: 敏感信号列表中缺少 count, 导致组合逻辑行为错误
34  PROCESS
35  BEGIN
36      CASE count IS
37          WHEN "0000" => seg <= "1000000"; -- 显示 0
38          WHEN "0001" => seg <= "1111001"; -- 显示 1
39          WHEN "0010" => seg <= "0100100"; -- 显示 2
40          WHEN "0011" => seg <= "0110000"; -- 显示 3
41          WHEN "0100" => seg <= "0011001"; -- 显示 4
42          WHEN "0101" => seg <= "0010010"; -- 显示 5
43          WHEN "0110" => seg <= "0000010"; -- 显示 6
44          WHEN "0111" => seg <= "1111000"; -- 显示 7
45          WHEN "1000" => seg <= "0000000"; -- 显示 8
46          WHEN "1001" => seg <= "0010000"; -- 显示 9
47          -- Error 3: 缺少 WHEN OTHERS 分支
48          --          当 count 为 "1010"~"1111" 时输出未定义, 综合可能产生锁
              存器
49      END CASE;
50  END PROCESS;
51
52  -- Error 4: STD_LOGIC_ARITH/STD_LOGIC_UNSIGNED 是过时的非标准包
53  --          应使用 IEEE.NUMERIC_STD.ALL
54  --
55  -- Error 5: count 是 STD_LOGIC_VECTOR 类型, 不能直接与字符 '1' 做算术运
              算
56  --          应使用 NUMERIC_STD 包并进行类型转换
57  --
58  -- Error 6: 进程 2 没有敏感信号列表也没有 WAIT 语句,
59  --          仿真时将无限循环 (组合进程的正确写法应包含敏感列表)
60  END behav;

```

答题要求: 按以下格式逐条列出 5 处错误 (每处 1 分, 共 5 分):

错 误 1 (第 ____ 行) : _____ 改正为

 (错误原因: _____)

错 误 2 (第 ____ 行) : _____ 改正为

(错误原因:_____)

错 误 3 (第 ____ 行) : _____ 改 正 为 _____

(错误原因:_____)

错 误 4 (第 ____ 行) : _____ 改 正 为 _____

(错误原因:_____)

错 误 5 (第 ____ 行) : _____ 改 正 为 _____

(错误原因:_____)

提示：错误类型涉及——(1) 异步复位进程中敏感信号列表不完整导致仿真-综合不匹配；(2) 过时的 IEEE 标准包引用（应使用 NUMERIC_STD）；(3) CASE 语句缺少 WHEN OTHERS 分支导致锁存器推断；(4) 组合进程缺少敏感信号列表（或 WAIT 语句）；(5) STD_LOGIC_VECTOR 类型信号与字符字面量 1 进’行算术运算时类型不匹配。

四、程序阅读分析题（共 1 题，共 5 分）

题目：阅读以下 VHDL 代码——这是一个“101”序列检测器（Sequence Detector）的状态机实现，回答下列问题。

Listing 19.2: “101”序列检测器——待分析 VHDL 程序

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY seq_detector IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;
8          din      : IN  STD_LOGIC;
9          detect   : OUT STD_LOGIC
10     );
11 END seq_detector;
12
13 ARCHITECTURE fsm OF seq_detector IS
14     TYPE state_type IS (S0, S1, S2);
15     SIGNAL current_state, next_state : state_type;
16 BEGIN
17     -- 进程1: 状态寄存器（时序逻辑）
18     PROCESS(clk, rst)
19     BEGIN
20         IF rst = '1' THEN
21             current_state <= S0;
22         ELSIF rising_edge(clk) THEN
23             current_state <= next_state;
24         END IF;
25     END PROCESS;
26
27     -- 进程2: 次态逻辑（组合逻辑）
28     PROCESS(current_state, din)
29     BEGIN
30         CASE current_state IS
31             WHEN S0 =>
32                 IF din = '1' THEN
33                     next_state <= S1;
```

```
34         ELSE
35             next_state <= S0;
36         END IF;
37
38     WHEN S1 =>
39         IF din = '0' THEN
40             next_state <= S2;
41         ELSE
42             next_state <= S1;
43         END IF;
44
45     WHEN S2 =>
46         IF din = '1' THEN
47             next_state <= S1;
48         ELSE
49             next_state <= S0;
50         END IF;
51     END CASE;
52 END PROCESS;
53
54 -- 进程3: 输出逻辑 (Moore型——输出仅依赖于当前状态)
55 PROCESS(current_state)
56 BEGIN
57     IF current_state = S2 THEN
58         detect <= '1';
59     ELSE
60         detect <= '0';
61     END IF;
62 END PROCESS;
63 END fsm;
```

问题:

- (1) (1 分) 该序列检测器的目标检测序列是什么? 请结合状态转移逻辑说明该状态机属于 Moore 型还是 Mealy 型。(提示: 观察输出信号 `detect` 是否仅依赖于当前状态, 还是同时依赖于输入 `din`。)
- (2) (1 分) 该状态机包含几个状态 (S0~S2)? 分别用一句话描述每个状态的含义

(即：处于该状态时，表明输入序列已匹配到什么阶段)。

- (3) (2 分) 假设复位后输入序列 `din` 依次为：1, 0, 1, 0, 1, 1, 0, 1, 0, 1
(每个值对应一个时钟周期)，请按下列表格格式填写每个时钟周期后的
`current_state` 和 `detect` 输出值：

时钟周期	1	2	3	4	5	6	7	8	9	10
<code>din</code>	1	0	1	0	1	1	0	1	0	1
<code>current_state</code>										
<code>detect</code>										

并回答：在哪些时钟周期 `detect` 输出为'1'？

- (4) (1 分) 该代码是否支持**重叠检测** (Overlapping Detection)? 即输入序列 “10101”
是否可以检测到两次目标序列 (第 1~3 位和第 3~5 位各形成一次 “101”)? 请
结合 S2 状态下 `din='1'` 时的转移目标 (S1 而不是 S0) 进行分析。

五、综合设计题（共 3 题，共 60 分）

设计题 1：4 位多功能 ALU（算术逻辑单元）设计（20 分）

设计一个 4 位 ALU（Arithmetic Logic Unit），支持加法、减法、按位与、按位或、按位异或和按位取反共 6 种运算。采用行为描述风格（PROCESS + CASE 语句）实现。

端口定义：

- `a(3 DOWNTO 0)`（输入）：操作数 A，4 位。
- `b(3 DOWNTO 0)`（输入）：操作数 B，4 位。
- `op_sel(2 DOWNTO 0)`（输入）：3 位操作码，定义如下：

op_sel	运算名	功能
"000"	ADD	加法: <code>result = a + b</code>
"001"	SUB	减法: <code>result = a - b</code>
"010"	AND	按位与: <code>result = a AND b</code>
"011"	OR	按位或: <code>result = a OR b</code>
"100"	XOR	按位异或: <code>result = a XOR b</code>
"101"	NOT	按位取反: <code>result = NOT a</code> （仅对 a 操作，忽略 b）
"110"/"111"	—	保留，输出全 0

- `result(3 DOWNTO 0)`（输出）：4 位运算结果。
- `carry`（输出）：进位/借位标志。加法有进位或减法有借位时输出'1'；其他运算时输出'0'。
- `zero`（输出）：零标志。当 `result` 为全零（"0000"）时输出'1'，否则输出'0'。

要求：

- (3 分) 列出该 ALU 的功能表。表头应包含：操作码 `op_sel`、运算名称、运算表达式（用端口信号名表示）以及标志位 `carry` 和 `zero` 的行为说明。
- (3 分) 画出该 ALU 的顶层端口框图,标注所有端口名称、方向及位宽(以 `op_sel=3` 位、数据通路 4 位标注)。
- (10 分) 编写完整的 VHDL 代码（实体 + 结构体）。要求：
 - 使用库 `IEEE.STD_LOGIC_1164.ALL` 和 `IEEE.NUMERIC_STD.ALL`;

- 在结构体中定义一个组合逻辑 **PROCESS**，敏感信号列表包含所有输入信号 (`a`, `b`, `op_sel`);
- 在 **PROCESS** 内部使用 **CASE** 语句 (以 `op_sel` 为选择表达式) 实现 6 种运算分支，并包含 **WHEN OTHERS** 分支;
- 加法/减法分支中,使用 **VARIABLE**(变量)定义 5 位中间结果(如 **VARIABLE** `tmp` : **UNSIGNED**(4 **DOWNTO** 0)), 将 4 位操作数扩展为 5 位后进行运算, 以捕获进位/借位 (第 4 位);
- 按位与/或/异或/取反使用 VHDL 内置逻辑运算符直接对 **STD_LOGIC_VECTOR** 操作;
- `zero` 标志在 **PROCESS** 中根据 `result` 的值进行判断赋值;
- 代码中必须包含清晰的中文注释, 说明每个 **CASE** 分支的运算功能和标志位产生逻辑。

(4) (4 分) 回答以下问题:

- (a) (2 分) 在 ALU 设计的 **PROCESS** 中, 为什么加法和减法分支推荐使用 **VARIABLE** (变量) 而非 **SIGNAL** (信号) 来存储中间运算结果? 请从 VHDL 中变量 (`:=` 立即赋值) 与信号 (`<=` 延迟赋值) 的语义差异角度说明。
- (b) (2 分) 若需要将该 4 位 ALU 扩展为 8 位 ALU, 需要修改哪些部分? 请逐一列出: 端口位宽、内部变量位宽以及 **CASE** 分支的位宽适配方式。

提示:

- 类型转换示例: `a_uns := UNSIGNED(a);` (需要先将 **STD_LOGIC_VECTOR** 转换为 **UNSIGNED** 再参与算术运算)。
- 加法进位检测思路: `tmp := ('0' & a_uns) + ('0' & b_uns);` 结果 `tmp(4)` 即为进位位。
- 减法借位检测思路: `tmp := ('0' & a_uns) - ('0' & b_uns);` 结果 `tmp(4)='1'` 时表示借位 (即 `a < b`)。
- 变量声明位置: 在 **PROCESS** 的声明区 (**BEGIN** 关键字之前) 声明, 不需要在 **ARCHITECTURE** 声明区声明。
- 输出赋值: 最终将中间变量的低 4 位赋值给 `result` (`result <= STD_LOGIC_VECTOR(tmp(3 DOWNTO 0));`)。

设计题 2：可编程分频器（Programmable Frequency Divider）设计（20 分）

设计一个参数化的可编程时钟分频器，通过 GENERIC 参数 DIV 设定分频比。分频器将输入高频时钟 `clk_in` 降频输出为 `clk_out`，并具备使能控制功能。

功能规格：

- 分频关系： $f_{out} = f_{in}/DIV$ ，其中 DIV 为 GENERIC 参数（默认值 8，整数，范围 $2 \sim 65535$ ）。
- 偶数 DIV 时，输出 `clk_out` 的占空比必须为严格的 50%。
- 奇数 DIV 时，输出占空比尽力接近 50%（高电平持续 $(DIV+1)/2$ 个 `clk_in` 周期，低电平持续 $(DIV-1)/2$ 个 `clk_in` 周期）。
- 使能信号 `en`：当 `en='1'` 时分频器正常工作；当 `en='0'` 时分频器暂停，`clk_out` 保持当前电平不变，计数器停止计数。
- 同步复位 `rst`（高电平有效）：复位后计数器清零，`clk_out` 输出'0'。

端口定义：

- **GENERIC 参数**：DIV : INTEGER := 8——分频比（须置于 PORT 声明之前）。
- `clk_in`（输入）：输入高频时钟信号。
- `rst`（输入）：同步复位，高电平有效。
- `en`（输入）：使能信号，高电平有效。
- `clk_out`（输出）：分频后的输出时钟信号。

要求：

- (1)（4 分）说明该分频器的工作原理。以 DIV=6（偶数）和 DIV=5（奇数）为例，分别画出两种情况下的时序波形图（至少包含 2 个完整的 `clk_out` 周期），标注：

- `clk_in` 波形；
- 内部计数器值的变化；
- `clk_out` 的翻转时刻及对应的计数值；
- 占空比标注。

(2) (2 分) 写出分频器内部计数器的计数范围以及翻转阈值的计算公式:

- 计数器计数范围: 从 0 到_____。
- 偶数 DIV 时, `clk_out` 的翻转计数值 = _____ (填写表达式); 计数器的清零计数值 = _____。
- 奇数 DIV 时, `clk_out` 的翻转计数值 = _____ (填写表达式)。

(3) (10 分) 编写完整的 VHDL 代码 (实体 + 结构体)。要求:

- 实体中使用 `GENERIC` 声明 `DIV` (默认值 8), 置于 `PORT` 声明之前;
- 内部使用 `INTEGER RANGE 0 TO DIV-1` 类型的信号作为分频计数器;
- 使用 `CONSTANT` 定义翻转阈值 `HALF` (即高电平时钟周期数), 提升代码可读性;
- 采用单进程时序描述风格 (`PROCESS(clk_in)`), 内部仅响应 `rising_edge(clk_in)`:
 - 同步复位 (`rst='1'`): 计数器清零, `clk_out <= '0'`;
 - `en='1'` 时计数器自增, 并在达到翻转阈值和清零阈值时分别执行 `clk_out` 翻转和计数器归零;
 - `en='0'` 时计数器保持, `clk_out` 保持;
- 代码中必须包含关键逻辑的中文注释 (包括翻转条件、偶数/奇数分频的处理)。

(4) (4 分) 回答以下问题:

- (a) (1.5 分) 本设计采用**同步复位**而非异步复位。请从 FPGA 时序收敛和抗毛刺干扰能力两个角度说明同步复位的优势。
- (b) (1.5 分) 如果 `DIV=1` (即要求不分频, `clk_out` 直通 `clk_in`), 当前的设计能否正确工作? 如果当前计数器范围为 `0 TO DIV-1` (即 `0 TO 0`), `clk_out` 的行为将如何? 请说明理由, 并提出必要的修改方案。
- (c) (1 分) `GENERIC` 参数 `DIV` 是在综合前确定的常量。若需要在系统运行过程中**动态修改**分频比 (即通过输入端口改变 `DIV` 的值), 应如何修改设计? 简要说明修改思路 (不要求写出完整代码)。

提示：

- 偶数 $DIV=8$ 时：计数器 0~7 循环；计数值 0~3 时 `clk_out` 为高电平（4 个周期），4~7 时为低电平（4 个周期），占空比 50%。
- 奇数 $DIV=5$ 时：计数器 0~4 循环；计数值 0~2 时 `clk_out` 为高电平（3 个周期），3~4 时为低电平（2 个周期），占空比 60%/40%。
- 翻转阈值（高电平持续周期数）的计算公式：
 $HALF := DIV/2$ （偶数 DIV 时）
 $HALF := (DIV+1)/2$ （奇数 DIV 时）
- 翻转逻辑：当计数值 = $HALF - 1$ 时翻转 `clk_out`（使用 `NOT clk_out`）；当计数值 = $DIV - 1$ 时计数器归零。
- 同步复位写法：所有操作均在 `IF rising_edge(clk_in) THEN` 内部判断 `rst`。

设计题 3：三层电梯控制器——Moore 型状态机设计（20 分）

设计一个简化版的三层电梯控制器（Elevator Controller）。电梯在一栋 3 层建筑内运行（楼层编号为 1、2、3）。采用 Moore 型有限状态机实现电梯的自动调度与控制。

系统描述：

电梯接收来自各楼层的呼叫请求，根据请求方向决定上升或下降。电梯每经过一个“时钟周期”移动一层（从第 n 层到第 $n+1$ 层或第 $n-1$ 层需要 1 个时钟周期）。楼层传感器在电梯到达某层时给出当前位置信号。

输入信号：

- `clk`（时钟）：系统工作时钟，每个上升沿代表电梯完成一个动作（移动一层或停靠判断）。
- `rst`（异步复位，高电平有效）：复位后电梯回到 1 层，处于空闲停靠状态。
- `f1`, `f2`, `f3`（楼层呼叫按钮，各 1 位）：分别对应 1 层、2 层、3 层的呼叫请求。高电平表示该层有乘梯请求（假设按钮信号持续保持直到被服务，即电梯到达该层并停靠后由外部逻辑清零）。
- `at_floor(1 DOWNTO 0)`（楼层位置传感器）：编码电梯当前所在楼层——“00”表示 1 层，“01”表示 2 层，“10”表示 3 层。当电梯在两层之间移动时，该信号保持前一停靠楼层的值，直到到达新楼层后才更新。

输出信号：

- `motor_up`（上升控制）：‘1’时电机驱动电梯上升一层。
- `motor_down`（下降控制）：‘1’时电机驱动电梯下降一层。
- `motor_stop`（停止/开门）：‘1’时电梯停靠当前楼层并开门。
- 互锁约束：三个输出信号 `motor_up`、`motor_down` 和 `motor_stop` 在同一时刻有且仅有其中一个为‘1’。
- `curr_floor(1 DOWNTO 0)`（当前楼层显示）：编码电梯当前所在楼层（编码同 `at_floor`），用于楼层指示灯。

电梯调度规则（简化版）：

- (1) 复位后，电梯停在 1 层，`motor_stop='1'`，`curr_floor="00"`。
- (2) 电梯在任意楼层停靠时，检查是否有任何楼层的呼叫请求（`f1`、`f2` 或 `f3`）：

- 若无任何请求，则保持当前楼层停靠（空闲状态）。
 - 若有请求且存在高于当前楼层的请求，则电梯启动上升（`motor_up='1'`）。
 - 若有请求且只有低于当前楼层的请求（无更高楼层请求），则电梯启动下降（`motor_down='1'`）。
- (3) 电梯在移动过程中（上升或下降），每经过一个时钟周期到达相邻楼层。到达每一层时，检查该层是否有请求或被包含在当前运行方向的服务范围内——如果该层有请求且运行方向匹配，则停靠（`motor_stop`）；否则继续移动。
- (4) 当电梯到达当前运行方向上的最远请求楼层后，停靠并重新判断：若有反向请求则改变方向，若无则进入空闲。
- (5) 简化：电梯在上升过程中只服务上方向的请求（被呼叫楼层 > 当前楼层），下降过程中只服务下方向的请求。

要求：

(1) 状态图与状态定义（6 分）

- (a)（2 分）定义 Moore 型状态机的所有状态，说明每个状态的含义及对应的输出值（`motor_up`, `motor_down`, `motor_stop`）。建议至少包含以下状态：

- IDLE_F1: 停靠在 1 层空闲
- IDLE_F2: 停靠在 2 层空闲
- IDLE_F3: 停靠在 3 层空闲
- MOVING_UP: 电梯正在上升
- MOVING_DOWN: 电梯正在下降
- STOPPED: 电梯在某层开门停靠（短暂停留）

（允许根据设计需要适当增删状态，但须保证功能完整。）

- (b)（4 分）画出完整的状态转移图（Moore 型）。要求：

- 每个状态节点内标注：状态名称和输出值（以三元组（`up,down,stop`）的形式，如（0,0,1）表示停止）；
- 每条转移弧上标注转移条件（如：`at_floor="01" AND (f3='1')`、`(f1 OR f2 OR f3)='0'` 等）；
- 图面清晰，状态布局合理。

(2) 状态转移表 (3 分)

列出 Moore 型状态转移表。建议表头为：

当前状态 stop	输入条件	次态	up	down

至少覆盖 12 行（覆盖各状态的主要转移情况）。

(3) VHDL 代码实现 (8 分)

编写完整的电梯控制器 VHDL 代码（实体 + 结构体）。要求：

- 实体端口定义与上述系统描述完全一致；
- 使用用户自定义枚举类型定义所有状态（如 `TYPE elevator_state IS (...);`）；
- 采用三进程描述风格（Moore 型状态机的标准模板）：
 - 进程 1（时序）：状态寄存器 + 异步复位（`rst='1'` 时回到 `IDLE_F1`）+ `curr_floor` 更新逻辑；
 - 进程 2（组合）：次态逻辑——根据当前状态、`at_floor`、`f1/f2/f3` 决定次态。必须使用 `CASE` 语句对当前状态进行分支，在每个分支内用 `IF` 语句判断输入条件；
 - 进程 3（组合）：输出逻辑——根据当前状态（仅取决于状态，不依赖于输入，Moore 型特征）产生 `motor_up`、`motor_down`、`motor_stop` 信号。
- 代码中关键判断条件需用中文注释说明。
- 代码缩进规范、信号命名清晰。

(4) 分析与思考 (3 分)

- (a) (1.5 分) 在电梯控制器的场景下，Moore 型与 Mealy 型状态机哪种更适合？请从输出稳定性和电机控制安全性角度说明理由。
- (b) (1.5 分) 当前设计中假设楼层按钮信号（`f1`，`f2`，`f3`）持续保持高电平直到被服务。在实际硬件中，按钮按下通常只产生一个短暂的脉冲。请说明：需要增加哪些内部信号，以及如何修改状态转移逻辑，以支持脉冲型按钮输入（将脉冲锁存为请求记录）。

提示：

- 状态转移条件编写建议：在次态逻辑进程中，可定义辅助信号/变量表示“是否有任何请求”“是否有上方向请求”“是否有下方向请求”，例如：

- `any_req <= f1 OR f2 OR f3;`（并发语句，放在进程外部）

- 上升方向请求：当前楼层为 1 层时，`f2 OR f3 = '1'` 即有上升请求；当前楼层为 2 层时，`f3 = '1'` 即有上升请求。

- 输出编码示例：

- 停止/开门：`motor_up <= '0'; motor_down <= '0'; motor_stop <= '1';`

- 上升：`motor_up <= '1'; motor_down <= '0'; motor_stop <= '0';`

- 下降：`motor_up <= '0'; motor_down <= '1'; motor_stop <= '0';`

- 楼层更新：在时序进程中，当状态为 `MOVING_UP` 或 `MOVING_DOWN` 时，可用 `at_floor` 更新 `curr_floor`。

- 枚举类型声明示例：

```
TYPE elevator_state IS (IDLE_F1, IDLE_F2, IDLE_F3,  
MOVING_UP, MOVING_DOWN, STOPPED);
```

——试卷结束——

第二十章 第 20 套试卷

FPGA 与 VHDL 基础进阶试卷（二十）

（满分：100 分 考试时间：120 分钟 难度：中等）

考查范围：FPGA 开发流程、LUT 原理、VHDL 描述风格、Testbench 基础、
GENERATE 语句、属性、配置、断言以及组合/时序/状态机设计。

一、选择题（共 6 题，每题 3 分，共 18 分）

1. 下列关于 FPGA 开发流程（Design Flow）的描述，正确的是（ ）。
 - A. 综合(Synthesis)阶段将 HDL 代码转化为门级网表(Netlist)；布局布线(Place & Route)阶段将网表中的逻辑单元映射到 FPGA 物理资源并完成互连；比特流生成(Bitstream Generation)阶段产生 FPGA 配置文件
 - B. 综合和布局布线是同一个步骤的不同叫法，均由综合工具在后台自动并行完成
 - C. 比特流生成(Bitstream Generation)在综合之前执行，用于确定逻辑资源分配方案
 - D. 布局布线(Place & Route)仅涉及 I/O 引脚分配，与内部逻辑单元的位置无关
2. 关于 FPGA 中查找表(LUT)实现组合逻辑的原理，以下描述正确的是（ ）。
 - A. k 输入 LUT 本质上是一个 $2^k \times 1$ 位的 SRAM，通过将真值表存入 SRAM 单元，地址端接输入信号，数据端输出函数值，从而可实现任意 k 输入布尔函数
 - B. 一个 4 输入 LUT 只能实现至多 4 种不同的逻辑函数，无法覆盖所有 16 种 2 输入布尔函数
 - C. LUT 的输出延迟与存入真值表的复杂度成正比，逻辑函数越复杂延迟越大
 - D. LUT 只能实现组合逻辑中的与、或、非三种基本操作，更复杂的逻辑需配合触发器实现

3. VHDL 中采用结构化描述 (Structural) 与行为描述 (Behavioral) 各自的特点, 以下说法正确的是 ()。

- A. 结构化描述使用 COMPONENT 声明、PORT MAP 例化来连接子模块, 类似绘制电路原理图, 适合描述层次化设计的拓扑结构; 行为描述使用 PROCESS、CASE、IF 等语句描述电路功能, 更抽象且接近算法级思维
- B. 行为描述方式综合出的电路总是比结构化描述占用更多逻辑资源
- C. 结构化描述中不允许使用 PROCESS 语句, 行为描述中不允许使用 COMPONENT 例化
- D. 两种描述风格在同一结构体 (ARCHITECTURE) 中不能混合使用

4. 关于 VHDL 测试平台 (Testbench) 的基本概念, 以下描述错误的是 ()。

- A. Testbench 是一个顶层无端口实体 (ENTITY 中无 PORT 声明), 其作用是为待测设计 (UUT/DUT) 生成输入激励, 并监测/验证输出响应
- B. Testbench 内部通过 COMPONENT 声明和 PORT MAP 例化连接 UUT, 激励信号在 PROCESS 中使用 WAIT FOR 或 AFTER 语句生成
- C. Testbench 中包含的 WAIT、ASSERT、REPORT 等语句均为可综合语句, 可被 FPGA 综合工具正常处理并下载到芯片中运行
- D. 在 Testbench 中使用 ASSERT 语句可以自动检查 UUT 输出是否与预期一致, 并在仿真控制台输出报错信息

5. 在 VHDL 中, FOR-GENERATE 与 IF-GENERATE 是两种并发生成语句, 以下关于它们区别的描述正确的是 ()。

- A. FOR-GENERATE 用于按固定次数重复生成一组并发语句（如重复实例化多个相同组件）；IF-GENERATE 用于根据条件选择性地生成某些并发语句（条件不成立时对应硬件不被生成）
 - B. FOR-GENERATE 只能在 PROCESS 内部作为顺序循环使用，IF-GENERATE 只能在结构体并发区使用
 - C. FOR-GENERATE 的循环变量必须在信号声明区显式声明（如 SIGNAL i : INTEGER），否则编译报错
 - D. IF-GENERATE 语句必须有 ELSE 分支（类似于 IF 语句中的 ELSE），否则该语句不被允许
6. 在 VHDL 中，关于属性（ATTRIBUTE）的用途和语法，以下描述正确的是（ ）。
- A. ATTRIBUTE（属性）用于将额外的元数据信息附加到 VHDL 对象（如信号、实体、结构体、类型等）上，可用于驱动综合工具行为（如指定状态编码方式、引脚位置等），ATTRIBUTE 声明使用关键字 ATTRIBUTE，属性指定使用运算符 '（撇号）
 - B. VHDL 仅支持预定义的固定属性（如 'EVENT、'RANGE 等），不允许用户自定义属性
 - C. ATTRIBUTE 指定的内容直接影响电路的逻辑功能，修改属性值会改变综合出的电路逻辑行为（真值表）
 - D. ATTRIBUTE 只能在 ARCHITECTURE 的声明区使用，不能在 ENTITY 或 PACKAGE 中使用

二、填空题（共 4 题，每题 3 分，共 12 分）

1. CONFIGURATION（配置）声明：

- 在 VHDL 中，CONFIGURATION 语句的基本语法结构如下（填写空白处的关键字）：

```
CONFIGURATION 配置名 OF 实体名 IS
    FOR 结构体名                -- 指定默认结构体
        [FOR 例化标号 : 元件名
            USE ENTITY 库名.实体名(结构体名); -- 为特定实例绑定具体实现
        END FOR;]
    END FOR;
END [CONFIGURATION] [配置名];
```

- 在上面语法中，关键字_____用于引入配置声明；内部_____子句用于将特定的 ENTITY-ARCHITECTURE 对绑定到某个元件例化标签（Label）上。
- 一个实体拥有多个不同结构体实现时，可以通过_____语句选择绑定哪一个，而不必修改顶层设计代码。

2. FOR-GENERATE 语句的基本语法：

- FOR-GENERATE 是一种_____（填“并发”或“顺序”）语句，必须放置在 ARCHITECTURE 的并发语句区域（BEGIN 之后），不能出现在 PROCESS 内部。
- 其基本语法格式为（填写空白关键字）：

```
标号：FOR 循环变量 IN 范围 GENERATE
    [并发语句组]
END GENERATE [标号];
```

- FOR-GENERATE 中的循环变量（如 i）由工具_____（填“隐式”或“显式”）声明，不需要在信号的声明区提前定义；循环变量的作用域仅限于该 GENERATE 语句内部。

3. Testbench 中的元件例化与激励生成:

- 在 Testbench 中例化待测设计 (UUT) 时, 通常使用_____ 关键字声明 UUT 的接口模板, 再通过_____ 将外层信号连接到 UUT 端口上。
- 激励(Stimulus)信号通常在单独的_____ 语句块中生成, 使用_____ 语句 (填 “WAIT FOR” 或 “AFTER”) 来控制时序间隔。
- Testbench 实体声明中 PORT 列表应为_____ (填 “显式列出所有端口” 或 “空”), 因为 Testbench 是顶层仿真环境, 不需要对外接口。

4. ASSERT 与 REPORT 语句:

- 在 VHDL 中, ASSERT 语句用于检查某个条件是否成立。其基本语法为:

```
ASSERT 条件表达式  
  REPORT "消息字符串"  
  SEVERITY 严重级别;
```

- 当条件表达式为_____ (填 TRUE 或 FALSE) 时, REPORT 后指定的消息字符串被输出到仿真控制台。
- SEVERITY (严重级别) 有 4 个预定义级别: NOTE (提示)、_____ (警告)、_____ (错误) 和 FAILURE (致命失败)。其中级别为 FAILURE 时, 仿真器通常会_____ (填 “停止仿真” 或 “忽略该断言”)。

三、程序改错题（共 5 分）

题目：以下 VHDL 代码意图使用 FOR-GENERATE 语句构建一个 8 位行波进位加法器（Ripple Carry Adder），由 8 个全加器（Full Adder）元件例化级联而成，并通过配置（CONFIGURATION）指定全加器的具体实现。代码中存在至少 7 处错误（涵盖 GENERATE 语法、端口位宽/类型不匹配、COMPONENT 声明丢失、配置语法错误等）。请找出任意 5 处（每处 1 分，共 5 分），指出错误位置、原因及正确写法。

Listing 20.1: 含错误的 8 位加法器 VHDL 代码（7 处错误，找出 5 处即满分）

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY ripple_adder8 IS
5      PORT(
6          a, b      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
7          cin       : IN  STD_LOGIC;
8          sum       : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
9          cout      : OUT STD_LOGIC
10     );
11 END ENTITY ripple_adder8;
12
13 ARCHITECTURE structural OF ripple_adder8 IS
14     -- 内部进位信号
15     SIGNAL carry : STD_LOGIC_VECTOR(8 DOWNTO 0);
16 BEGIN
17     carry(0) <= cin;
18
19     -- (错误 1-3 所在区域) FOR-GENERATE 实例化 8 个全加器
20     gen_fa: FOR i IN 0 TO 7 GENERATE
21         fa_inst: full_adder PORT MAP(      -- 错误 1: COMPONENT 未声明
22             a      => a(i),                -- 错误 2: a 为 STD_LOGIC_VECTOR,
23             a(i)    -- 实际返回 STD_LOGIC——此处正确但可能被误认为错误
24             b      => b(i),
25             cin     => carry(i),
26             sum     => sum(i),
27             cout    => carry(i+1)
28         );
29     END GENERATE;      -- 错误 3: 缺少 GENERATE 标号 (或多
30                         了不该有的部分)

```

```
29
30     cout <= carry(8);
31 END ARCHITECTURE structural;
32
33 -- (错误4-5所在区域) CONFIGURATION配置
34 CONFIGURATION rca_config OF ripple_adder8 IS
35     FOR structural                                -- 错误4: 应为完整的"FOR 结构体名
        "语法
36     FOR gen_fa: full_adder                        -- FOR-GENERATE中的组件使用特殊
        FOR语法
37     FOR ALL: full_adder                          -- 错误5: 配置语法错误——应使用"
        FOR OTHERS"或"FOR ALL"
38         USE ENTITY work.full_adder(behav);
39     END FOR;
40 END FOR;
41 END FOR;
42 END CONFIGURATION rca_config;
43
44 -- 以下为全加器实体声明 (位于同一文件中)
45 ENTITY full_adder IS
46     PORT(
47         a, b, cin : IN  STD_LOGIC;
48         sum, cout : OUT STD_LOGIC
49     );
50 END ENTITY full_adder;
51
52 ARCHITECTURE behav OF full_adder IS
53 BEGIN
54     -- (错误6) 组合逻辑描述
55     PROCESS(a, b)
56     BEGIN
57         sum  <= a XOR b XOR cin;                -- 错误6: 敏感信号表不完整, cin
            未列入但被读取
58         cout <= (a AND b) OR (a AND cin) OR (b AND cin);
59     END PROCESS;
60 END ARCHITECTURE behav;
```

答题要求：按以下格式逐条列出 5 处错误（每处 1 分，共 5 分）。代码中共有至少 7 处错误，请选择 5 处作答。

错误 1（第 ____ 行）：_____ 原因：_____

错误 2（第 ____ 行）：_____ 原因：_____

错误 3（第 ____ 行）：_____ 原因：_____

错误 4（第 ____ 行）：_____ 原因：_____

错误 5（第 ____ 行）：_____ 原因：_____

错误类型提示：涉及 (1) COMPONENT 声明缺失；(2) FOR-GENERATE 中 END GENERATE 标号语法；(3) CONFIGURATION 的 FOR 结构体语法不完整；(4) 组合进程敏感信号表遗漏；(5) IF-GENERATE 被误用为 FOR-GENERATE（条件生成与循环生成的混淆）；(6) ENTITY 声明应放在 ARCHITECTURE 之前（代码顺序问题）；(7) 配置中对 GENERATE 实例的引用语法错误。

四、程序阅读分析题（共 5 分）

题目：阅读以下 VHDL 代码——这是一个算术逻辑单元（ALU）的描述，回答下列问题。

Listing 20.2: ALU 模块 VHDL 代码——待分析

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY alu IS
6      GENERIC(
7          DATA_WIDTH : POSITIVE := 8
8      );
9      PORT(
10         a, b      : IN  STD_LOGIC_VECTOR(DATA_WIDTH-1 DOWNT0 0);
11         op_sel    : IN  STD_LOGIC_VECTOR(2 DOWNT0 0);
12         result    : OUT STD_LOGIC_VECTOR(DATA_WIDTH-1 DOWNT0 0);
13         zero      : OUT STD_LOGIC;
14         sign      : OUT STD_LOGIC;
15         carry     : OUT STD_LOGIC
16     );
17 END ENTITY alu;
18
19 ARCHITECTURE behav OF alu IS
20     SIGNAL a_int, b_int, res_int : SIGNED(DATA_WIDTH DOWNT0 0);
21     SIGNAL res_std : STD_LOGIC_VECTOR(DATA_WIDTH-1 DOWNT0 0);
22 BEGIN
23     -- 将输入扩展1位以捕捉进位
24     a_int <= SIGNED('0' & a);
25     b_int <= SIGNED('0' & b);
26
27     PROCESS(a, b, op_sel)
28     BEGIN
29         carry <= '0';    -- 默认值
30         CASE op_sel IS
31             WHEN "000" =>
32                 res_int <= a_int + b_int;
33                 carry <= res_int(DATA_WIDTH);

```

```
34     WHEN "001" =>
35         res_int <= a_int - b_int;
36         carry <= res_int(DATA_WIDTH);
37     WHEN "010" =>
38         res_int <= '0' & SIGNED(a AND b);
39     WHEN "011" =>
40         res_int <= '0' & SIGNED(a OR b);
41     WHEN "100" =>
42         res_int <= '0' & SIGNED(a XOR b);
43     WHEN "101" =>
44         res_int <= '0' & SIGNED(NOT a);
45     WHEN "110" =>
46         res_int <= a_int;      -- 直通 a
47     WHEN "111" =>
48         res_int <= b_int;      -- 直通 b
49     WHEN OTHERS =>
50         res_int <= (OTHERS => '0');
51     END CASE;
52 END PROCESS;
53
54 res_std <= STD_LOGIC_VECTOR(res_int(DATA_WIDTH-1 DOWNT0 0));
55 result  <= res_std;
56
57 -- 标志位生成
58 zero   <= '1' WHEN res_std = (res_std'RANGE => '0') ELSE '0';
59 sign   <= res_std(DATA_WIDTH-1);
60 END ARCHITECTURE behav;
```

问题：

- (1) (1 分) 该 ALU 的数据位宽是多少？由什么参数控制？若需修改为 16 位 ALU，需要对代码做哪些修改？
- (2) (2 分) 根据代码，列出该 ALU 支持的**全部 8 种操作**（按 op_sel 值从"000"到"111"依次说明），并指出每种操作的助记符名称（如 ADD、SUB、AND 等）和功能。
- (3) (1 分) 该 ALU 内部为什么要将操作数 a 和 b 扩展为 DATA_WIDTH+1 位的

SIGNED 类型（第 22–23 行）？这种扩展对进位标志 `carry` 的检测有什么帮助？

- (4) (1 分) 当 `op_sel="001"`（减法操作），且 `a = "00001010"` (10)、`b = "00000101"` (5) 时，写出 `result` 和三个标志位 (`zero`, `sign`, `carry`) 的输出值。若 `a = "00000101"` (5)、`b = "00001010"` (10) 时，结果又如何？（提示：注意有符号数的借位情况）

五、综合设计题（共 3 题，共 60 分）

设计题 1：8 位行波进位加法器——结构化设计（20 分）

设计一个 8 位行波进位加法器（Ripple Carry Adder, RCA），要求采用**结构化描述风格**（Structural VHDL），通过 **FOR-GENERATE** 语句实例化 8 个 1 位全加器（Full Adder）并级联构成完整的 8 位加法器。

设计说明：一个 N 位行波进位加法器由 N 个 1 位全加器级联组成。每个全加器有三个输入（被加数位 a_i 、加数位 b_i 、低位进位 c_i ）和两个输出（和位 s_i 、向高位的进位 c_{i+1} ）。 N 个全加器的进位链首尾相连： $c_0 = cin$ （外部最低进位）， $c_i \rightarrow c_{i+1}$ 依次传递， $c_N = cout$ （最高进位输出）。

1 位全加器的逻辑方程：

- $s_i = a_i \oplus b_i \oplus c_i$
- $c_{i+1} = (a_i \wedge b_i) \vee (a_i \wedge c_i) \vee (b_i \wedge c_i)$

端口定义：

- `a(7 downto 0)`（输入）：8 位被加数。
- `b(7 downto 0)`（输入）：8 位加数。
- `cin`（输入）：最低位进位输入。
- `sum(7 downto 0)`（输出）：8 位和。
- `cout`（输出）：最高位进位输出（可用于检测溢出）。

要求：

- (1)（3 分）画出 8 位行波进位加法器的结构框图，标注：8 个全加器（FA0~FA7）的级联方式、进位链信号（ $c_0 \sim c_8$ ）的连接关系，以及所有外部端口。
- (2)（3 分）编写 1 位全加器 `full_adder` 的完整 VHDL 代码（实体 + 结构体）。要求：
 - 实体中声明正确的端口（`a`, `b`, `cin` 输入；`sum`, `cout` 输出，均为 `STD_LOGIC` 类型）；
 - 结构体采用**数据流描述风格**（使用并发信号赋值语句，直接以布尔表达式描述 s_i 和 c_{i+1} ）；

- 也可采用**行为描述**（PROCESS 敏感所有输入），但需在注释中标注选择的描述风格。

(3) (10 分) 编写顶层模块 `ripple_adder8` 的完整 VHDL 代码。要求：

- 实体中正确声明 8 位加法器的所有端口；
- 在结构体声明区使用 COMPONENT 声明 `full_adder`；
- 声明内部进位信号 `carry(8 DOWNTO 0)` (`carry(0)` 接 `cin`, `carry(8)` 接 `cout`)；
- 使用 **FOR-GENERATE** (`FOR i IN 0 TO 7 GENERATE`) 循环实例化 8 个全加器，通过 PORT MAP 将每个全加器连接到对应的输入位和进位链；
- `sum` 和 `cout` 通过并发信号赋值连接到正确位置；
- 代码中添加清晰的中文注释。

(4) (2 分) 在代码注释中说明：FOR-GENERATE 在此设计中的优势（相比逐一写出 8 个例化语句），以及为什么 FOR-GENERATE 不能放在 PROCESS 内部。

(5) (2 分) 分析：若将 8 位加法器扩展为 32 位加法器 (`a(31 downto 0)`、`b(31 downto 0)`)，代码的哪些地方需要修改？GENERIC 参数化在此场景中有何优势？

提示：

- COMPONENT 声明示例：

```
COMPONENT full_adder IS
  PORT( a, b, cin : IN  STD_LOGIC;
        sum, cout : OUT STD_LOGIC );
END COMPONENT;
```

- FOR-GENERATE 中的例化格式：

```
gen_fa: FOR i IN 0 TO 7 GENERATE
  fa_i: full_adder PORT MAP(
    a    => a(i),
    b    => b(i),
```

```
        cin => carry(i),  
        sum => sum(i),  
        cout => carry(i+1)  
    );  
END GENERATE gen_fa;
```

- 注意：FOR-GENERATE 中的循环变量 `i` 自动隐式声明，无需在信号声明区定义。这在纯组合逻辑端口映射中完全合法。
- 内部进位信号 `carry` 的声明：`SIGNAL carry : STD_LOGIC_VECTOR(8 DOWNT0 0);`

设计题 2：4 位 Johnson 计数器（扭环形计数器）设计（20 分）

设计一个 4 位 Johnson 计数器（Johnson Counter，又称扭环形计数器），包含自启动逻辑（非法状态自动恢复），确保在任何初始状态下均能在有限个时钟周期内进入有效的 8 状态循环。

Johnson 计数器原理： N 级 Johnson 计数器由 N 个 D 触发器首尾相接构成，最后一个触发器的反相输出 ($\overline{Q}$) 反馈到第一个触发器的 D 输入端。 N 级 Johnson 计数器共有 $2N$ 个有效状态。以 $N = 4$ 为例，8 个有效状态为：

状态序号	Q_3	Q_2	Q_1	Q_0
S0	0	0	0	0
S1	1	0	0	0
S2	1	1	0	0
S3	1	1	1	0
S4	1	1	1	1
S5	0	1	1	1
S6	0	0	1	1
S7	0	0	0	1

状态循环：S0 → S1 → S2 → S3 → S4 → S5 → S6 → S7 → S0 → …

自启动问题： 4 位二进制共有 16 种可能状态，但 Johnson 计数器仅有 8 个有效状态。其余 8 个状态为非法状态（如“1010”、“0101”等），若因干扰或上电初始值恰好进入非法状态，普通 Johnson 计数器将永远在非法状态子循环中运行，无法回到有效循环。因此需要设计**自启动逻辑**，检测非法状态并在下一个时钟沿自动将计数值纠正至有效状态（通常选择复位至 S0=“0000”或最近的有效状态）。

端口定义：

- **clk**（输入）：时钟信号，上升沿触发。
- **rst**（输入）：异步复位，高电平有效；复位后输出 $q = "0000"$ （状态 S0）。
- **en**（输入）：计数器使能，高电平有效。**en = '1'** 时每个时钟上升沿状态转移一次；**en = '0'** 时保持当前状态。
- **q(3 downto 0)**（输出）：4 位 Johnson 计数器当前状态值。

要求：

- (1)（3 分）画出 $N = 4$ 时 Johnson 计数器的完整状态转移图，包括：

- 标注全部 8 个有效状态 ($S_0 \sim S_7$) 以及它们之间的转移关系 (单向循环);
- 列出其余 8 个非法状态, 标注各自的自启动恢复路径 (即非法状态如何在一个时钟周期后转入有效状态或 S_0);
- 在图上标注反馈逻辑: $D_0 = \overline{Q_3}$ 。

(2) (2 分) 分析非法状态集合 (共 8 个状态), 设计自启动逻辑。请写出:

- (a) 列出全部 8 个非法状态的二进制值;
- (b) 说明你选择的自启动策略 (例如: 检测 Q 值不属于有效状态集合时, 次态强制为 S_0 ; 或利用特定的逻辑条件判断);
- (c) 写出该自启动逻辑的布尔表达式或判断条件 (可以用伪代码或 VHDL 条件表达式表示)。

(3) (12 分) 编写完整的 VHDL 代码 (实体 + 结构体)。要求:

- 实体中正确声明所有端口;
- 结构体采用行为描述风格 (PROCESS), 内部使用一个 `STD_LOGIC_VECTOR(3 DOWNTO 0)` 信号 `q_reg` 作为状态寄存器;
- 异步复位 (`rst='1'`) 时 `q_reg <= "0000"`;
- 在时钟上升沿 (`rst='0'` 且 `clk'EVENT AND clk='1'` 或 `rising_edge(clk)`):
 - 若 `en='1'`, 执行状态转移逻辑;
 - 若当前状态属于 8 个有效状态之一, 按 Johnson 计数器正常逻辑转移:
`q_reg <= q_reg(2 DOWNTO 0) & (NOT q_reg(3));`
 - 若当前状态为非法状态 (自启动逻辑), 强制下一个状态为 "0000" (或其他有效状态, 需在注释中说明理由);
- `en='0'` 时 `q_reg` 保持不变;
- 输出 `q <= q_reg`;
- 代码中包含详细的中文注释, 解释自启动逻辑的设计思路。

(4) (3 分) 回答以下问题:

- (a) Johnson 计数器与普通环形计数器 (Ring Counter, 即 Q_N 直接反馈到 D_0 而非反相) 的主要区别是什么? 两者的有效状态数分别为多少 (N 级情况下)?
- (b) 如果 $N = 8$ (8 级 Johnson 计数器), 有效状态数为多少? 此时非法状态有多少个? 自启动逻辑的复杂度如何变化?
- (c) 在本设计中, 为何推荐在非法状态时将输出强制复位至 "0000" 而不是其他有效状态? 从电路可靠性角度简要说明。

提示:

- 有效状态判断可通过列出全部 8 个有效状态的二进制编码, 使用组合逻辑进行比较 (例如: IF q_reg = "0000" OR q_reg = "1000" OR ... THEN)。也可使用更简洁的表达式。
- 自启动逻辑的典型写法框架:

```
PROCESS(clk, rst)
BEGIN
    IF rst = '1' THEN
        q_reg <= "0000";
    ELSIF rising_edge(clk) THEN
        IF en = '1' THEN
            -- 判断是否为合法状态
            CASE q_reg IS
                WHEN "0000"|"1000"|"1100"|"1110"|
                    "1111"|"0111"|"0011"|"0001" =>
                    q_reg <= q_reg(2 DOWNTO 0) & (NOT q_reg(3));
                WHEN OTHERS =>
                    q_reg <= "0000"; -- 非法状态→复位至S0
            END CASE;
        END IF;
    END IF;
END PROCESS;
```

- Johnson 计数器的反馈逻辑 $q_reg(2 \text{ DOWNTO } 0) \& (\text{NOT } q_reg(3))$ 等价于: 将所有位右移一位, 新的最高位由原来最高位的反相值填充。

设计题 3：8 位波形发生器——Moore 型状态机设计（20 分）

设计一个波形发生器（Waveform Generator），能够以串行方式循环输出一个预定义的 8 位波形模式"10011001"。每个时钟周期在 1 位输出端口 `wave_out` 上输出模式中的 1 位。该系统使用 Moore 型有限状态机（FSM）实现，通过状态序列控制输出。

功能描述：

- 待输出的 8 位模式为："10011001"（bit7=1, bit6=0, bit5=0, bit4=1, bit3=1, bit2=0, bit1=0, bit0=1）。输出顺序为：先输出 bit7（最高位），依次至 bit0（最低位），然后循环回到 bit7。
- 每个时钟周期输出 1 位，经过 8 个时钟周期完成一个完整模式，然后自动循环重复。
- 系统使用 Moore 型状态机控制：共有 8 个状态 S0~S7，每个状态对应固定的 1 位输出值——S0 对应输出 bit₇，S1 对应 bit₆，...，S7 对应 bit₀。
- Moore 型状态机的输出仅取决于当前状态，与外部输入无关。

状态与输出映射（Moore 型）：

状态	含义	输出 <code>wave_out</code>
S0	输出位 7（MSB）	'1'
S1	输出位 6	'0'
S2	输出位 5	'0'
S3	输出位 4	'1'
S4	输出位 3	'1'
S5	输出位 2	'0'
S6	输出位 1	'0'
S7	输出位 0（LSB）	'1'

状态转移：S0 → S1 → S2 → S3 → S4 → S5 → S6 → S7 → S0 → ...（无条件循环转移，每个时钟沿转移一次）。

端口定义：

- `clk`（输入）：系统时钟，上升沿触发。
- `rst`（输入）：异步复位，高电平有效。复位后状态回到 S0，输出 `wave_out` = '1'。
- `en`（输入）：使能信号，高电平有效。`en` = '1' 时每个时钟周期状态转移一次并更新输出；`en` = '0' 时状态和输出保持不变。

- `wave_out` (输出): 1 位波形输出信号。
- `state_disp(2 downto 0)` (输出, 可选): 当前状态的 3 位二进制编码, 用于调试显示 (`S0="000"`, `S1="001"`, ..., `S7="111"`)。

要求:

(1) (4 分) 画出该波形发生器的 **Moore 型状态转移图**。要求:

- 标注全部 8 个状态 (`S0~S7`);
- 每个状态节点内标注该状态下的输出值 (`wave_out = ?`);
- 标注状态间的转移方向 (单向循环);
- 标注异步复位后进入的初始状态。

(2) (3 分) 列出该状态机的**状态转移表** (State Transition Table), 包括以下列: 当前状态、`en` 输入、次态、输出 (`wave_out`)。注意 Moore 型输出仅由当前状态决定, 与 `en` 无关。

(3) (10 分) 编写完整的 VHDL 代码 (实体 + 结构体)。要求:

- 使用用户自定义枚举类型定义 8 个状态: `TYPE wave_state IS (S0, S1, S2, S3, S4, S5, S6, S7);`
- 采用**双进程描述风格**:
 - 进程 1 (时序进程): 状态寄存器 + 异步复位 (`rst='1'` 回到 `S0`) + 时钟上升沿检测 + `en` 使能控制;
 - 进程 2 (组合进程): Moore 型输出逻辑——根据当前状态生成 `wave_out` 和 `state_disp`;
- 在输出进程中使用 CASE 语句 (或 WITH-SELECT 语句) 为每个状态赋值对应的 `wave_out`;
- `state_disp` 使用 3 位二进制编码表示 8 个状态 (`S0→"000"`, `S1→"001"`, ..., `S7→"111"`), 可使用 `STD_LOGIC_VECTOR` 类型或 `INTEGER` 类型信号配合类型转换;
- 代码中包含详细的中文注释。

(4) (3 分) 回答以下问题:

- (a) **(关键问题)** 这个设计本质上只是依次输出一个固定的 8 位模式，为什么说它是一个“状态机”？请从状态、状态转移、输出与状态的对应关系三个角度论证。（至少 3 句话）
- (b) 若改用 Mealy 型状态机实现相同功能（同样的 8 位模式输出），状态数和输出逻辑会有何不同？Mealy 型能减少状态数吗？为什么？
- (c) 如果希望输出模式不是固定的”10011001”，而是可由外部输入 `pattern(7 downto 0)` 动态配置，当前的 Moore 型设计需要做哪些修改？（仅文字说明，不要求写代码）

提示：

- 枚举类型定义与信号声明示例：

```
TYPE wave_state IS (S0, S1, S2, S3, S4, S5, S6, S7);  
SIGNAL cur_state, nxt_state : wave_state;
```

- 时序进程（进程 1）框架：

```
PROCESS(clk, rst)  
BEGIN  
    IF rst = '1' THEN  
        cur_state <= S0;  
    ELSIF rising_edge(clk) THEN  
        IF en = '1' THEN  
            cur_state <= nxt_state;  
        END IF;  
    END IF;  
END PROCESS;
```

- 组合进程（进程 2）中次态逻辑：

```
PROCESS(cur_state)  
BEGIN  
    CASE cur_state IS
```

```
        WHEN S0 => nxt_state <= S1;
        WHEN S1 => nxt_state <= S2;
        ...
        WHEN S7 => nxt_state <= S0;
    END CASE;
END PROCESS;
```

- 也可将次态逻辑合并到时序进程中，使用双进程（状态时序 + 输出组合）保持 Moore 型结构清晰。
- **思考：**为什么这个设计是状态机？即使没有外部条件判断，状态仍然按照既定规则转移，且输出是当前状态的函数——这是经典 Moore 型状态机的简化形式（无输入条件的状态转移）。

——试卷结束——

